Agenda

Pod Networking

Services

Service Types and Load Balancers

Ingress Resource

Container-Native Load Balancing

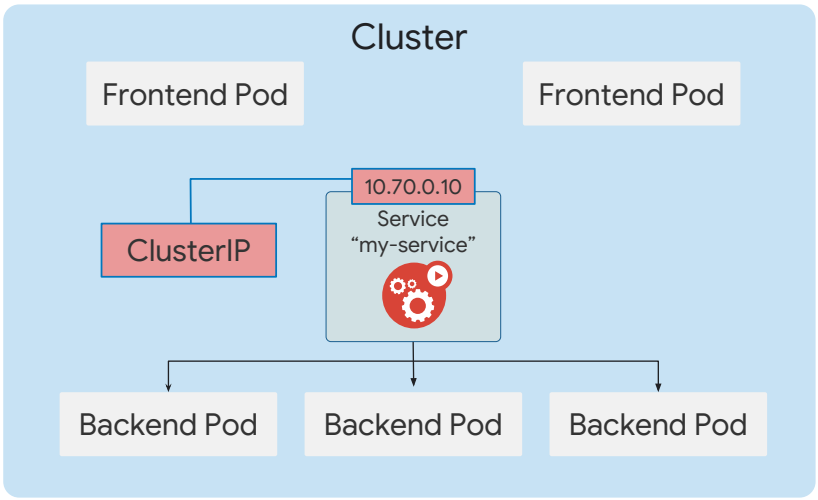
Network Security



In this lesson we'll identify the different types of Kubernetes Services as well as discuss Load Balancers.

There are three principal types of Services: ClusterIP, NodePort, and LoadBalancer. These Services build conceptually on one another, adding functionality with each step. We'll start with the basis of all the Kubernetes Services, the ClusterIP Service.

A ClusterIP Service has a static IP address



A Kubernetes ClusterIP Service has a static IP address and operates as a traffic distributor within the cluster, but ClusterIP Services aren’t accessible by resources outside the cluster.

Other Pods will use this ClusterIP as their destination IP address when communicating with the Service.

Setting up a ClusterIP Service

```
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  type: ClusterIP
  selector:
    app: Backend
  ports:
    - protocol: TCP
      port: 3306
      targetPort: 6000
```



Let's consider the step-by-step process of setting up a ClusterIP Service type. Here, you create a Service object by defining its kind. If you don't specify a Service type during the creation of the service, it will default to a Service type of ClusterIP.

Next, you use a label selector to select the Pods that will run the target application. In this case, the Pods with a label of "app:Backend" are selected and included as endpoints for this Service. You should always create a Service before creating any workloads that need to access that Service. If you create a Service before its corresponding backend workloads, such as Deployments or StatefulSets, the Pods that make up that Service get a nice bonus: they get the hostname and IP address of the Service in an environment variable.

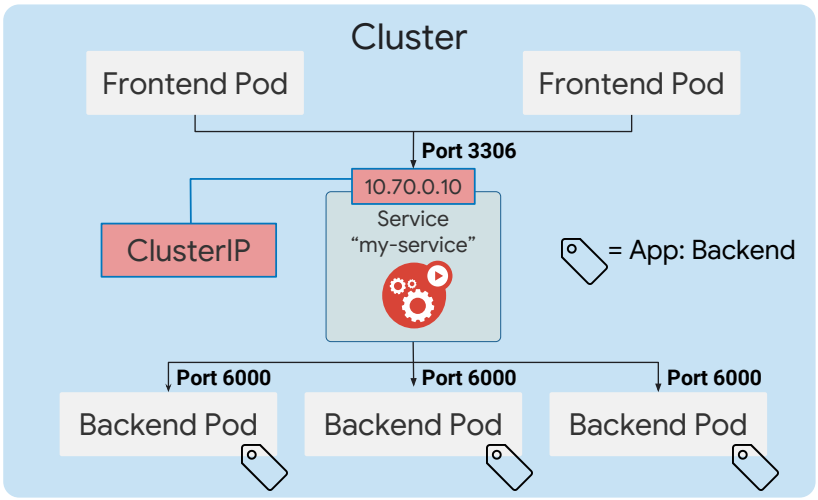
But remember that relying on environment variables for service

discovery is not as flexible as using DNS, so this practice isn't mandatory.

Next, you specify the port that the target containers are using, which in this case is TCP port 6000. This Service will receive traffic on port 3306 and then remap it to 6000 as it delivers it to the target Pods.

Creating, loading or applying this manifest will create the ClusterIP Service named "my-service". But how does this Service work?

How does the ClusterIP Service work?

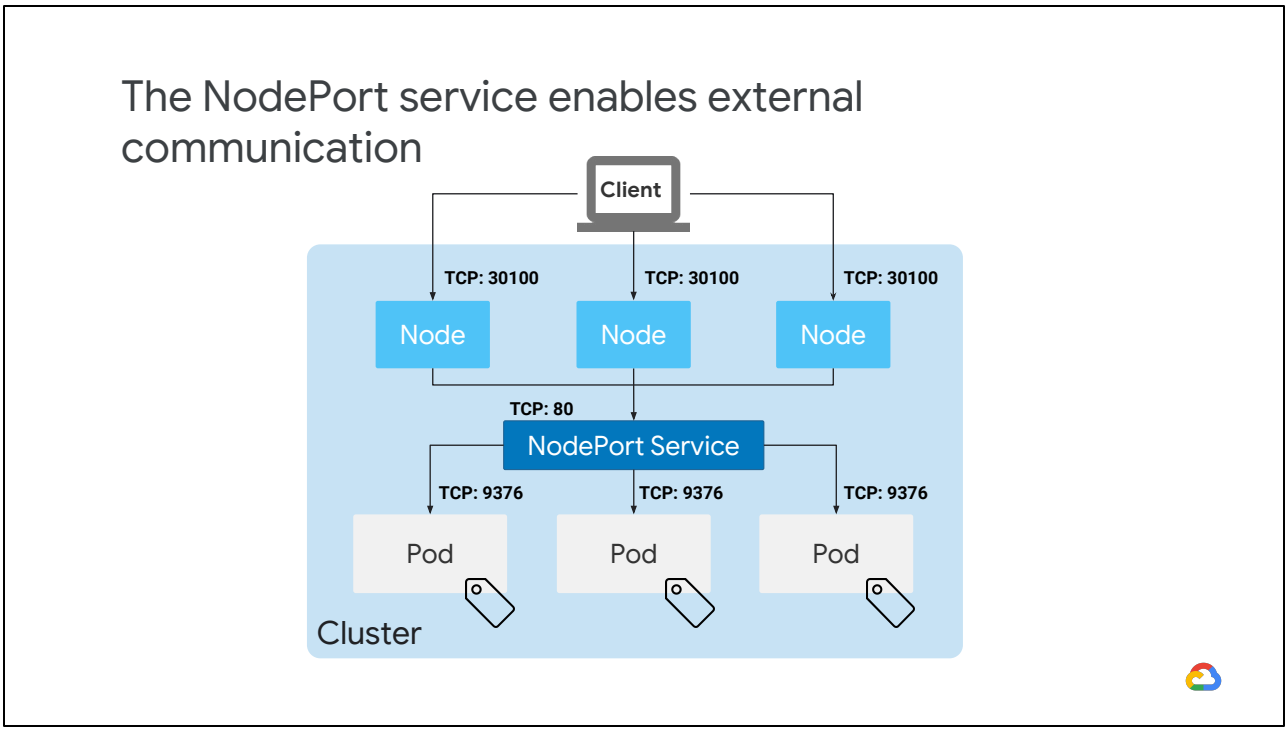


In this example, you have frontend Pods that must be able to locate the backend Pods.

When you create the Service, the cluster master assigns a virtual IP address—also known as ClusterIP—from a reserved pool of Alias IP addresses in the cluster's VPC. This IP address won't change throughout the lifespan of the Service.

The cluster master selects Pods to include in the Service's endpoints based on the label selector on the Service and the labels on the Pods.

The ip-addresses of these backend pods are mapped to the target port, TCP port 6000 in this example, to create the endpoint resources that the service forwards requests to. The ClusterIP Service will answer requests, on TCP port 3306 in this example, and forward the requests to the backend pods using their ip-addresses and target port as the endpoint resources.



ClusterIP is useful for internal communication within a cluster, but what about the external communication?
NodePort enables this. NodePort is built on top of ClusterIP Service; therefore, when you create a NodePort Service, a ClusterIP Service is automatically created in the process to distribute the inbound traffic internally across a set of pods.

In this example we now have a Service that can be reached from outside of the cluster using the IP address of any node and the corresponding NodePort number. Traffic through this port is directed to a Service on Port 80 in this example and further redirected to one of the Pods on port 9376. NodePort Service can be useful to expose a Service through an external load balancer that you set up and manage yourself. Using this approach, you would have to deal with node management, making sure there are no port collisions.

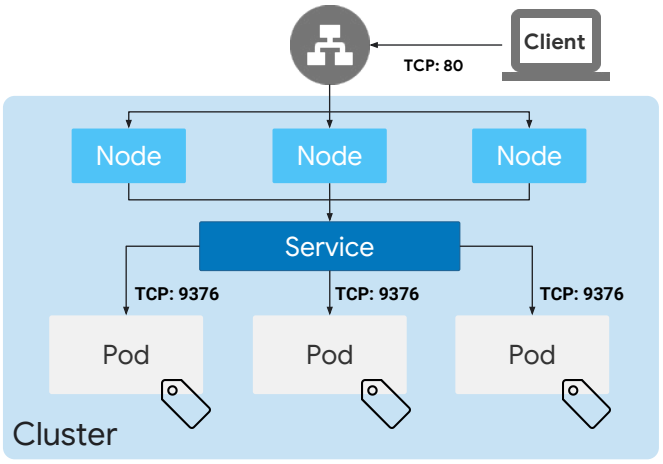
Creating a NodePort Service

```
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  type: NodePort
  selector:
    app: external
  ports:
    - protocol: TCP
      nodePort: 30100
      port: 80
      targetPort: 9376
```



In addition to the setup of the internal ClusterIP Service, a specific port is exposed for external traffic on every node. This port—also known as nodePort —is automatically allocated from the range 30,000 to 32767. In some cases users may want to manually specify it, which is allowed as long as the value also falls within that range, but this is not usually necessary.

The LoadBalancer Service can be used to expose a Service to resources outside the cluster



Conceptually, the LoadBalancer Service type builds on the ClusterIP Service and can be used to expose a Service to resources outside the cluster. With GKE, the LoadBalancer Service is implemented using GCP's network load balancer.

When you create a LoadBalancer Service, GKE automatically provisions a GCP network load balancer for inbound access to the Services from outside the cluster. Traffic will be directed to the IP address of the network load balancer, and the load balancer forwards the traffic on to the nodes for this Service.

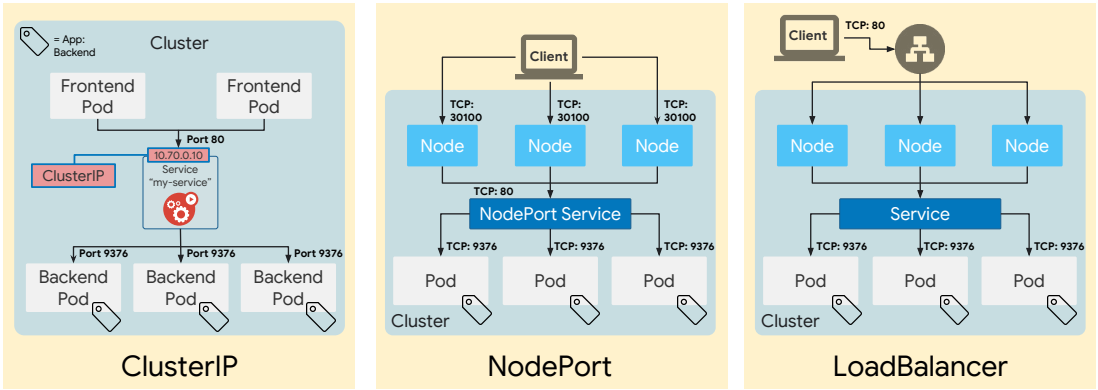
Creating a LoadBalancer Service

```
[...]
spec:
  type: LoadBalancer
  selector:
      app: external
    ports:
      - protocol: TCP
      port: 80
        targetPort: 9376
```



Here's how you create a LoadBalancer Service. You only need to specify the type: LoadBalancer. GCP will assign a static load balancer IP address that is accessible from outside your project.

Service types summary



You've seen how ClusterIP Services can be used within the cluster to provide a stable endpoint that allows Pods to connect to other Pods without the risk of the IP address changing.

You learned how NodePort Services extend the ClusterIP Services concept to expose and then bind a port number on each of the nodes in the cluster to the Service. This allows you to access resources within the cluster from external resources such as GCP compute instances.

Finally, you saw how the LoadBalancer Service type implementation in GKE improved on the NodePort Service by using the cloud provider API to create a GCP network load balancer for traffic distribution across the nodes.

Now let's look at a way to group Services together using the Ingress resource.

Agenda

Deployments

Jobs and CronJobs

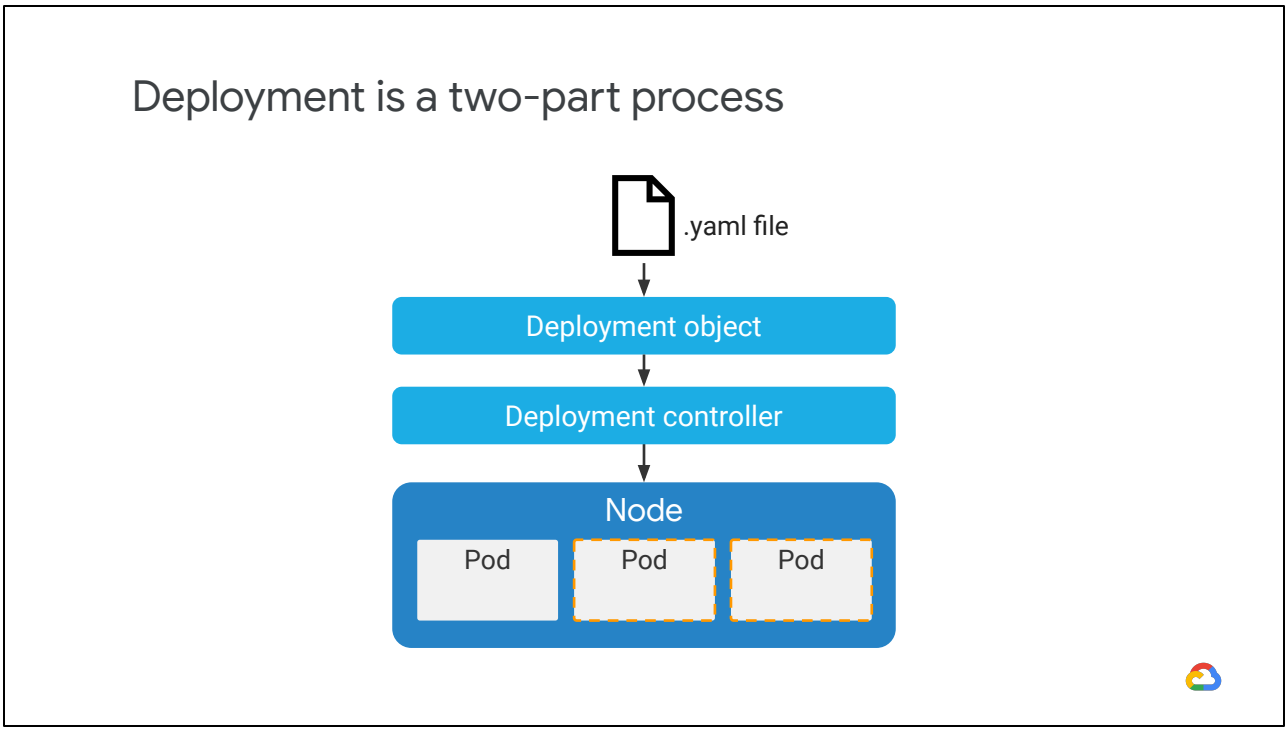
Cluster Scaling

Controlling Pod Placement

Getting Software into Your Cluster



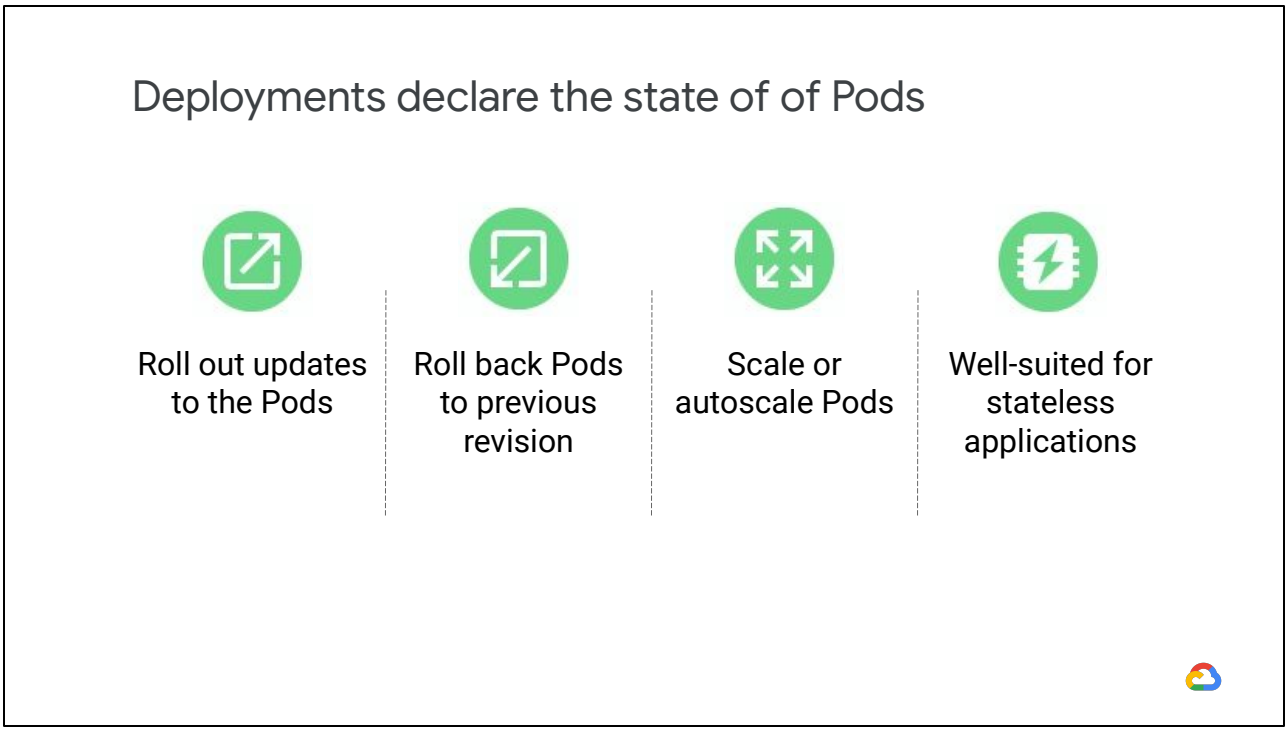
Deployments describe a desired state of Pods. For example, a desired state could be that you want to make sure that 5 nginx pods are running at all times. Its declarative stance means that Kubernetes will continuously make sure the configuration is running across your cluster. Kubernetes also supports various update mechanisms for Deployments, which I'll tell you about later in this module.



The desired state is described in a Deployment YAML file containing the characteristics of the pods, coupled with how to operationally run these pods and handle their lifecycle events. After you submit this file to the Kubernetes master, it creates a deployment controller, which is responsible for converting the desired state into reality and keeping that desired state over time. Remember what a controller is: it's a loop process created by Kubernetes that takes care of routine tasks to ensure the desired state of an object, or set of objects, running on the cluster matches the observed state.

During this process, a ReplicaSet is created. A ReplicaSet is a controller that ensures that a certain number of Pod replicas are running at any given time. The Deployment is a high level controller for a Pod that declares its state. The Deployment configures a ReplicaSet controller to instantiate and maintain a specific version

of the Pods specified in the Deployment.



Every time you update the specification of the pods, for example, updating them to use a newer container image, a new ReplicaSet is created that matches the altered version of the Deployment. This is how deployments roll out updated Pods in a controlled manner: old Pods from the old ReplicaSet are replaced with newer Pods in a new ReplicaSet.

If the updated Pods are not stable, the administrator can roll back the Pod to a previous Deployment revision.

You can scale Pods manually by modifying the Deployment configuration. You can also configure the Deployment to manage the workload automatically.

Deployments are designed for stateless applications. Stateless applications don’t store data or application state to a cluster or to

persistent storage. A typical example of a stateless application is a Web front end. Some backend owns the problem of making sure that data gets stored durably, and you'll use Kubernetes objects other than Deployments to manage these back ends.

Deployment object file in YAML format

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
spec:
  replicas: 3
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
      - name: my-app
        image: gcr.io/demo/my-app:1.0
        ports:
        - containerPort: 8080
```

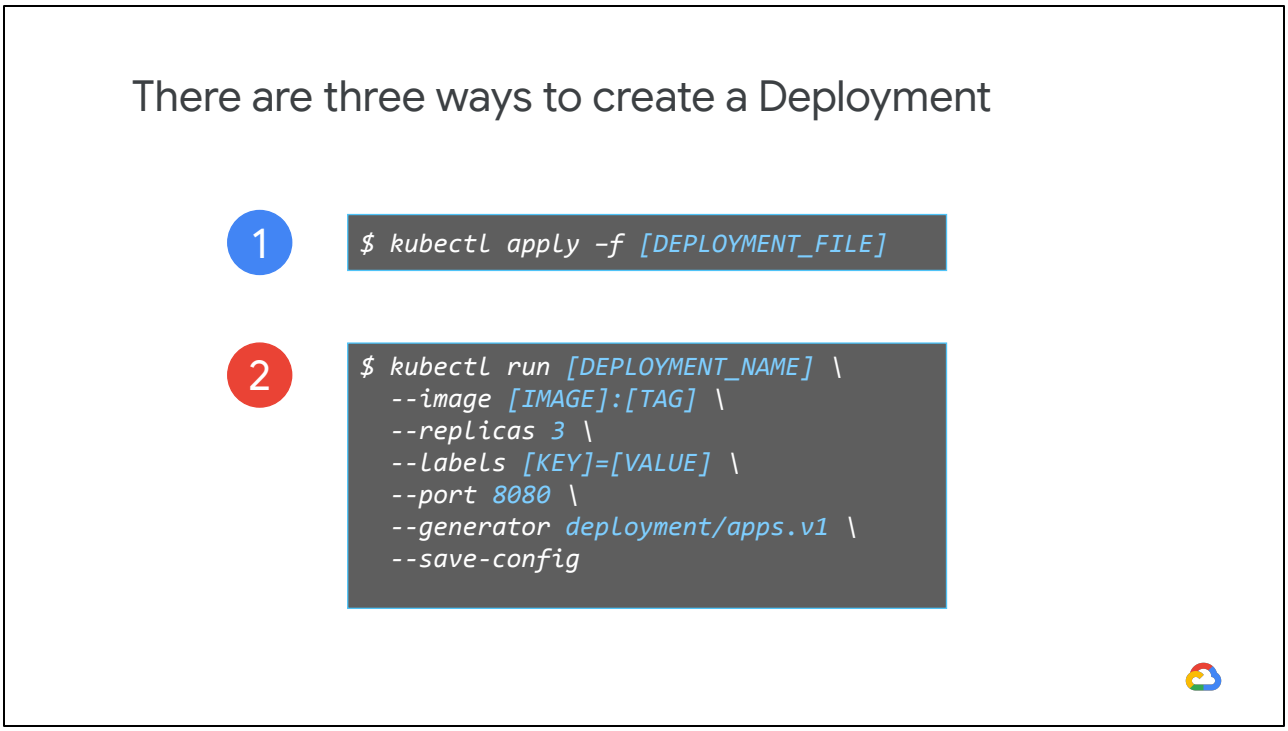


Here is a simple example of a Deployment object file in YAML format.

The Deployment named my-app is created with 3 replicated Pods.

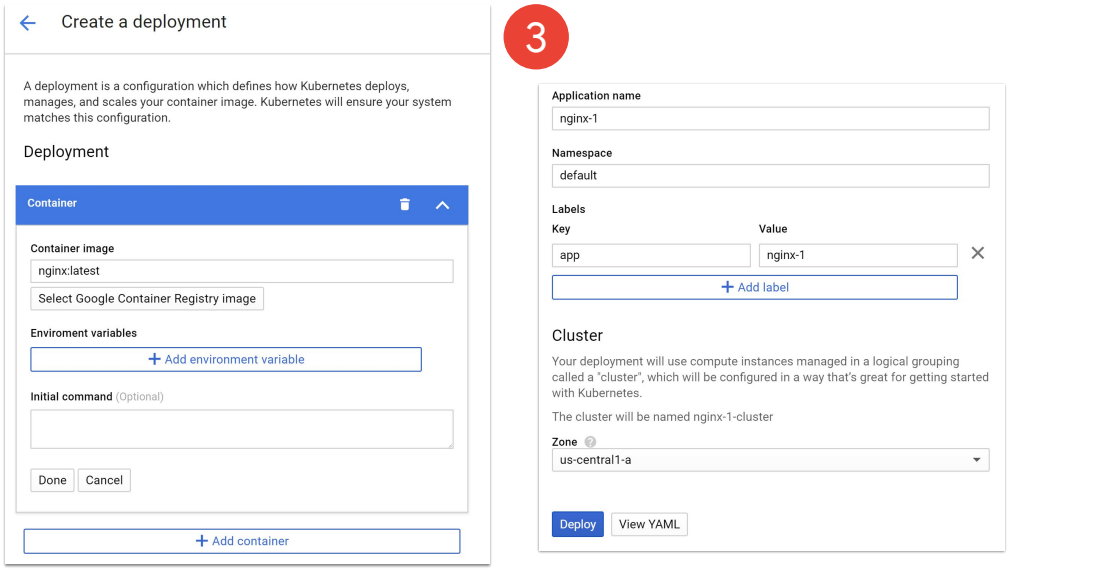
In the spec.template section, a Pod template defines the metadata and specification of each of the Pods in this ReplicaSet.

In the Pod specification, an image is pulled from Google Container Registry, and port 8080 is exposed to send and accept traffic for the container.



You can create a Deployment in three different ways. First, you create the Deployment declaratively using a manifest file, such as the YAML file you've just seen, and a kubectl apply command.

The second method creates a Deployment imperatively using a kubectl run command that specifies the parameters inline. Here, the image and tag specifies which image and image version to run in the container. This Deployment will launch 3 replicas and expose port 8080. Labels are defined using key and value. --generator specifies the API version to be used, and --save-config saves the configuration for future use.



Your third option is to use the GKE Workloads menu in the GCP Console. Here, you can specify the container image and version, or even select it directly from Container Registry. You can specify environment variables and initialization commands. You can also add an application name and namespace along with labels. You can use the View YAML button on the last page of the Deployment wizard to view the Deployment specification in YAML format.

Use kubectl to inspect your Deployment, or output the Deployment config in a YAML format

```
$ kubectl get deployment [DEPLOYMENT_NAME]
```

```
master $ kubectl get deployment nginx-deployment
NAME               DESIRED   CURRENT   UP-TO-DATE   AVAILABLE   AGE
nginx-deployment   3         3         3            3           3m
```

```
$ kubectl get deployment [DEPLOYMENT_NAME] -o yaml > this.yaml
```



The ReplicaSet created by the Deployment ensures that the desired number of Pods are running and always available at any given time. If a Pod fails or is evicted, the ReplicaSet automatically launches a new Pod. You can use the kubectl ‘get’ and ‘describe’ commands to inspect the state and details of the Deployment.

As shown here, you can get the desired, current, up-to-date, and available status of all the replicas within a Deployment, along with their ages, using the kubectl ‘get deployment’ command.

‘Desired’ shows the desired number of replicas in the Deployment specification.
‘Current’ is the number of replicas currently running.
‘Up-to-date’ shows the number of replicas that are fully up to date as per the current Deployment specification.
‘Available’ displays the number of replicas available to the users.

You can also output the Deployment configuration in a YAML format. Maybe you originally created a Deployment with kubectl run, and then you decide you'd like to make it a permanent, managed part of your infrastructure. Edit that YAML file to remove the unique details of the Deployment you created it from, and then you can add it to your repository of YAML files for future Deployments.

Use the ‘describe’ command to get detailed info

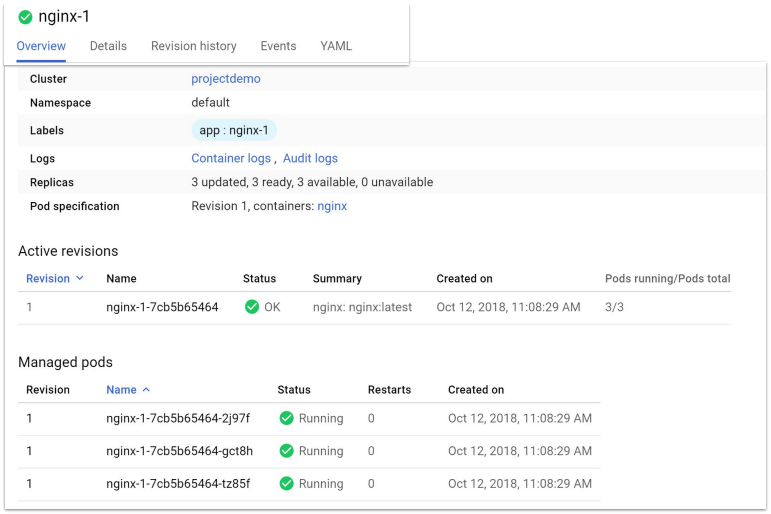
```
$ kubectl describe deployment [DEPLOYMENT_NAME]
```

```
master $ kubectl describe deployment nginx-deployment
Name:                   nginx-deployment
Namespace:              default
CreationTimestamp:      Fri, 12 Oct 2018 15:23:46 +0000
Labels:                 app=nginx
Annotations:            deployment.kubernetes.io/revision=1
Selector:               app=nginx
Replicas:               3 desired | 3 updated | 3 total | 3 available | 0 unavailable
StrategyType:           RollingUpdate
MinReadySeconds:        0
RollingUpdateStrategy:  25% max unavailable, 25% max surge
Pod Template:
  Labels:  app=nginx
  Containers:
   nginx:
    Image:        nginx:1.15.4
    Port:         80/TCP
    Host Port:    0/TCP
```



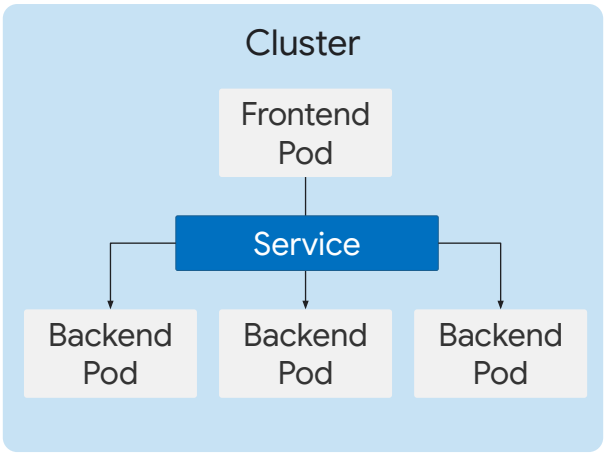
For more detailed information about the Deployment, use the kubectl ‘describe’ command. You’ll learn more about this command in the lab.

Or use the GCP Console



Another way to inspect a Deployment is to use the GCP Console. Here you can see detailed information about the Deployment, revision history, the Pods, events, and also view the live configuration in YAML format.

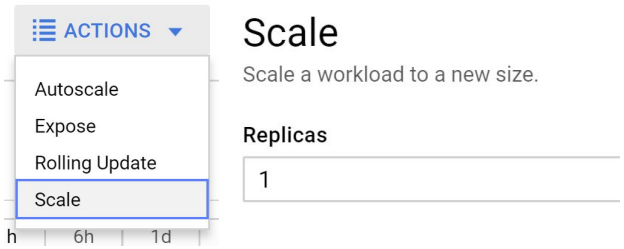
Service is a stable network representation of a set of pods



We haven't yet discussed how to locate and connect to the applications running in these Pods, especially as new Pods are created or updated by your Deployments. While you can connect to individual Pods directly, Pods themselves are transient. A Kubernetes Service is a static IP address that represents a Service, or a function, in your infrastructure. It's a stable network abstraction for a set of Pods that deliver that Service and, and it hides the ephemeral nature of the individual Pods. Services will be covered in detail in the Networking module.

You can scale the Deployment manually

```
$ kubectl scale deployment
[DEPLOYMENT_NAME] -replicas=5
```

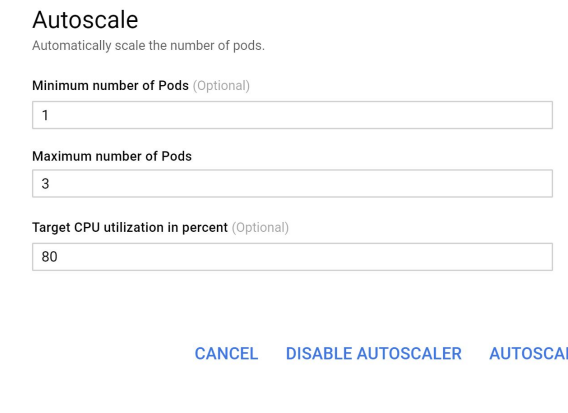




You now understand that a Deployment will maintain the desired number of replicas for an application. However, at some point you'll probably need to scale the Deployment. Maybe you need more web front end instances, for example. You can scale the Deployment manually using a kubectl command, or in the GCP Console by defining the total number of replicas. Also, manually changing the manifest will scale the Deployment.

You can also autoscale the Deployment

```
$ kubectl autoscale deployment [DEPLOYMENT_NAME] --min=5 --max=15
--cpu-percent=75
```





You can also autoscale the Deployment by specifying the minimum and maximum number of desired Pods along with a CPU utilization threshold. Again, you can perform autoscaling by using the kubectl autoscale command, or from the GCP Console directly. This leads to the creation of a Kubernetes object called HorizontalPodAutoscaler. This object performs the actual scaling to match the target CPU utilization. Keep in mind that we're not scaling the cluster as a whole, just a particular Deployment within that cluster. Later in this module you'll learn how to scale clusters.

One problem of any type of autoscaling is thrashing

Cooldown/delay support:

```
--horizontal-pod-autoscaler-downscale-delay
```



Thrashing sounds bad, and it is bad. It's a phenomenon where the number of deployed replicas frequently fluctuates, because the metric you used to control scaling also frequently fluctuates. The Horizontal Pod Autoscaler supports a cooldown, or delay, feature. It allows you to specify a wait period before performing another scale-down action. The default value is 5 minutes.

You can update a Deployment in different ways

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
spec:
  replicas: 3
  template:
    spec:
      containers:
      - name: my-app
        image: gcr.io/demo/my-app:1.0
        ports:
        - containerPort: 8080
```

```
$ kubectl apply -f [DEPLOYMENT_FILE]
```

```
$ kubectl set image deployment
[DEPLOYMENT_NAME] [IMAGE] [IMAGE]:[TAG]
```

```
$ kubectl edit \
   deployment/[DEPLOYMENT_NAME]
```



When you make a change to a Deployment's Pod specification, such as changing the image version, an automatic update rollout happens. Again, note that these automatic updates are only applicable to the changes in Pod specifications.

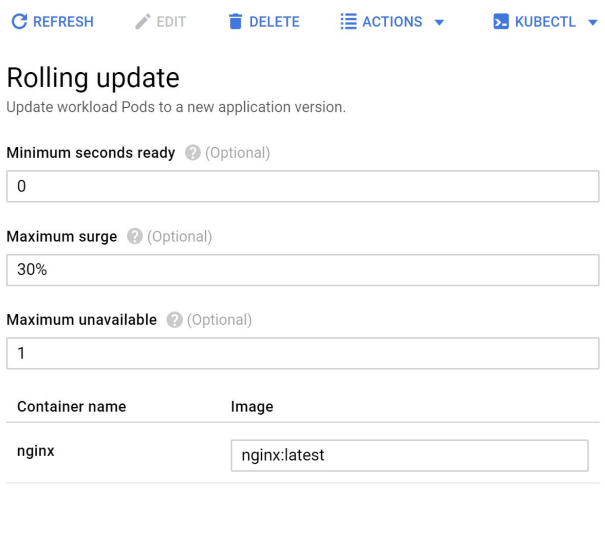
You can update a Deployment in different ways. One way is to use the kubectl 'apply' command with an updated Deployment specification YAML file. This method allows you to update other specifications of a Deployment, such as the number of replicas, outside the Pod template.

You can also use a kubectl 'set' command. This allows you to change the Pod template specifications for the Deployment, such as the image, resources, and selector values.

Another way is to use a kubectl 'edit' command. This opens the

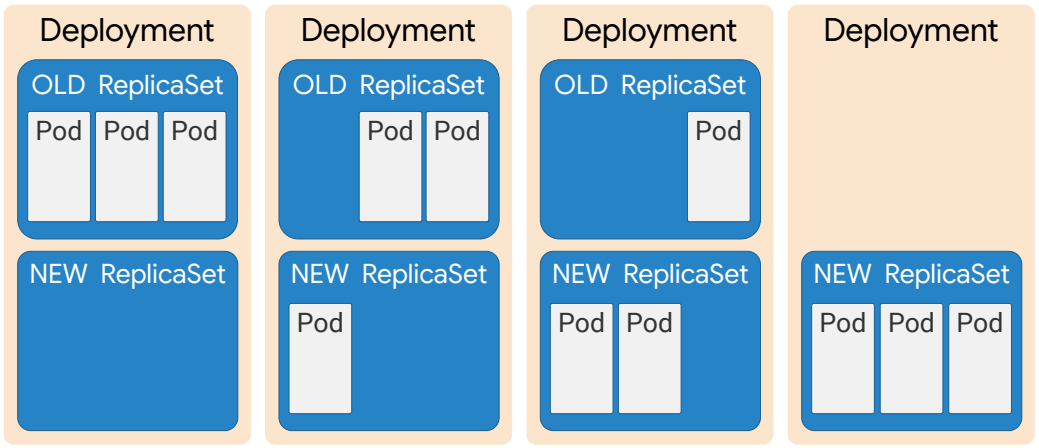
specification file using the vim editor that allows you to make changes directly. Once you exit and save the file, kubectl automatically applies the updated file.

You can update a Deployment in different ways



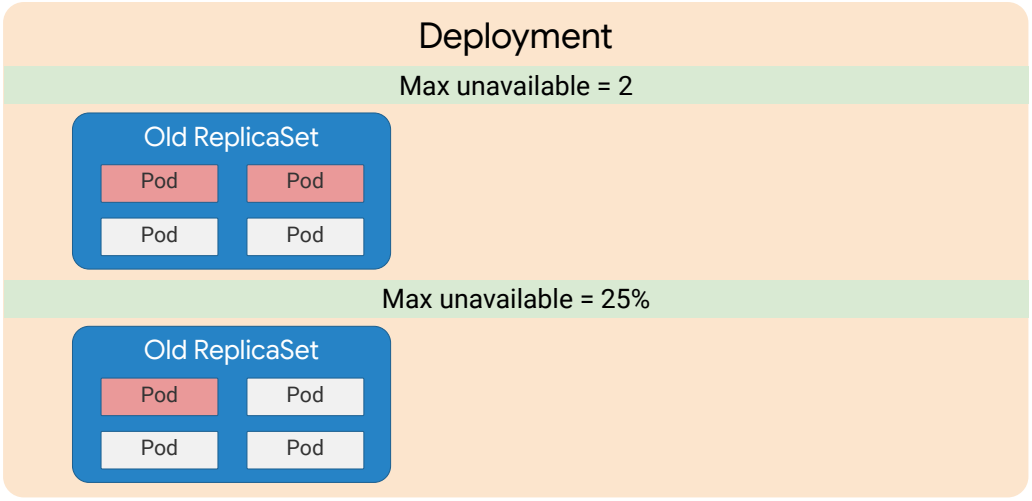
The last option for you to update a Deployment is through the GCP Console. You can edit the Deployment manifest from the GCP Console and perform a rolling update along with its additional options. Rolling updates are discussed next.

The process behind updating a Deployment



When a Deployment is updated, it launches a new ReplicaSet and creates a new set of Pods in a controlled fashion.
First, new Pods are launched in a new ReplicaSet.
Next, old Pods are deleted from the old ReplicaSet.
This is an example of a rolling update strategy also known as a ramped strategy.
Its advantage is that updates are slowly released, which ensures the availability of the application. However, this process can take time, and there's no control over how the traffic is directed to the old and new Pods.

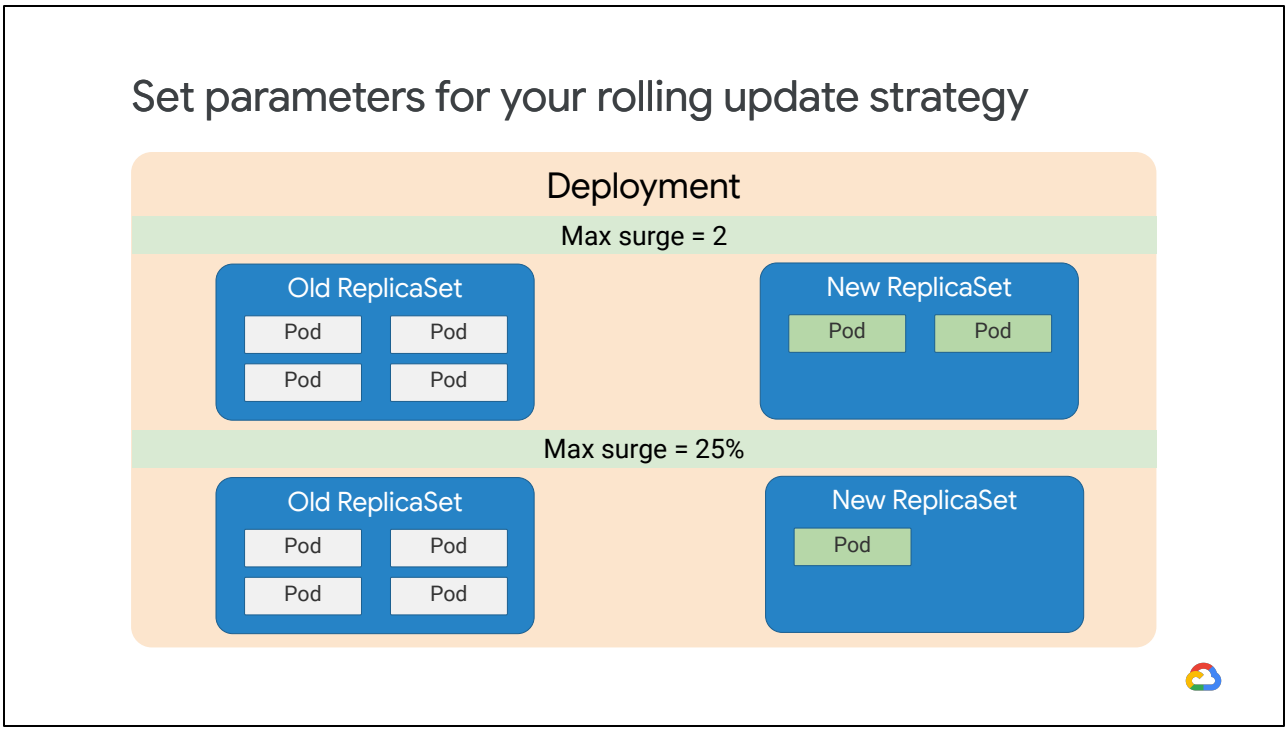
Set parameters for your rolling update strategy



In a rolling update strategy, the max unavailable and max surge fields can be used to control how the Pods are updated. These fields define a range for the total number of running Pods within the Deployment, regardless of whether they are in the old or new ReplicaSets.

The max unavailable field lets you specify the maximum number of Pods that can be unavailable during the rollout process. This number can be either absolute or a percentage. For example, you can say you want to have no more than 2 Pods unavailable during the upgrade process.

Specifying max unavailable at 25% means you want to have at least 75% of the total desired number of Pods running at the same time. The default max unavailable is 25%.



Max surge allows you to specify the maximum number of extra Pods that can be created concurrently in a new ReplicaSet. For example, you can specify that you want to add up to two new Pods at a time in a new ReplicaSet, and the Deployment controller will do exactly that.

You can also set max surge as a percentage. The Deployment controller looks at the total number of running Pods in both ReplicaSets, Old and New.

In this example, a Deployment with the desired number of Pods as 4 and a max surge of 25% will allow a maximum total of 5 Pods running at any given time between the old and new ReplicaSets. In other words, it'll allow 125% of the desired number of Pods, which is 5. Again, the default max surge is 25%.

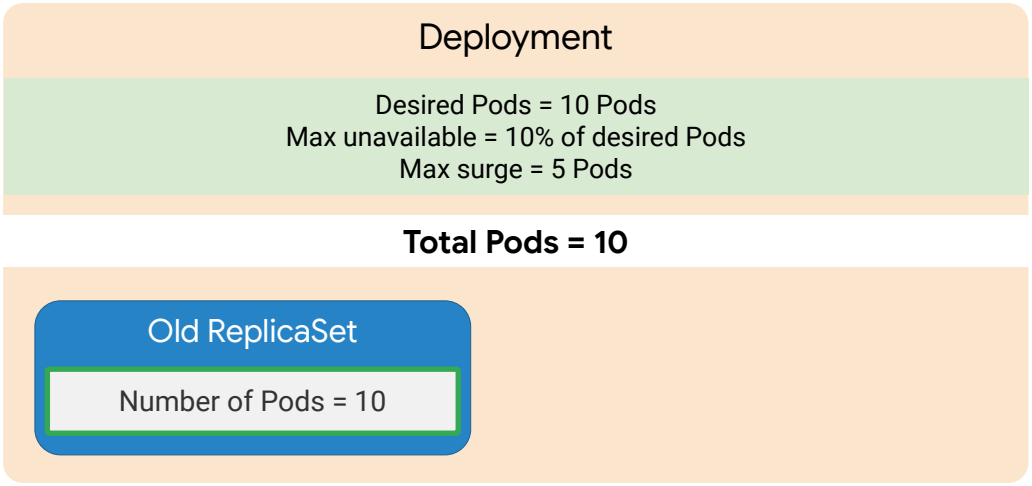
An example of a rolling update strategy

```
[...]
kind: deployment
spec:
  replicas: 10
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 5
      maxUnavailable: 30%
[...]
```



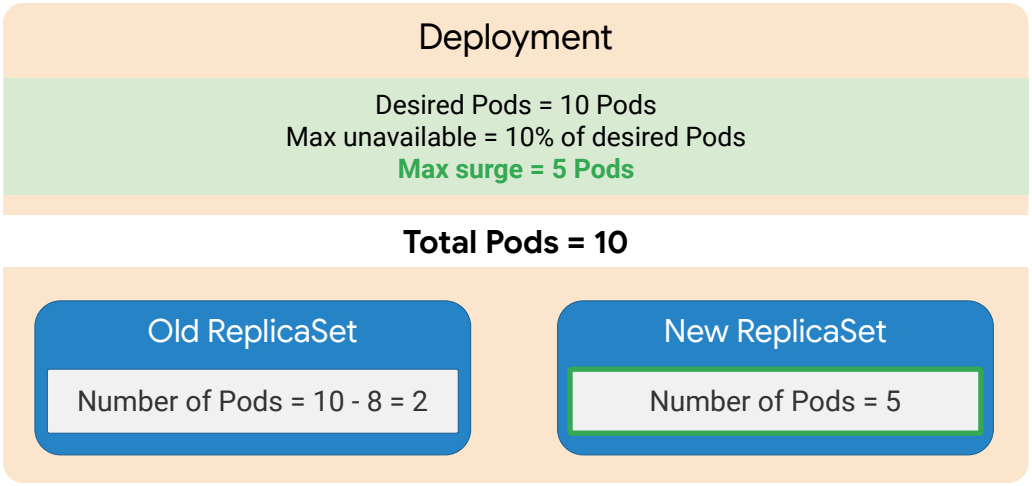
Let's look at a Deployment with a desired number of Pods set to 10, max unavailable set to 10%, and max surge set to 5.

An example of a rolling update strategy



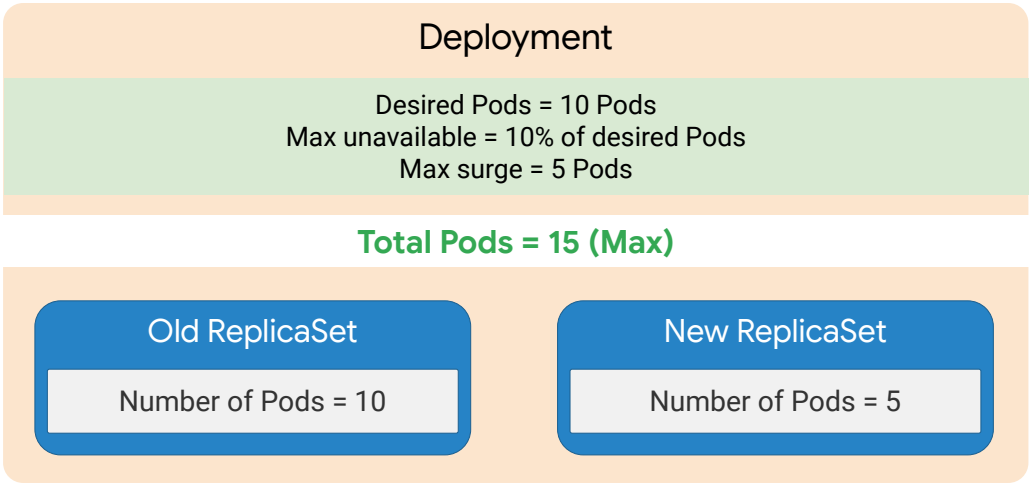
The Old ReplicaSet has 10 Pods.

An example of a rolling update strategy

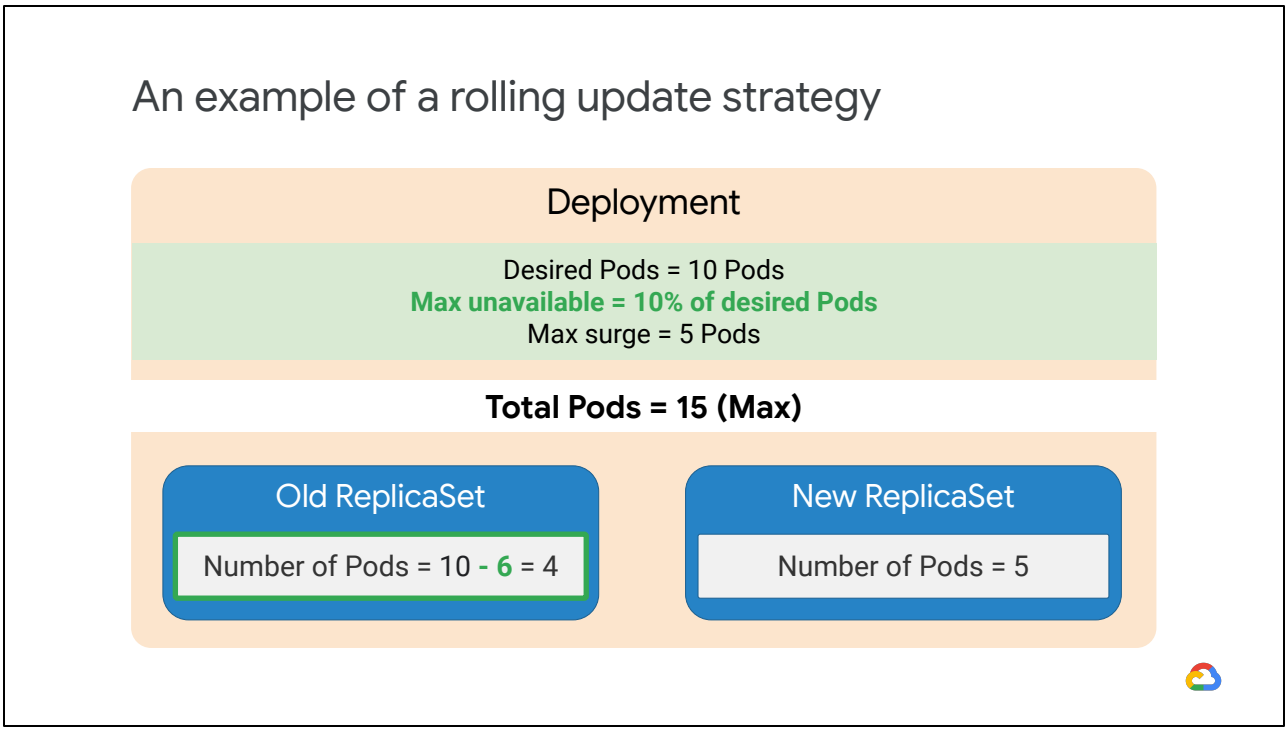


The Deployment will begin by creating 5 new Pods in a new ReplicaSet based on max surge.

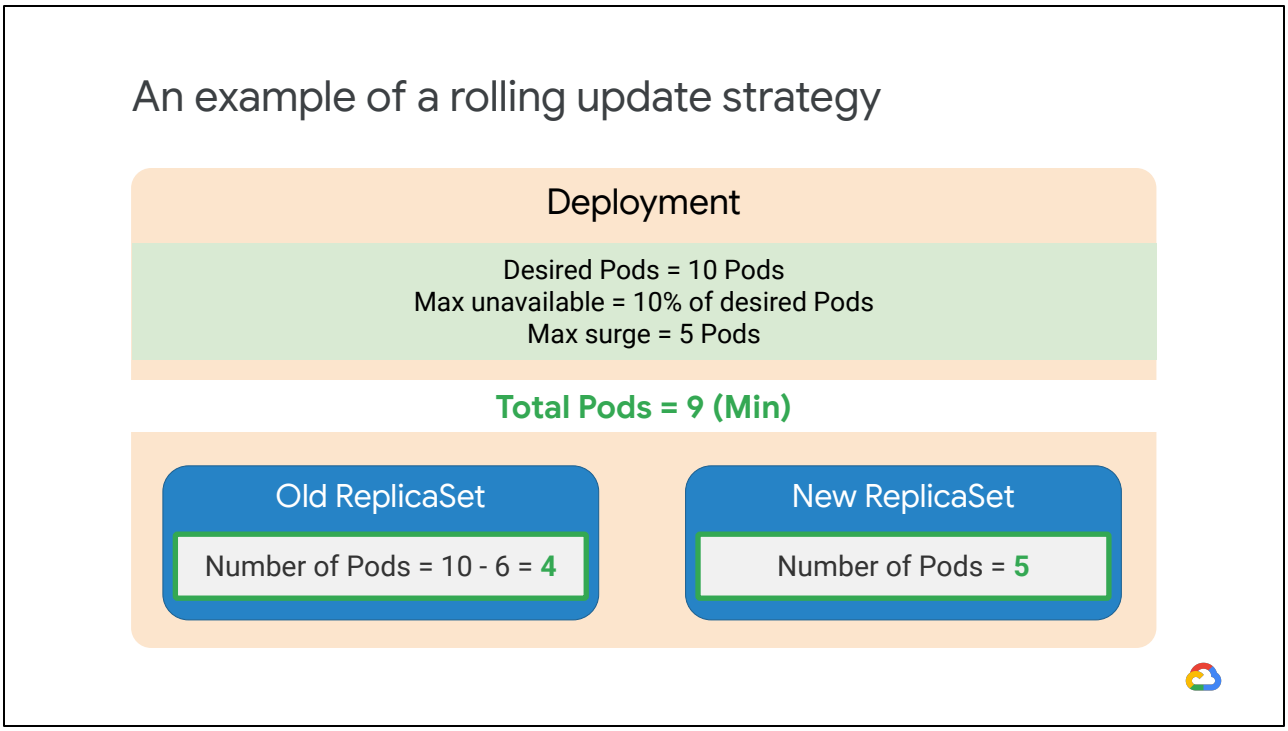
An example of a rolling update strategy



When those new Pods are ready, the total number of Pods changes to 15.

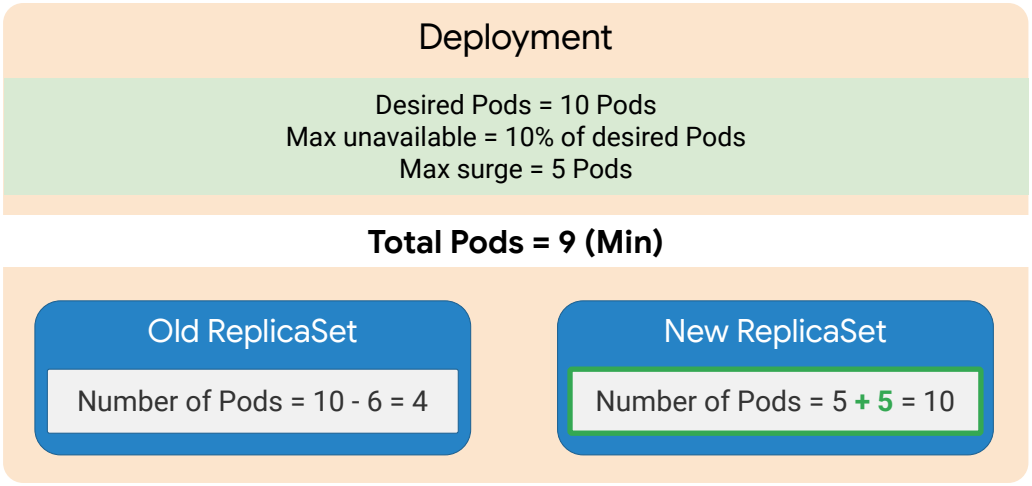


With max unavailable set to 10%, the minimum number of Pods running, regardless of old or new ReplicaSet, is 10 minus 10%, which equates to 9 Pods. So the rollout can remove old Pods until the overall total is no lower than 9 Pods. This means that 6 Pods can be removed from the old ReplicaSet at this stage.

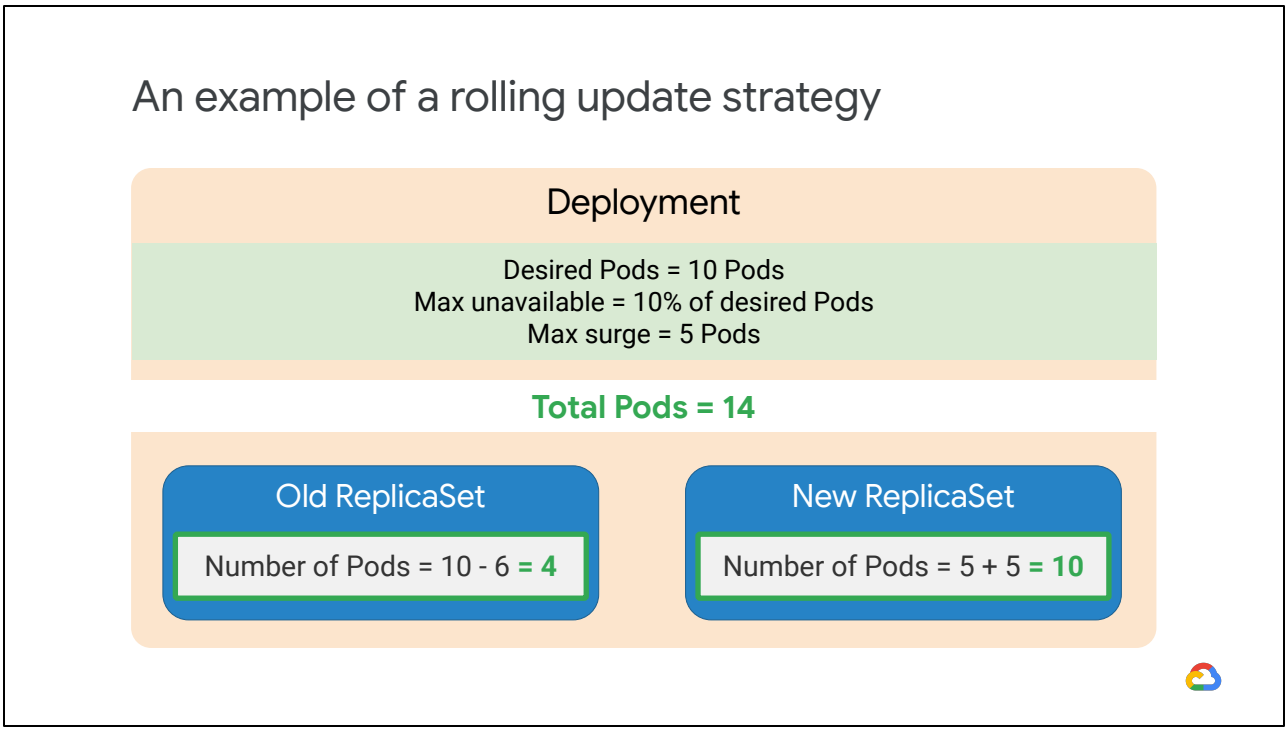


This keeps the minimum total at 9 Pods: 5 in the new ReplicaSet, and 4 in the old ReplicaSet.

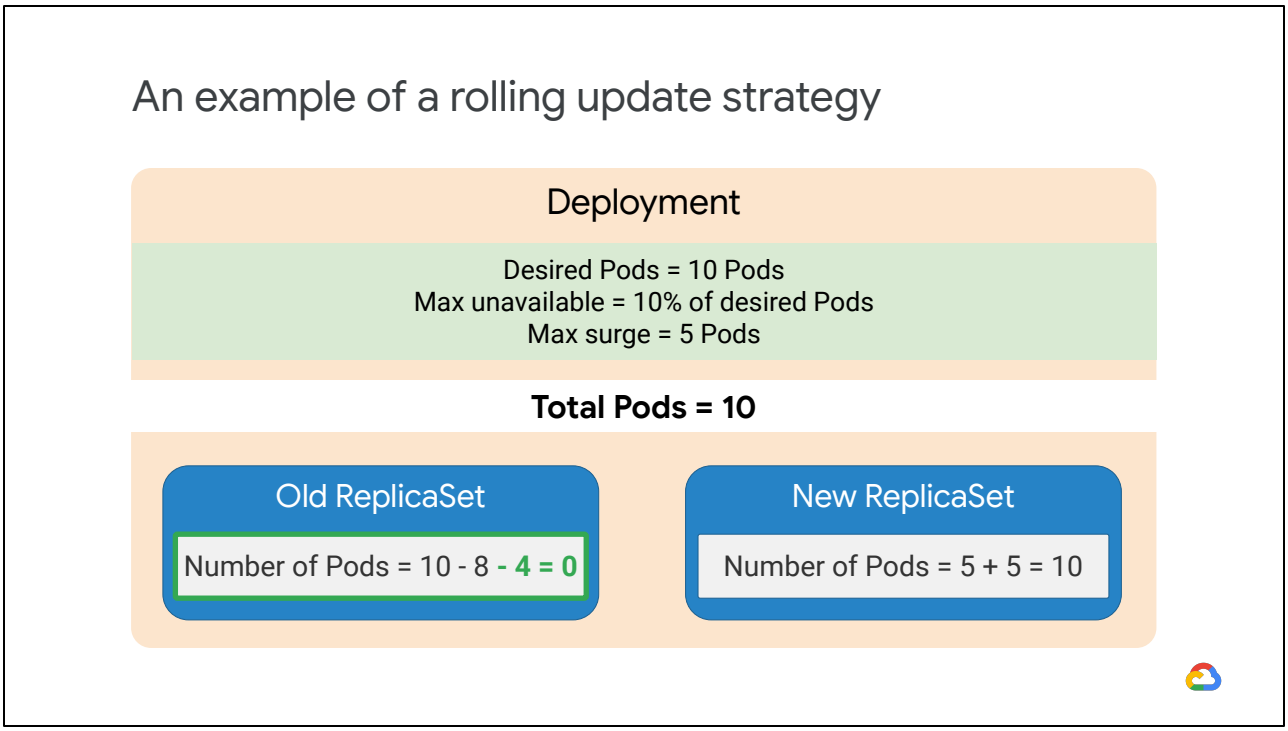
An example of a rolling update strategy



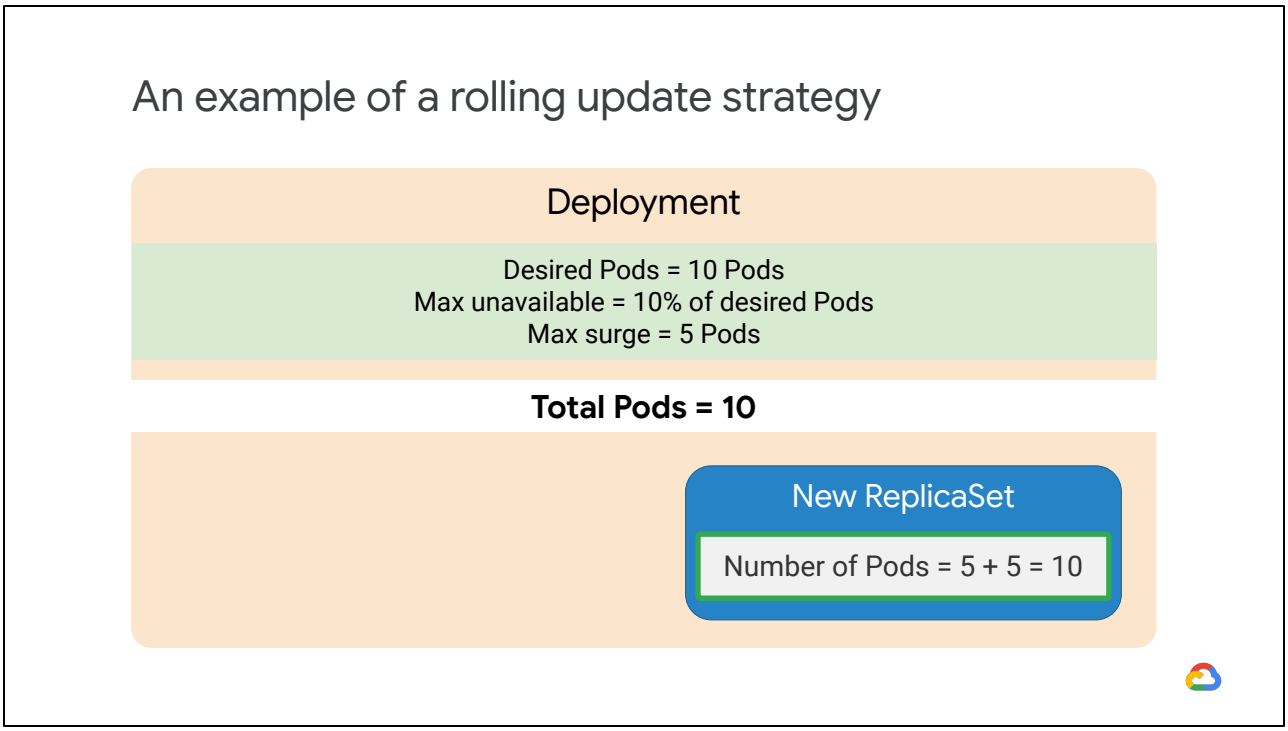
Next, 5 more Pods are launched in the new ReplicaSet.



This creates a total of 10 Pods in a new ReplicaSet and a total of 14 across all ReplicaSets.



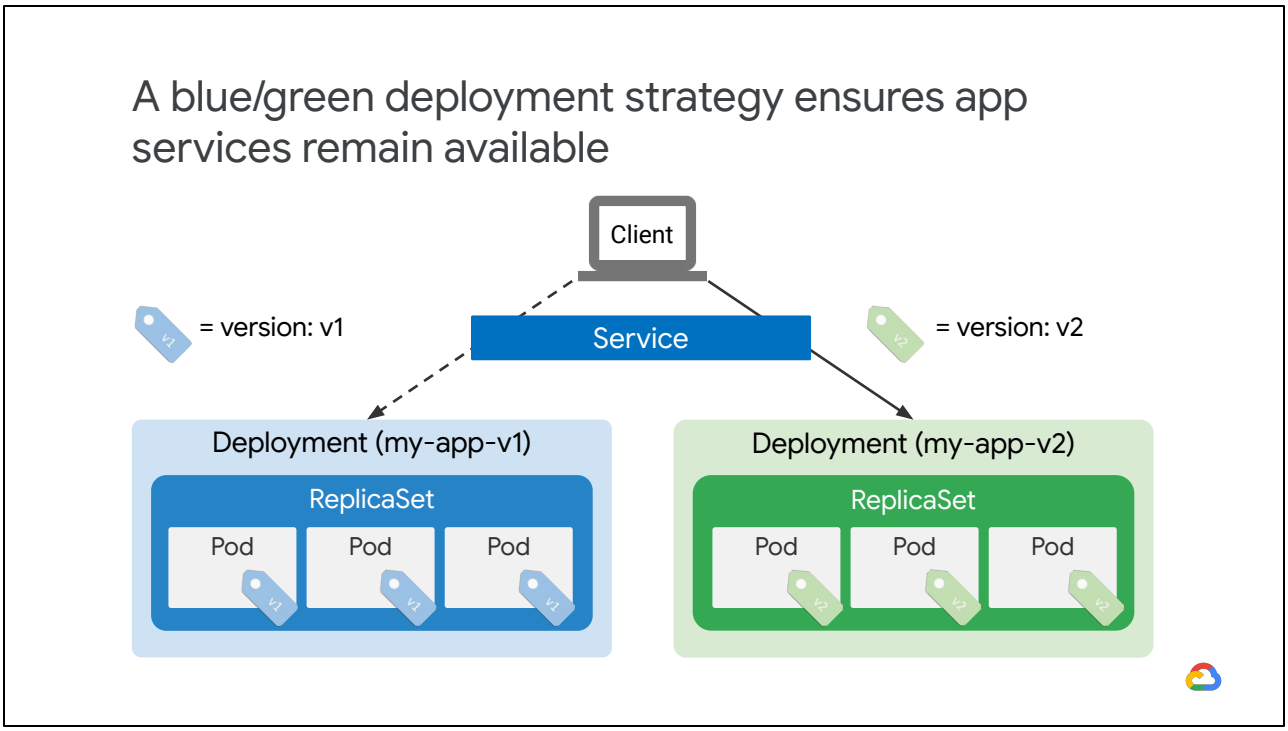
Finally, the remaining 4Pods in the old set are deleted. The old ReplicaSet is retained for rollback even though it is now empty.



This leaves 10 Pods in a new ReplicaSet.

There are a few additional options, such as ‘Min Ready Seconds’ and ‘Progress Deadline Seconds.’ ‘Min Ready Seconds’ defines the number of seconds to wait before a Pod is considered Available, without crashing any of its containers.

The default for minReadySeconds is 0, meaning that as soon as the Pod is ready, it’s made available.
Another option is ‘progressDeadlineSeconds,’ where you specify the wait period before a Deployment reports that it has failed to progress.



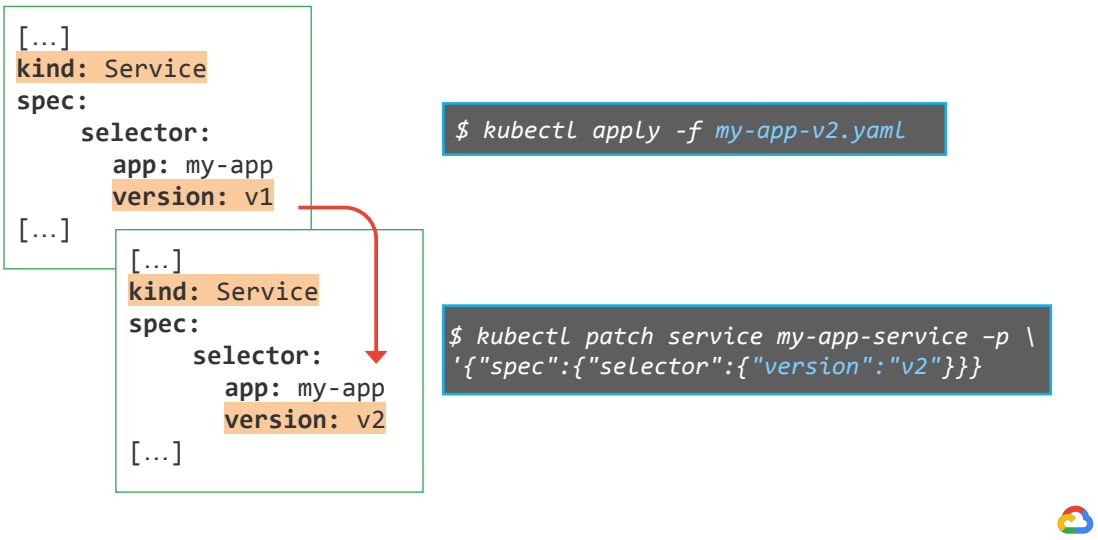
A blue/green deployment strategy is useful when you want to deploy a new version of an application and also ensure that application services remain available while the Deployment is updated.

With a blue/green update strategy, a completely new Deployment is created with a newer version of the application. In this case, it's my-app-v2.

When the Pods in the new Deployment are ready, the traffic can be switched from the old blue version to the new green version. But how can you do this?

This is where a Kubernetes Service is used. Services allow you to manage the network traffic flows to a selection of Pods. This set of Pods is selected using a label selector.

Applying a blue/green deployment strategy

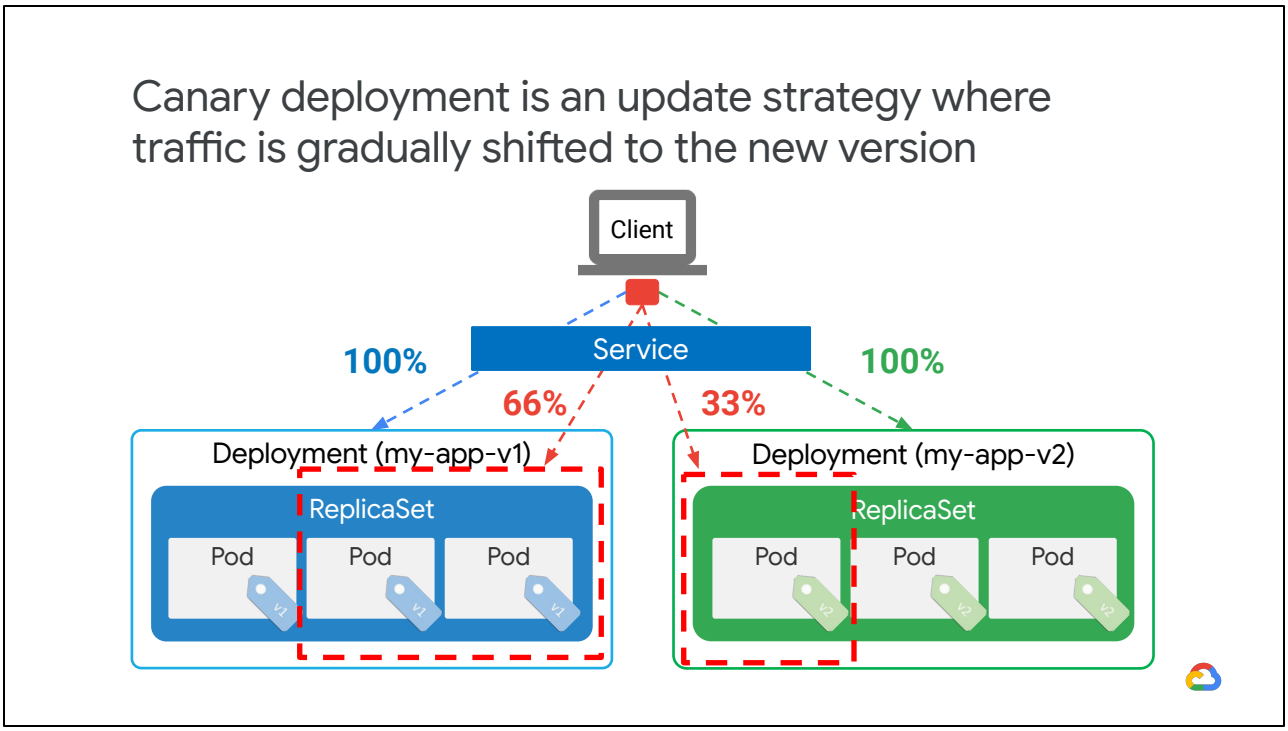


Here, in the Service definition, Pods are selected based on the label selector, where Pods in this example belong to my-app and to version v1.

When a new Deployment, labelled v2 in this case, is created and is ready, the version label in the Service is changed to the newer version, labeled v2 in this example. Now, the traffic will be directed to the newer set of Pods, the green deployment with the v2 version label, instead of to the old blue deployment Pods that have the v1 version label. The blue Deployment with the older version can then be deleted.

The advantage of this update strategy is that the rollouts can be instantaneous, and the newer versions can be tested internally before releasing them to the entire user base, for example by using a separate service definition for test user access.

The disadvantage is that resource usage is doubled during the Deployment process.



The canary method is another update strategy based on the blue/green method, but traffic is *gradually* shifted to the new version. The main advantages of using canary deployments are that you can minimize excess resource usage during the update, and because the rollout is gradual, issues can be identified before they affect all instances of the application.

In this example, 100% of the application traffic is directed initially to my-app-v1 .

When the canary deployment starts, a subset of the traffic, 33% in this case, or a single pod, is redirected to the new version, my-app-v2, while 66%, or two pods, from the older version, my-app-v1, remain running.

When the stability of the new version is confirmed, 100% of the traffic can be routed to this new version. How is this done?

Applying a canary deployment

```
[...]
kind: Service
spec:
    selector:
      app: my-app
[...]
```

```
$ kubectl apply -f my-app-v2.yaml
```

```
$ kubectl scale deploy/my-app-v2 –replicas=10
```

```
$ kubectl delete -f my-app-v1.yaml
```



In the blue/green update strategy covered previously, both the app and version labels were selected by the Service, so traffic would only be sent to the Pods that are running the version defined in the Service.

In a Canary update strategy, the Service selector is based only on the application label and does not specify the version. The selector in this example covers all Pods with the app:my-app label. This means that with this Canary update strategy version of the Service, traffic is sent to all Pods, regardless of the version label.

This setting allows the Service to select and direct the traffic to the Pods from both Deployments. Initially, the new version of the Deployment will start with zero replicas running. Over time, as the new version is scaled up, the old version of the Deployment can be scaled down and eventually deleted.

With the canary update strategy, a subset of users will be directed to the new version. This allows you to monitor for errors and performance issues as these users use the new version, and you can roll back quickly, minimizing the impact on your overall user base, if any issues arise.

However, the complete rollout of a Deployment using the canary strategy can be a slow process and may require tools such as Istio to accurately shift the traffic. There are other deployment strategies, such as A/B testing and shadow testing. These strategies are outside the scope of this course.

Applying a Recreate strategy

```
[...]
kind: deployment
spec:
    replicas: 10
    strategy:
        type: Recreate
[...]
```



‘Recreate’ is a strategy type where all the old Pods are deleted before new Pods are created. This clearly affects the availability of your application, because the new Pods must be created and will not all be available instantly. For example, what if the contract of communication between parts of your application is changing, and you need to make a clean break? In such situations a continuous deployment strategy doesn’t make sense. All the replicas need to change at once. That’s when the Recreate strategy is recommended.

Rolling back a Deployment

```
$ kubectl rollout undo deployment [DEPLOYMENT_NAME]
```

```
$ kubectl rollout undo deployment [DEPLOYMENT_NAME] --to-revision=2
```

```
$ kubectl rollout history deployment [DEPLOYMENT_NAME] --revision=2
```

Clean up Policy:

- Default: 10 Revision
- Change: .spec.revisionHistoryLimit



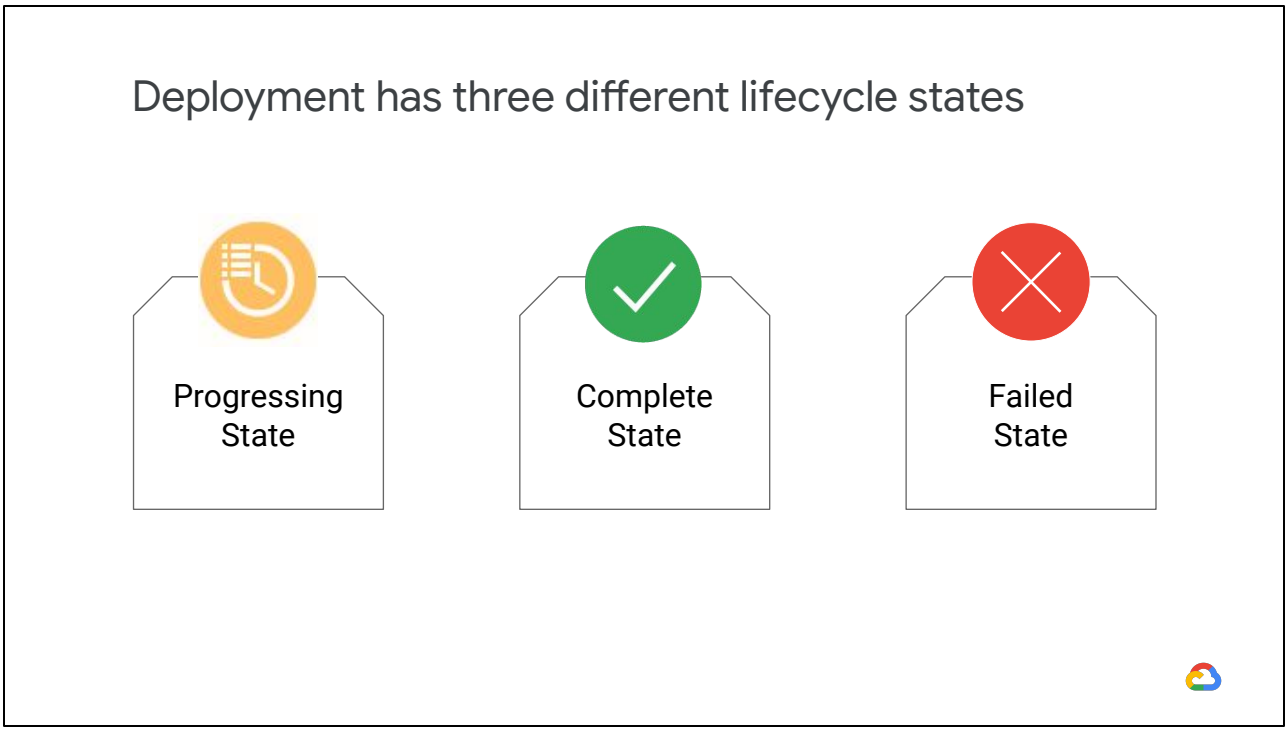
So that's 'rollout.' Next we'll discuss how to roll back updates, especially in rolling update and recreate strategies.

You roll back using a kubectl 'rollout undo' command. A simple 'rollout undo' command will revert the Deployment to its previous revision.

You roll back to a specific version by specifying the revision number.

If you're not sure of the changes, you can inspect the rollout history using the kubectl 'rollout history' command.
The GCP Console doesn't have a direct rollback feature; however, you can start Cloud Shell from your Console and use these commands. The GCP Console also shows you the revision list with summaries and creation dates.

By default, the details of 10 previous ReplicaSets are retained, so that you can roll back to them. You can change this default by specifying a revision history limit under the Deployment specification.



Any Deployment has three different lifecycle states.

The Deployment's *Progressing* state indicates that a task is being performed. What tasks? Creating a new ReplicaSet... or scaling up or scaling down a ReplicaSet.

The Deployment's *Complete* state indicates that all new replicas have been updated to the latest version and are available, and no old replicas are running.

Finally, the *Failed* state occurs when the creation of a new ReplicaSet could not be completed. Why might that happen? Maybe Kubernetes couldn't pull images for the new Pods. Or maybe there wasn't enough of some resource quota to complete the operation. Or maybe the user who launched the operation lacks permissions.

When you apply many small fixes across many rollouts, that translates to a large number of revisions, and to management complexity. You have to remember which small fix was applied with which rollout, which can make it challenging to figure out which revision to roll back to when issues arise. Remember, earlier in this specialization, we recommended that you keep your YAML files in a source code repository? That will help you manage some of this complexity.

Different actions can be applied to a Deployment

Pause

```
$ kubectl rollout pause deployment [DEPLOYMENT_NAME]
```

Resume

```
$ kubectl rollout resume deployment [DEPLOYMENT_NAME]
```

Monitor

```
$ kubectl rollout status deployment [DEPLOYMENT_NAME]
```



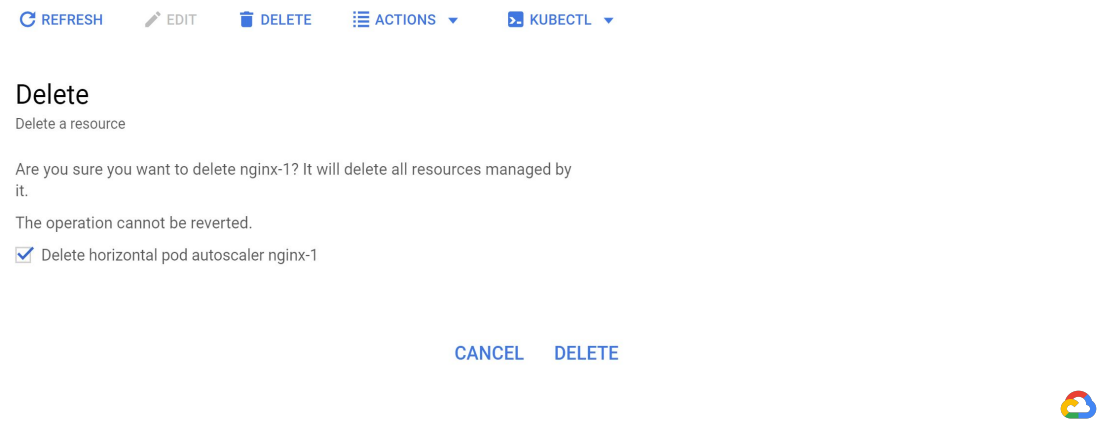
When you edit a deployment, your action normally triggers an automatic rollout. But if you have an environment where small fixes are released frequently, you'll have a large number of rollouts. In a situation like that, you'll find it more difficult to link issues with specific rollouts. To help, you can temporarily pause this rollouts by using the kubectl rollout pause command. The initial state of the Deployment prior to pausing it will continue its function, but new updates to the Deployment will not have any effect while the rollout is paused. The changes will only be implemented once the rollout is resumed.

When you resume the rollout, all these new changes will be rolled out with a single revision.

You can also monitor the rollout status by using the kubectl 'rollout status' command.

Delete a Deployment

```
$ kubectl delete deployment [DEPLOYMENT_NAME]
```



What if you're done with a Deployment? You can delete it easily by using the kubectl 'delete' command, and you can also delete it from the GCP Console. Either way, Kubernetes will delete all resources managed by the Deployment, especially running Pods.

Lab

Creating Google Kubernetes Engine Deployments



In this lab, you'll explore the basics of using deployment manifests.

The first task that you'll learn to perform is to create a deployment manifest for a Pod inside the cluster. You'll then use both the GCP Console and Cloud Shell to manually scale Pods up and down. The next task will be to trigger a deployment rollout and a deployment rollback. Various types of service types (ClusterIP, NodePort, LoadBalancer) can be used with deployments to manage connectivity and availability during updates. You'll perform a task where you define service types in the manifest and verify LoadBalancer creation. In your final task, you'll create a new canary deployment for the release of your application.

Agenda

Deployments

Jobs and CronJobs

Cluster Scaling

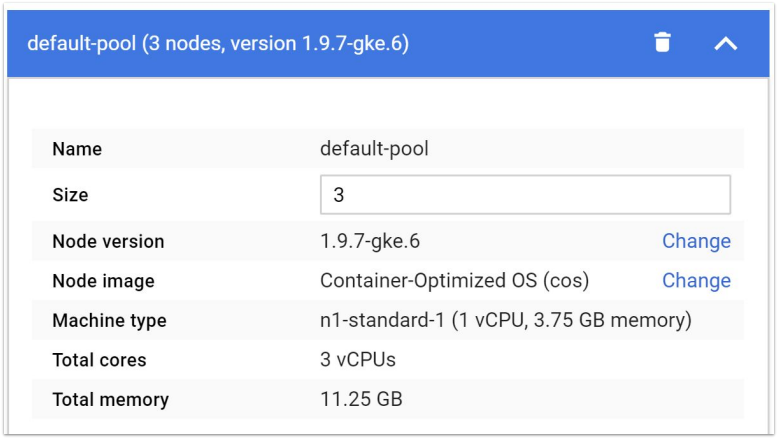
Controlling Pod Placement

Getting Software into Your Cluster



In this lesson, we look at how you manage cluster scaling in GKE. I will review the concept of node pools. You'll learn how you can modify the capacity of your entire cluster… either by manually changing the number of nodes in node pools, or by configuring additional node pools. You will then learn how the GKE cluster autoscaler works, and how you can configure it to manage the size of your cluster automatically.

Scaling a cluster from the GCP Console



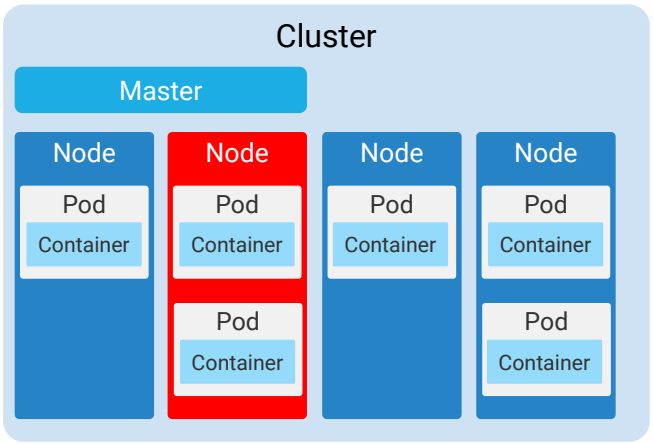
The level of resources your applications need will vary over time. If you need to change the amount of resources available in your Kubernetes Engine clusters, you can increase or decrease the number of Google Kubernetes Engine nodes in your cluster directly from the GCP Console. New nodes created during the process automatically self-register to the cluster within the GCP environment. A node pool is a subset of node instances within a cluster that all have the same configuration.

A node pool is a subset of node instances within a cluster that all have the same configuration. Node pools use a NodeConfig specification. Each node in the pool has a Kubernetes node label which has the node pool's name as its value. When you create a container cluster, the number and type of nodes that you specify becomes the default node pool. Then, you can add additional custom node pools of different sizes and types to your cluster. All

nodes in any given node pool are identical to one another.

In the GCP Console, you can manually increase the size of the cluster by increasing the size of the node pools in the cluster. The Node Pool size represents the number of nodes in the node pool per zone. For example, if this particular pool spans two compute zones, and you increase the node size from 3 to 6, each zone will have 6 nodes registered, and the total number of nodes in this pool will be 12. Existing Pods are not moved to the newer nodes when the cluster size is increased.

Manual Cluster Scale down selects nodes randomly



You can also manually decrease the cluster size. When you reduce the size of a cluster, the nodes to be removed are selected randomly. The resize process doesn't differentiate between nodes that are running Pods and ones that are empty.

When you remove a node from the cluster, all the Pods within that node will be terminated gracefully. Graceful termination means that a TERM signal is first sent to the main process in each container. A grace period is then allowed before a KILL signal is sent and the Pod is deleted. This grace period is defined for each Pod.

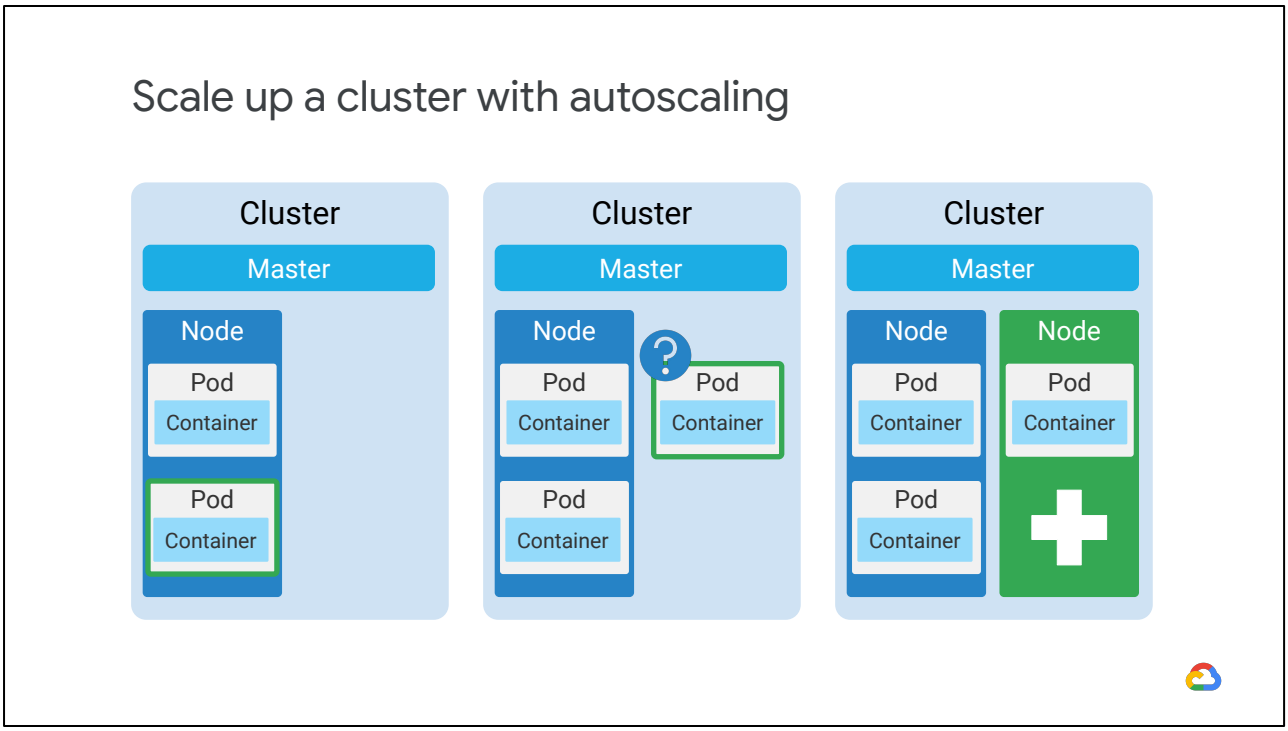
If these Pods are managed by a replication controller such as a ReplicaSet or StatefulSet, they'll be rescheduled on the remaining nodes. Otherwise, the Pods won't be restarted elsewhere.

Scaling a cluster using the gcloud command

```
gcloud container clusters resize
projectdemo --node-pool default-pool \
    --size 6
```



You can also resize a cluster manually from the command line using the ‘resize’ gcloud command. As with the GCP Console, if you reduce the size of the cluster, then nodes that are running Pods and nodes without Pods aren’t differentiated. Resize will pick instances to remove at random, and any running Pods will be terminated gracefully. If your Pods aren’t managed by a replication controller, they won’t be restarted.



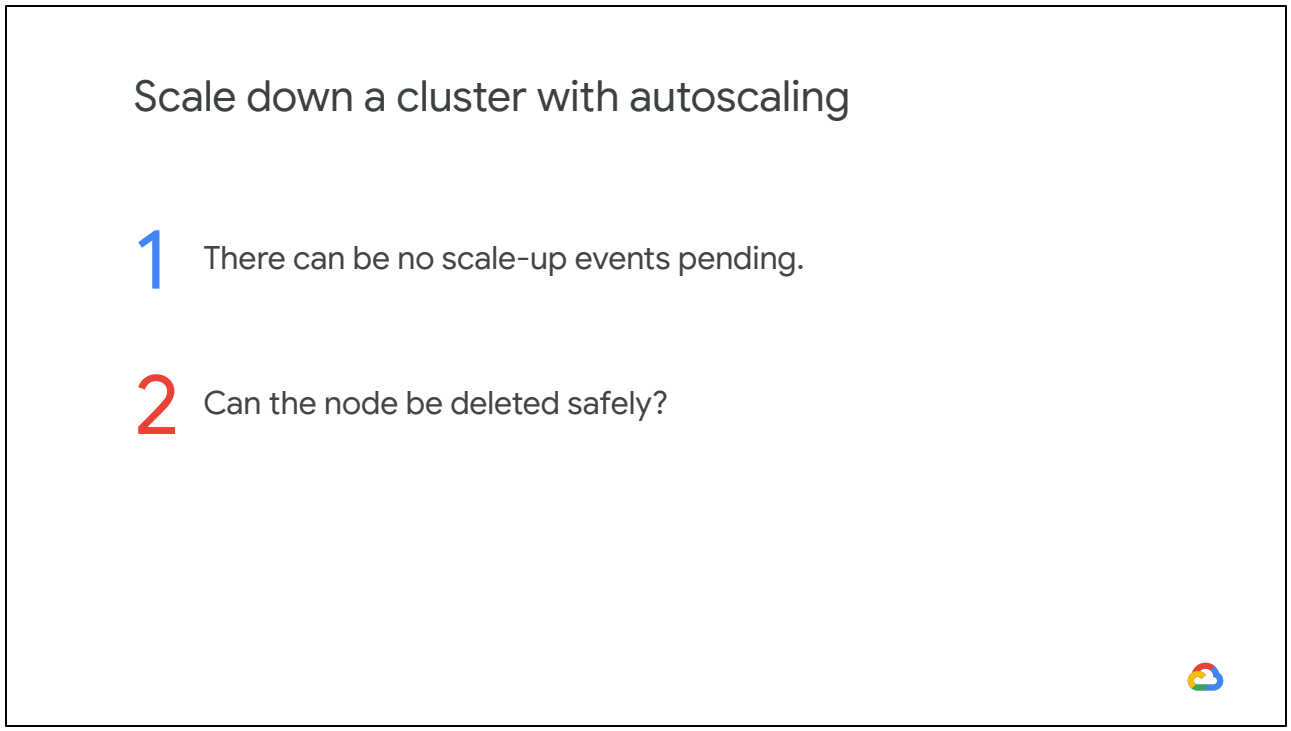
Cluster autoscaler controls the number of worker nodes in response to workload demands. GKE's cluster autoscaler can automatically resize a cluster based on the resource demands of your workload. By default, the cluster autoscaler is disabled. Cluster autoscaling allows you to pay only for resources that are needed at any given moment and to automatically get additional resources when demand increases. When autoscaling is enabled, GKE automatically adds a new node to your cluster if you've created new Pods that don't have enough capacity to run. If a node in your cluster is underutilized and its Pods can be run on other nodes, GKE can delete the node. Keep in mind that when nodes are deleted, your applications can experience some disruption. Before enabling autoscaling, you should make sure that your services can tolerate the potential disruption.

Pods have their own CPU and memory resource requirements,

based on the resource requests and limits of their containers. When it schedules a Pod, the Kubernetes scheduler must allocate that Pod to a node that can meet the demands of all of the Pod's containers.

If there isn't enough resource capacity across any of the node pools, the Pod will have to wait until either other Pods terminate and free up capacity or additional nodes are added. When the Pod has to wait for resource capacity, the scheduler marks the Pod as unschedulable by setting its 'schedulable' Pod Condition to false with the reason – Unschedulable.

If you enabled autoscaling, the GKE autoscaler checks whether a scale-up action will help the situation as soon as it detects that any Pod is considered unschedulable. If so, it adds a new node to the node pool where the Pod is waiting for resources to become available, and the Pod is then scheduled on that node. However, this requires a new VM instance to be deployed, which will need to start up and initialize before it can be used to schedule Pods. Also note that while the autoscaler ensures that all nodes in a single node pool have the same set of labels applied, labels that have been manually added after initial cluster or node pool creation are not automatically carried over to the new nodes when the cluster autoscales.



Deploying a Pod to an existing running node may only take a few seconds, but it might take minutes before the new node added by the autoscaler can be used. And adding nodes means spending money, too. So you should think of cluster scaling as a coarse-grained operation that should happen infrequently, and Pod scaling with Deployments as a fine-grained operation that should happen frequently. You can use both kinds of scaling together to balance your performance and your spending. The GKE cluster autoscaler can also scale down nodes. Let's look at how the autoscaler manages scale-down.

First, the cluster autoscaler ensures that there's no scale-up event pending. If a scale-up event happens during the scale-down process, the scale-down is not executed. Second, it checks that the node can be deleted safely.

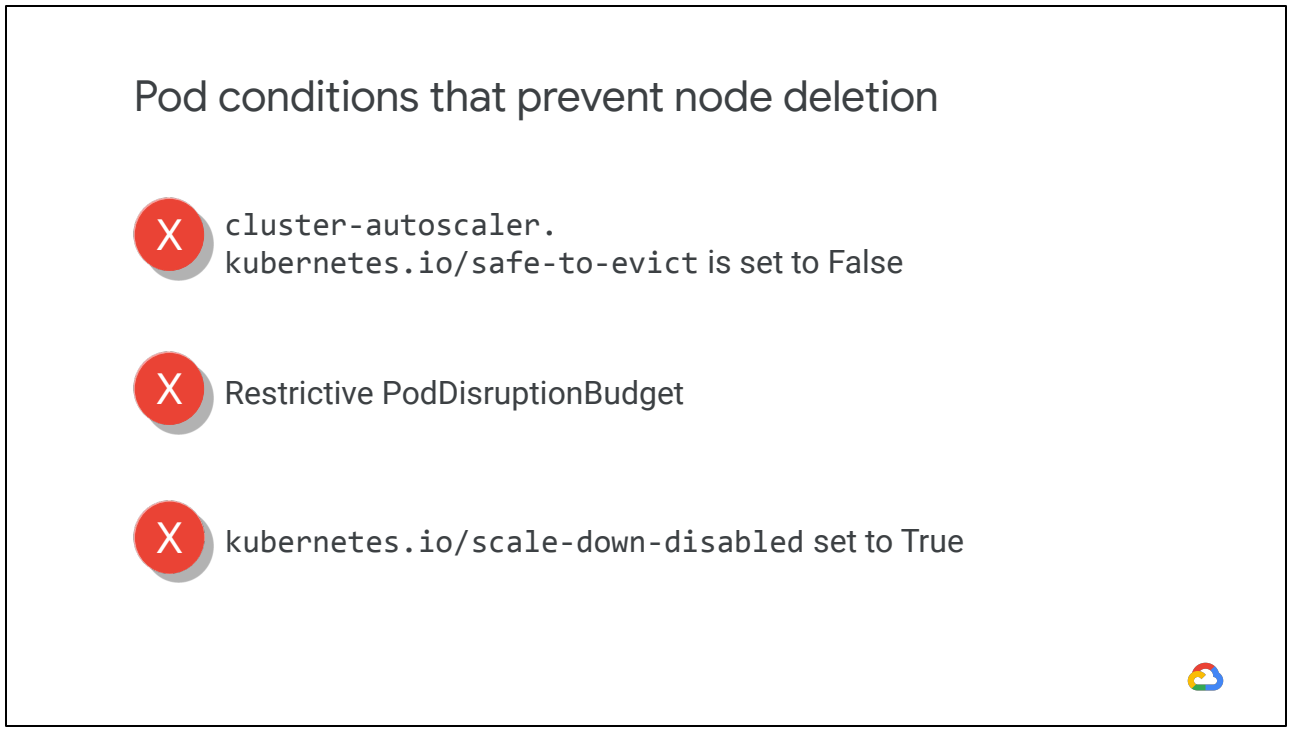
Pod conditions that prevent node deletion

- X Not run by a controller
- X Has local storage
- X Restricted by constraint rules



If the node contains Pods that meet any of the following conditions, then the node cannot be deleted ...

Pods that are not managed by a controller. These are any Pods that are not in a Deployment, ReplicaSet, Job, StatefulSet, etc.
Pods that have local storage.
Pods that are restricted by constraint rules that prevent them from running on any other node, which will be explained in detail later in this module.



There are a number of non-default settings that can be explicitly set to prevent node deletion.

Pods that have the safe-to-evict annotation set to ‘False.’ The safe-to-evict annotation provides a direct setting at the Pod level that tells the autoscaler that the Pod cannot be evicted. As a result, the node that it’s running on won’t be selected for deletion when the cluster is scaled down.

Pods that have a restricted PodDisruptionBudget can also prevent a node from being deleted. You can define PodDisruptionBudget to specify the number of controller replicas that must be available at any given time. For example, in a Deployment with 3 replicas, and PodDisruptionBudget set to 2, only one replica can be evicted or disrupted at a time.

At the node level, if the node’s scale-down-disabled annotation is set to ‘True,’ that node will always be excluded from scale-down actions.

Agenda

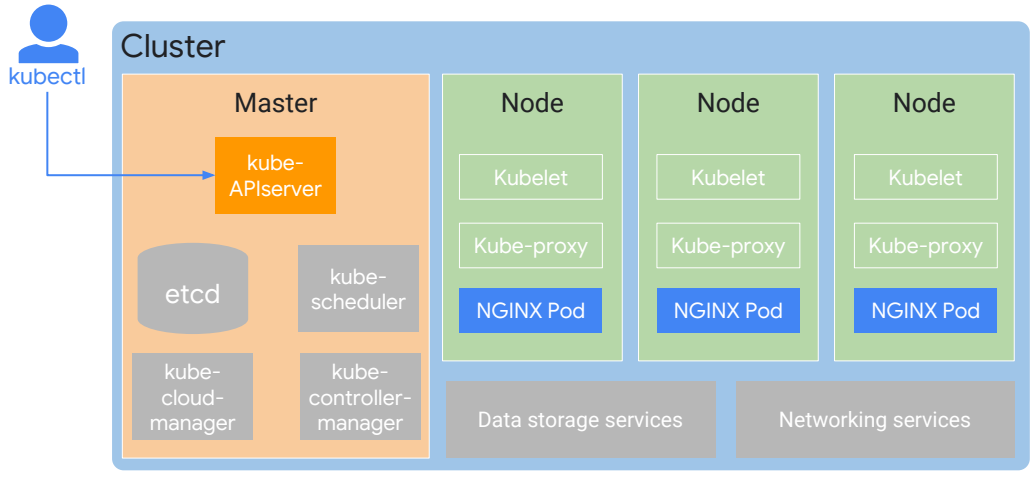
The kubectl command

Introspection



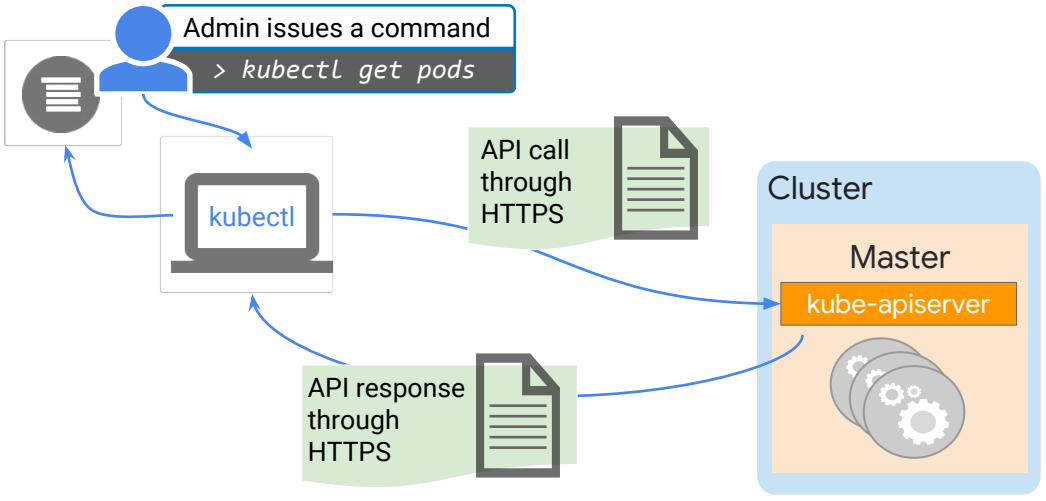
Let’s start by discussing the kubectl command.
Kubectl is a utility used by administrators to control Kubernetes clusters. You use it to communicate with the Kube API server on your master.

Kubectl transforms your command-line entries into API calls

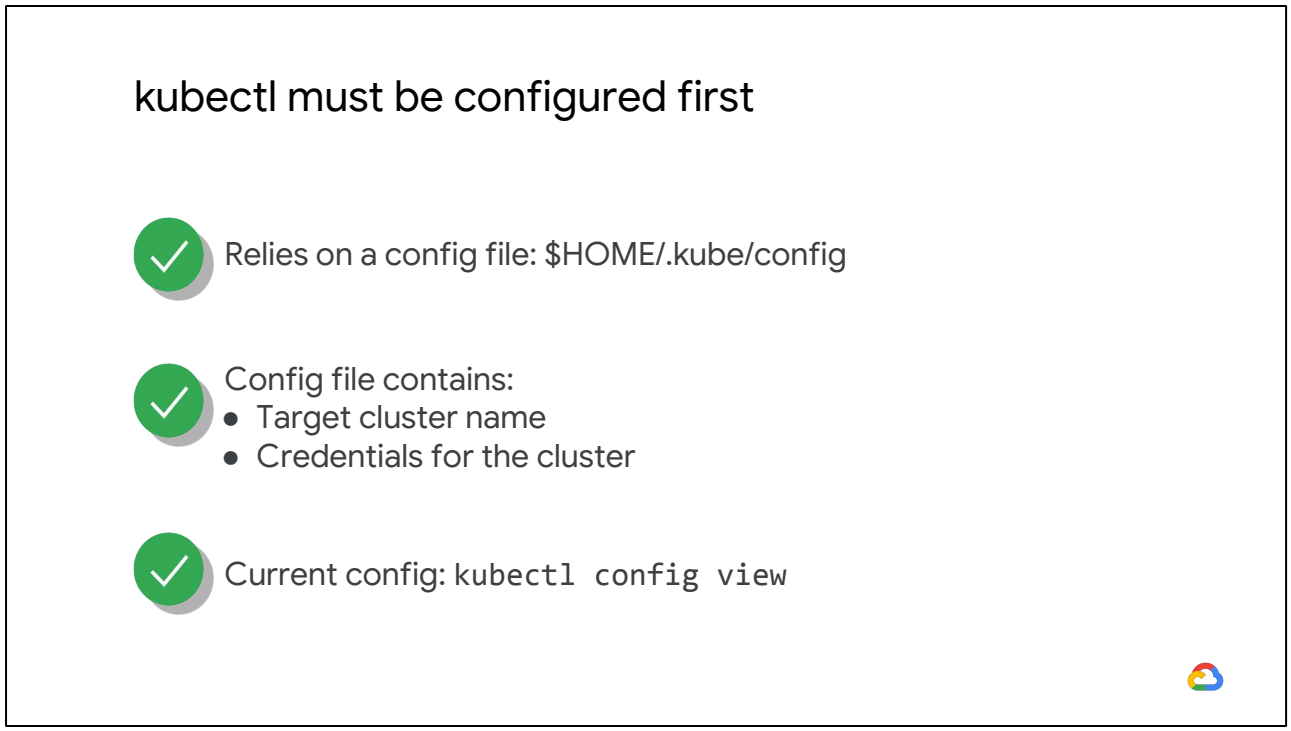


Kubectl transforms your command-line entries into API calls that it sends to the Kube API server within your selected Kubernetes cluster. Before it can do any work for you, kubectl must be configured with the location and credentials of a Kubernetes cluster.

Use kubectl to see a list of Pods in a cluster



For example, take an administrator who wants to see a list of Pods in a cluster. After connecting kubectl to the cluster with proper credentials, the administrator can issue the kubectl ‘get pods’ command.
Kubectl converts this into an API call, which it sends to the Kube API server through HTTPS on the cluster’s master server.
The Kube API server processes the request by querying etcd.
The Kube API server then returns the results to kubectl through HTTPS.
Finally, kubectl interprets the API response and displays the results to the administrator at the command prompt.



Before you can use kubectl to configure your cluster, you must configure it first.

Kubectl stores its configuration in a file in your home directory in a hidden folder named .kube.

The configuration file contains the list of clusters and the credentials that you'll use to attach to each of those clusters. You may be wondering where you get these credentials. For GKE, the service provides them to you through the gcloud command. I'll show you how that works in a moment.

To view the configuration, you can either open the config file or use the kubectl command: 'config view'. Just to be clear here: kubectl config view tells you about the configuration of the kubectl *command itself*. Other kubectl commands tell you about the configurations of your cluster and workloads.

Connecting to a Google Kubernetes Engine (GKE) cluster

```
$ gcloud container clusters \
  get-credentials [CLUSTER_NAME] \
  --zone [ZONE_NAME]
```



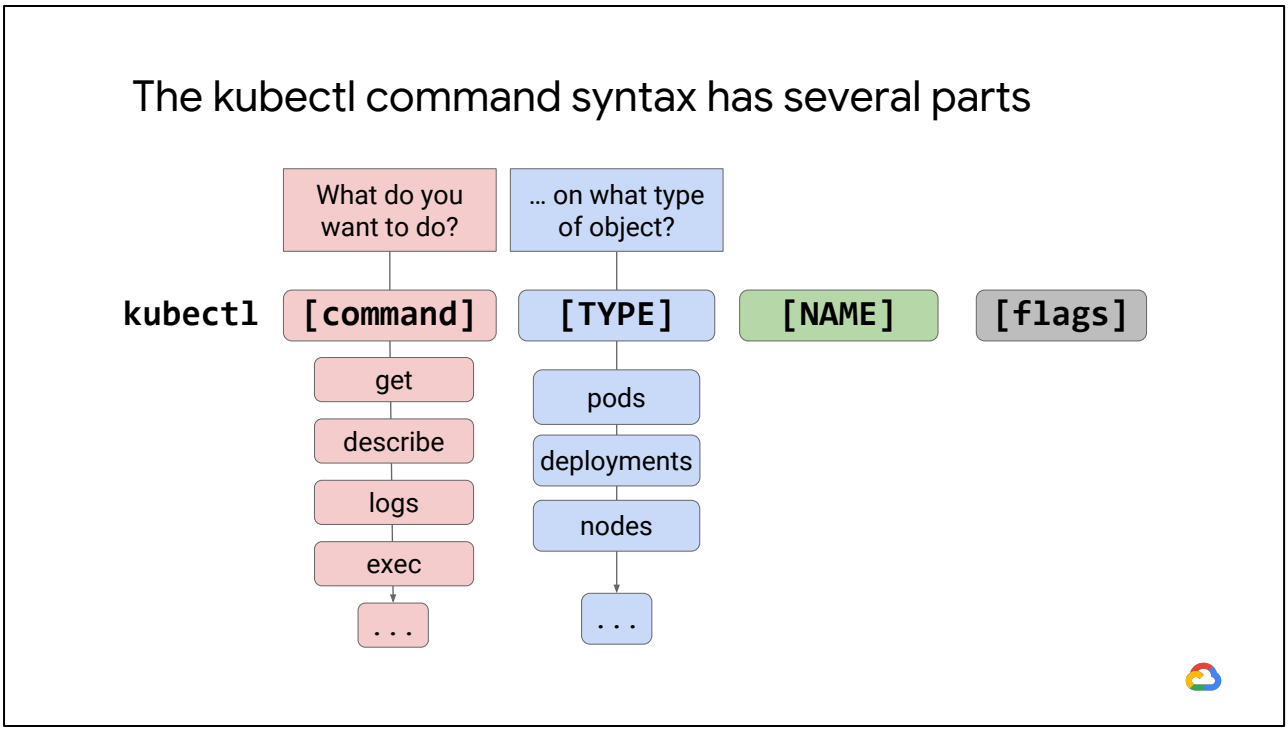
To connect to a GKE cluster with kubectl, retrieve your credentials for the specified cluster first. To do this, use the ‘get-credentials’ gcloud command in any other environment where you’ve installed the gcloud command-line tool and kubectl. Both of these tools are installed by default in the Cloud Shell.

The gcloud ‘get-credentials’ command writes configuration information into a config file in the .kube directory in the $HOME directory by default. If you rerun this command for a different cluster it’ll update the config file with the credentials for the new cluster. You only need to perform this configuration process once per cluster in your Cloud Shell, because the .kube directory and its contents stay in your $HOME directory.

Can you figure out why the command is “gcloud get-credentials” rather than “kubectl get-credentials”? It’s because the kubectl command requires credentials to work at all. The gcloud command is how authorized users interact with GCP from the command line. The gcloud get-credentials command gives you the credentials you

need to connect with a GKE cluster if you are authorized to do so.

In general, kubectl is a tool for administering the internal state of an existing cluster. But kubectl can’t create new clusters or change the shape of existing clusters; for that you need the GKE control plane, which the gcloud command and the GCP Console are your interfaces to.

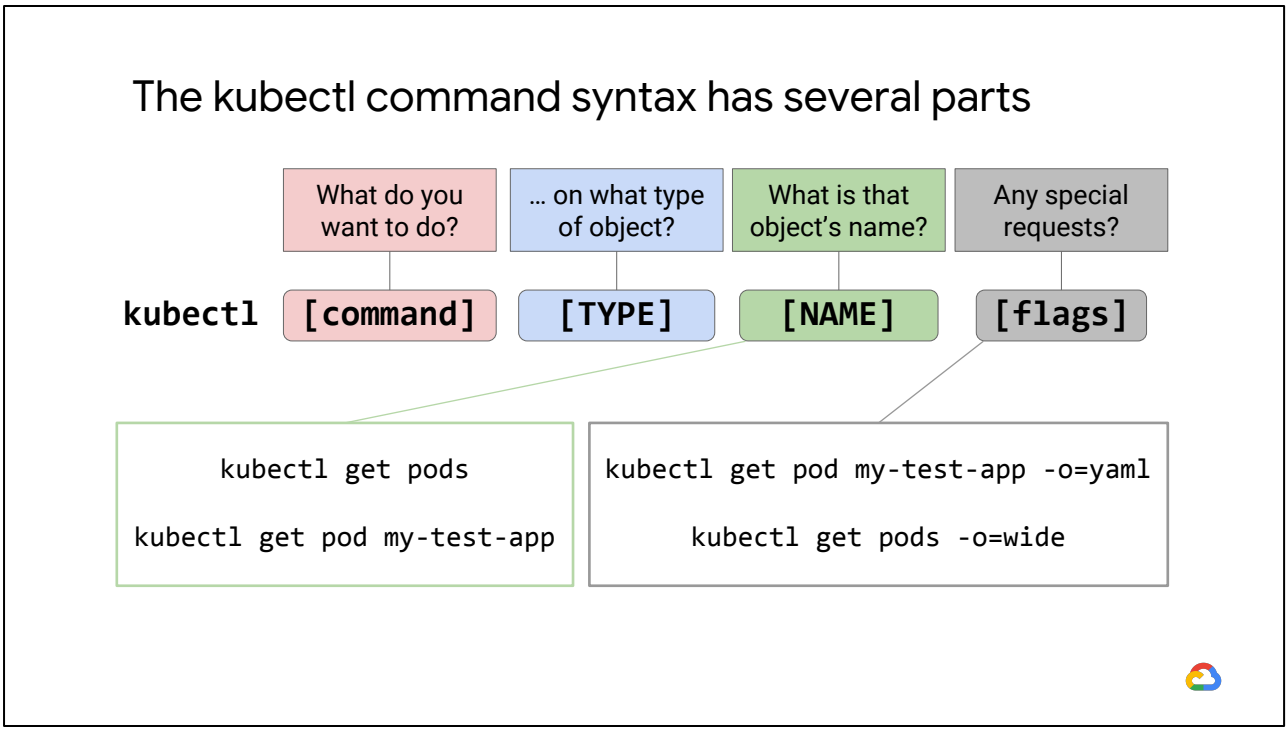


Once the config file in the .kube folder has been configured, the kubectl command automatically references this file and connects to the default cluster without prompting you for credentials. Now let's talk about how to use the kubectl command. Its syntax is composed of several parts: the command, the type, the name, and optional flags.

'Command' specifies the action that you want to perform, such as get, describe, logs, or exec. Some commands show you information, while others allow you to change the cluster's configuration.

'TYPE' defines the Kubernetes object that the 'command' acts upon. For example, you could specify Pods, Deployments, nodes, or other objects, including the cluster itself.

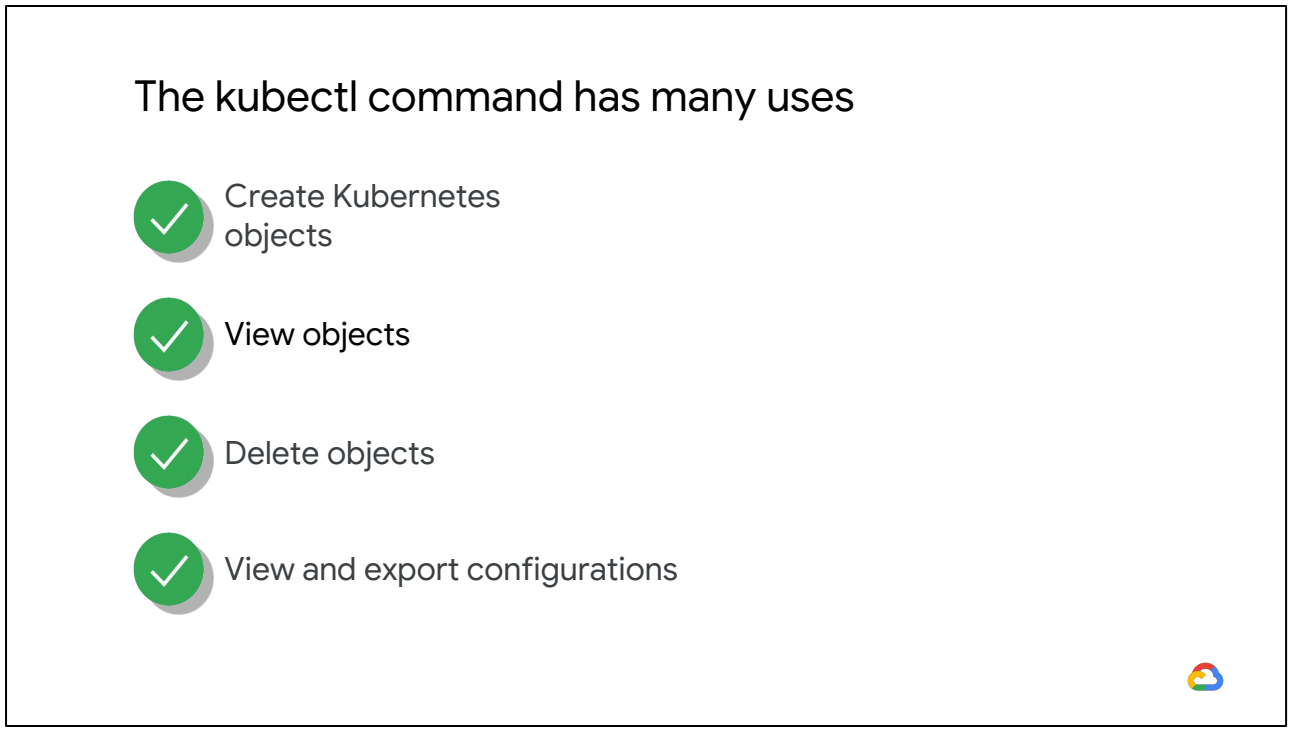
TYPE used in combination with 'command' tells kubectl what you want to do and the type of object you want to perform that action on.



'NAME' specifies the object defined in 'TYPE.' The Name field isn't always needed, especially when you're using commands that list or show you information.
For example, if you run the command "kubectl get pods" without specifying a name, the command returns the list of all Pods. To filter this list you specify a Pod's name, such as "kubectl get pod my-test-app ". kubectl then returns information only on the Pod named 'my-test-app'.

Some commands support additional optional flags that you can include at the end of the command.
Think of this as making a special request, like formatting the output in a certain way.
You could view the state of a Pod by using the command "kubectl get pod my-test-app -o=yaml". By the way, telling kubectl to give you output in YAML format is a really useful tool. You'll often want to capture the existing state of a Kubernetes object in a YAML file so that, for example, you can recreate it in a different cluster.

You can also use flags to display more information than you normally see. For instance, you can run the command “kubectl get pods -o=wide” to display the list of Pods in “wide” format, which means you see additional columns of data for each of the Pods in the list. One noteworthy piece of extra information you get in wide format: which Node each Pod is running on.



You can do many things with the kubectl command, from creating Kubernetes objects, to viewing them, deleting them, and viewing or exporting configuration files.

Just remember to configure kubectl first or to use the --kubeconfig or --context parameters, so that the commands you type are performed on the cluster you intended.

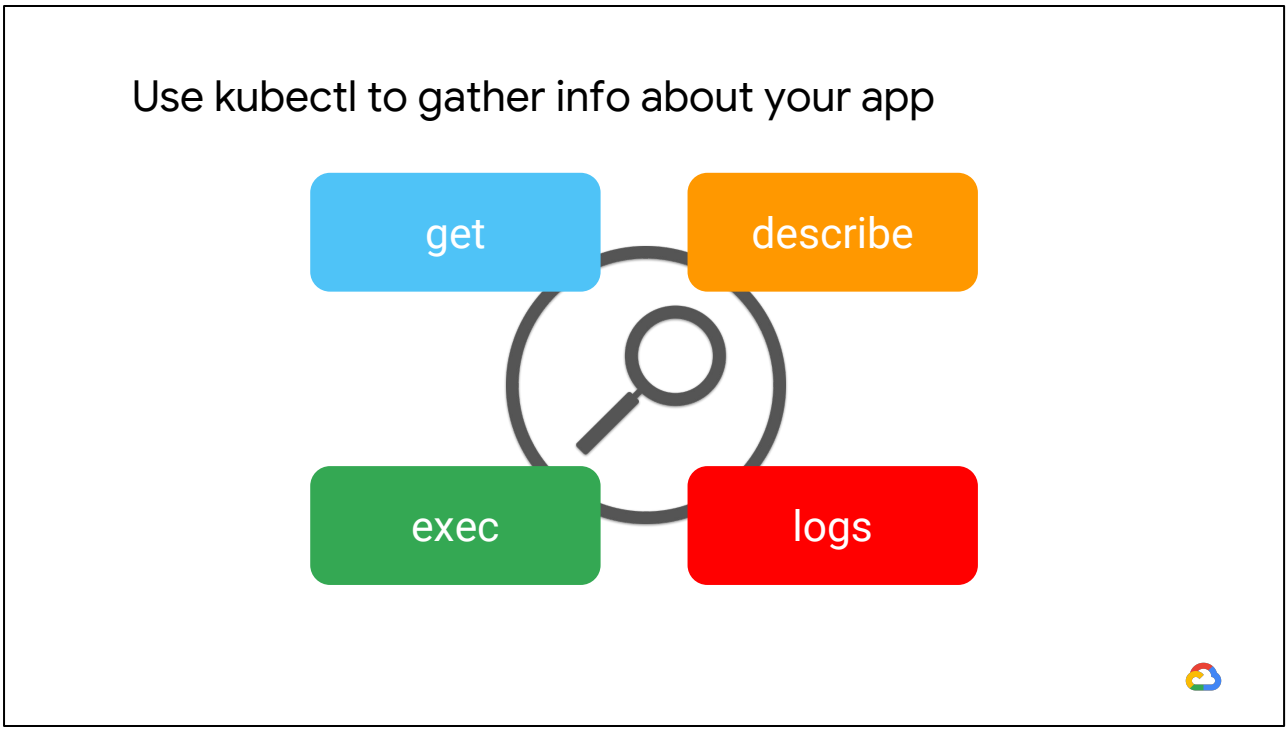
Agenda

The kubectl command

Introspection



In the previous lesson, we discussed the Command part of kubectl's syntax. It's the way to specify the action you want to perform. When your application is running, you'll inevitably need to debug problems with it. Introspection is the act of gathering information about the containers, Pods, Services, and other engines running within the cluster. In this lesson, we'll revisit kubectl's syntax and discuss its get, describe, exec, and logs commands in more detail.



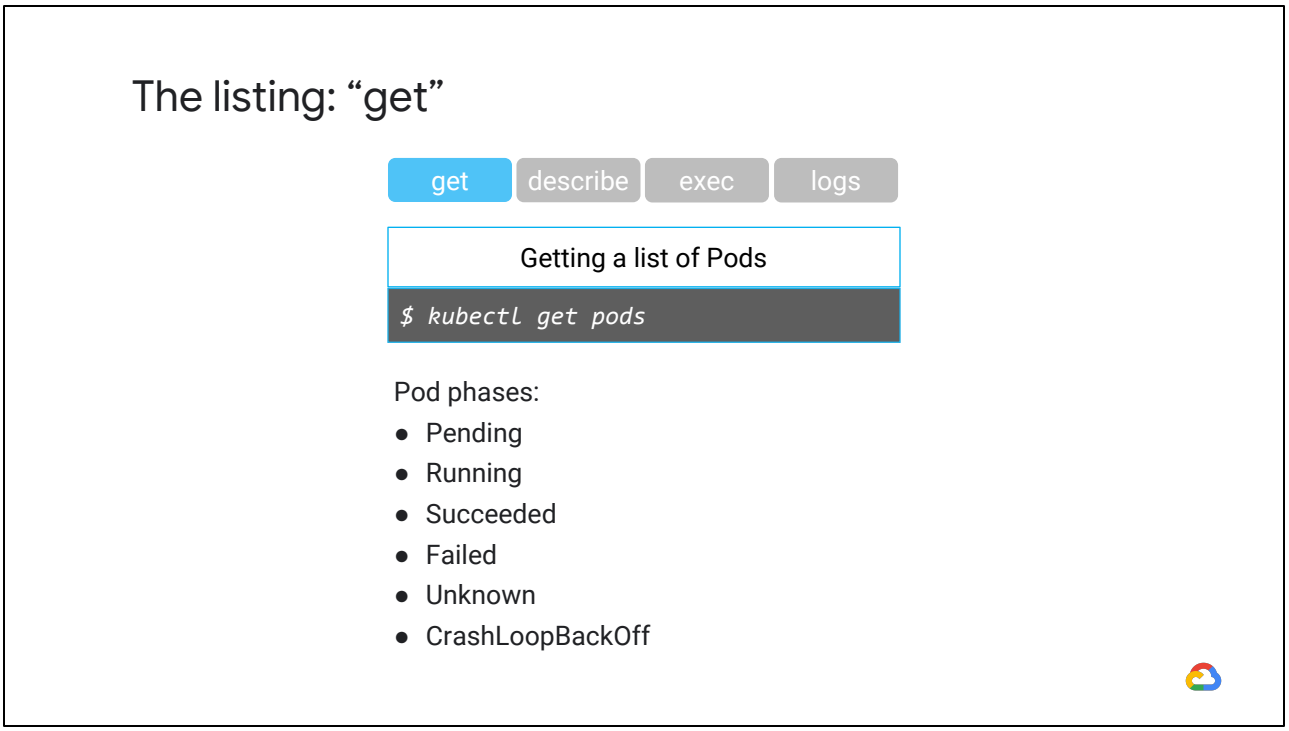
You can retrieve a list of objects using a command like ‘kubectl **get** pods’, which returns a list of all the Pods in the cluster and tells you their status.

For more information on a specific Pod, run the command ‘kubectl **describe** my-pod-name’, which gives you detailed information about the Pod named “my-pod-name”.

You can even test and debug within your Pod using the “kubectl **exec** my-pod-name” command to execute commands and applications.

And the **logs** command provides a powerful tool to see what is happening inside a Pod. Logs are always useful in troubleshooting; the logs command allows you to quickly and easily view errors or debugging messages written out by the applications running inside Pods. The logs command is particularly useful when you need to find out more information about containers that are failing to run successfully.

Now, let’s focus on troubleshooting Kubernetes and GKE. A good place to start is with Pods, the basic units of Kubernetes.



A simple kubectl ‘get pods’ command tells you whether your Pod is running. It shows the Pod’s phase status as *Pending, Running, Succeeded, Failed* or *Unknown*. A Pod’s phase provides a high-level summary of its lifecycle, not the comprehensive details about a Pod or its containers.

Pending means a Pod has been accepted by Kubernetes, but it’s still being scheduled. This means that the container images that are defined for the Pod have not yet been created by the container runtime. For example, when images are being pulled from the repository, the Pod will be in the *Pending* phase.

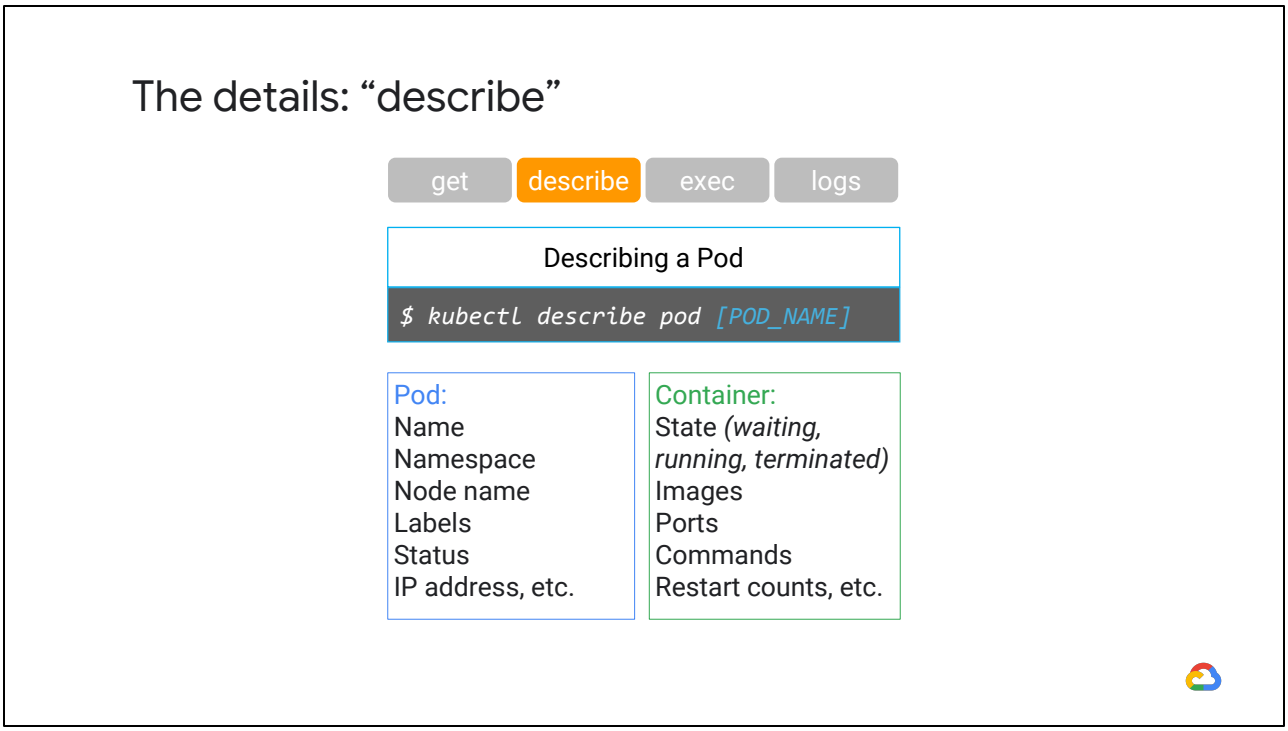
A Pod is *Running* when it has successfully attached to a node and all of its containers have been created. Containers inside a Pod can be starting, restarting, or running continuously.

Succeeded means that all containers have finished running successfully. In other words, they've terminated successfully and they won't be restarting.

Failed means a container has terminated with a failure, and it won't be restarting.

Unknown is where the state of the Pod simply cannot be retrieved, probably because of a communication error between the master and a kubelet. It's not a commonly seen state.

CrashLoopBackOff means that one of the containers in the Pod exited unexpectedly even after it was restarted at least once. This is a common error. Usually, CrashLoopBackOff means that the Pod isn't configured correctly.



To investigate a Pod in more detail, use the kubectl ‘describe pod’ command to fetch details.

This gives you information about a Pod and its containers such as labels, resource requirements, and volumes. It also details the status information about the container and Pod, such as state, readiness, restart count, and events.

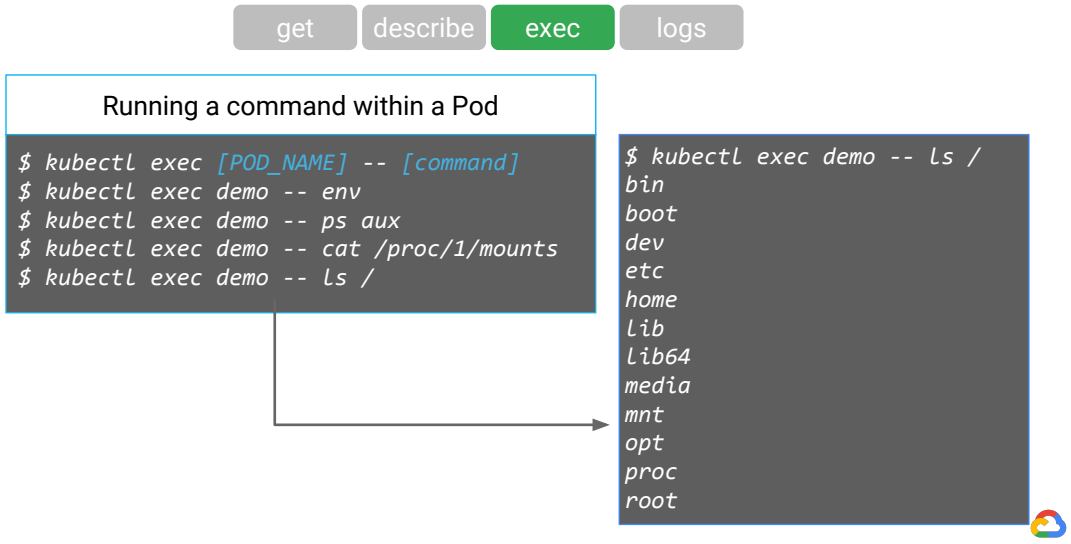
A container state can be either *Waiting*, *Running*, or *Terminated*.

You can view restart counts and configuration options like the restartPolicy. For example, with restartPolicy set to Always, the container will restart continuously.

At the end of the output, you can review events. Imagine a Pod and its container running successfully. You then attempt to update the

container image to a newer version, but the new container image fails to download. In this case, the Pod will change to *Pending* phase, and the container state will become a *Waiting* state, with the reason being “ImagePullBackOff”.

Interaction: “exec”



If you need to troubleshoot an application, you can run a single command in the container, or if you need to do more than that you can get a shell in the container. You might need to troubleshoot connectivity or another application for example.

Single command execution using the exec command as shown here allows you to run a single command inside a container and view the results in your own command shell. This is useful when a single command, such as ping, will do.

Working inside a Pod

Running a command within a Pod

```
$ kubectl exec -it [POD_NAME] -- [command]
```

```
$ kubectl exec -it demo -- /bin/bash
root@demo:/# ls
bin  boot  dev  etc  home  lib  lib64  media  mnt  opt
proc  root  run  sbin  srv  sys  tmp  usr  var
root@demo:/#
```

Pass terminal's standard input (stdin) to the container.

Display the container's standard output (stdout) in your terminal window.

Uses the flags -i and -t (or -it).



If you need to install a package, like a network monitoring tool or a text editor, before you can begin troubleshooting, you can launch an interactive shell using the -it switch which connects your shell to the container so that you can work inside the container.

Note that it isn't a best practice to install software directly into a container. Changes made by containers to their file systems are usually ephemeral. You should build container images that have exactly the software you need, instead of temporarily repairing them at run time. The interactive shell will allow you to figure out what needs to be changed to solve a problem, but you should then integrate those changes into your container images and redeploy them.

This syntax attaches the standard input and standard output of the container to your terminal window or command shell.

The -i argument tells kubectl to pass the terminal's standard input to the container, and the -t argument tells kubectl that the input is a

TTY.

If you don’t use these arguments then the exec command will be executed in the remote container and return immediately to your local shell.

If a Pod has more than one container use the -c argument to specify the specific container to attach to in the Pod.

The history: “logs”

get | describe | exec | **logs**

Getting logs for a Pod

```
$ kubectl logs [POD_NAME]
```

The Pod logs include:

- `stdout`: Standard output on the console
- `stderr`: Error messages



You can also use the kubectl command to view the logs of a Pod. If the Pod has multiple containers, you can use the -c argument to show the logs for a specific container inside the Pod.

The logs contain both the standard output and standard error messages that the applications within the container have generated.

Lab

Deploying Google Kubernetes Engine Clusters from Cloud Shell



In this lab, you'll use the command line to build GKE clusters. You'll inspect the kubeconfig file and use kubectl to manipulate the cluster.

The tasks that you'll learn to perform include using Cloud Shell to deploy GKE clusters, modifying the number of nodes in a GKE cluster, and authenticating to a GKE cluster. You'll then use the Cloud Shell code editor to inspect the kubectl configuration files. You'll also use Cloud Shell and kubectl to inspect a GKE cluster and use Cloud Shell to deploy Pods to GKE clusters. Lastly, you'll introspect GKE Pods by connecting to a Pod to adjust settings, edit files, and make other live changes to the Pod.

Agenda

Kubernetes Concepts

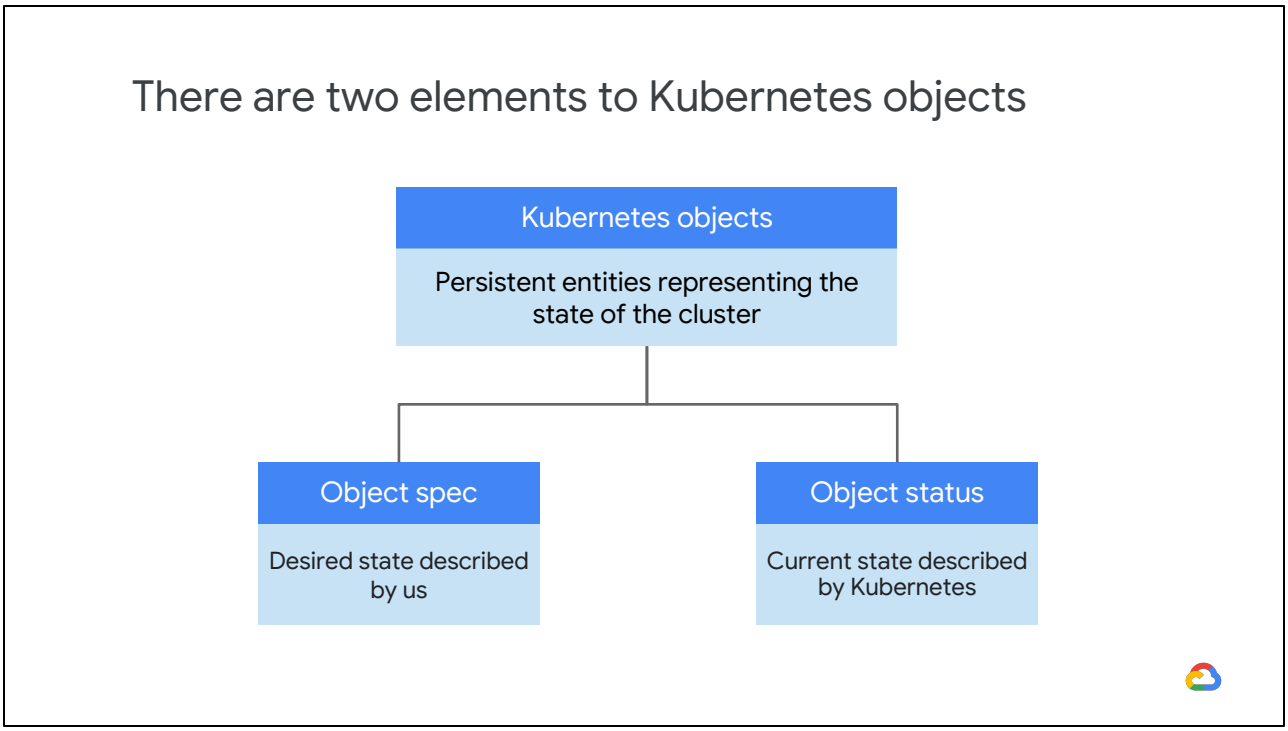
Kubernetes Components

Google Kubernetes Engine Concepts

Object Management



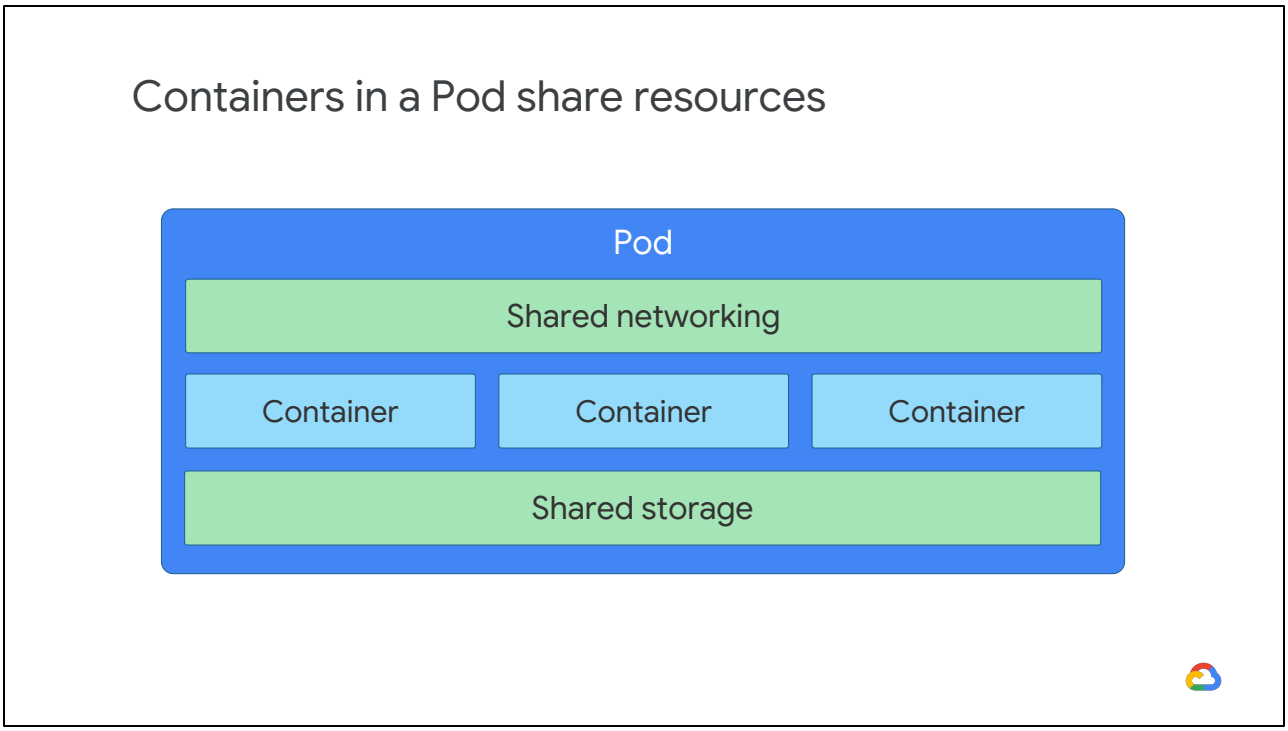
In this lesson, we'll lay out the fundamental components of the Kubernetes operating philosophy.
To understand how Kubernetes works, there are two related concepts you need to understand. The first is the Kubernetes object model. Each thing Kubernetes manages is represented by an object, and you can view and change these objects' attributes and state. The second is the principle of declarative management. Kubernetes expects you to tell it what you want the state of the objects under its management to be; it will work to bring that state into being and keep it there. How does it do that? By means of its so-called "watch loop."



Formally, a Kubernetes object is defined as a persistent entity that represent the state of something running in a cluster: its desired state and its current state. Various kinds of objects represent containerized applications, the resources that are available to them, and the policies that affect their behavior. Kubernetes objects have two important elements.

You give Kubernetes an Object *spec* for each object you want to create. With this spec, you define the desired state of the object by providing the characteristics that you want.

The Object *status* is simply the current state of the object provided by the Kubernetes control plane. By the way, we use this term “Kubernetes control plane” to refer to the various system processes that collaborate to make a Kubernetes cluster work. You’ll learn about these processes later in this module.



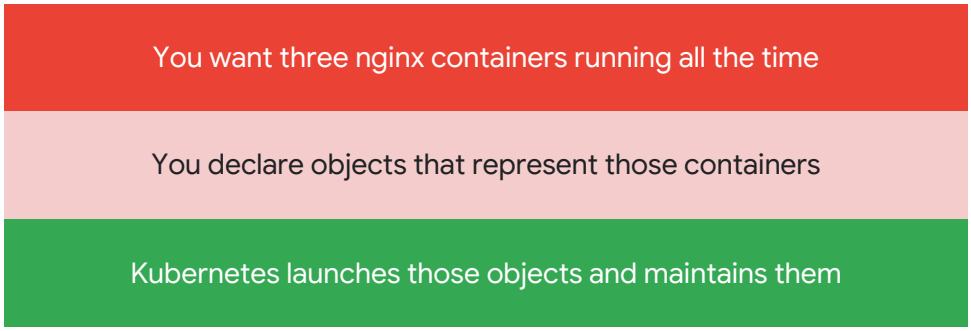
Each object is of a certain type, or “Kind,” as Kubernetes calls them. Pods are the basic building block of the standard Kubernetes model, and they’re the smallest deployable Kubernetes object. Maybe you were expecting me to say that the smallest Kubernetes object is the container. Not so. Every running container in a Kubernetes system is in a Pod.

A Pod embodies the environment where the containers live, and that environment can accommodate one *or more* containers.

If there is more than one container in a pod, they are tightly coupled and share resources including networking and storage. Kubernetes assigns each Pod a unique IP address. Every container within a Pod shares the network namespace, including IP address and network ports. Containers within the same Pod can communicate through localhost, 127.0.0.1. A Pod can also specify a set of storage Volumes, to be shared among its containers. (By the way,

later in this specialization, you'll learn how Pods can share storage with one another, not just within a single Pod.)

Running three nginx containers

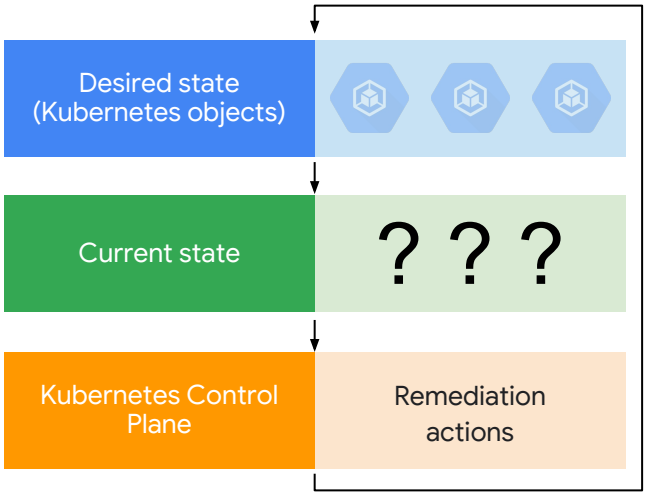


Let's consider a simple example where you want three instances of the nginx Web server, each in its own container, running all the time.

How is this achieved in Kubernetes? Remember that Kubernetes embodies the principle of declarative management. You declare some objects to represent those nginx containers. What Kind of object? Perhaps Pods.

Now it is Kubernetes's job to launch those Pods and keep them in existence. Be careful: Pods are not self-healing. If we want to keep all our nginx Web servers not just in existence but also working together as a team, we might want to ask for them using a more sophisticated Kind of object. I'll tell you how later in this module.

Desired state compared to current state



Let's suppose we have given Kubernetes a desired state that consists of three nginx Pods, always kept running. We did this by telling Kubernetes to create and maintain one or more objects that represent them.

Now Kubernetes compares the desired state to the current state. Let's imagine that our declaration of three nginx containers is completely new. The current state does not match the desired state.

So Kubernetes, specifically its control plane, will remedy the situation. Because the number of desired Pods running for the object we declared is 3, and 0 are presently running, 3 will be launched.

And the Kubernetes control plane will continuously monitor the state of the cluster, endlessly comparing reality to what has been declared, and remedying the state as needed.

Agenda

Kubernetes Concepts

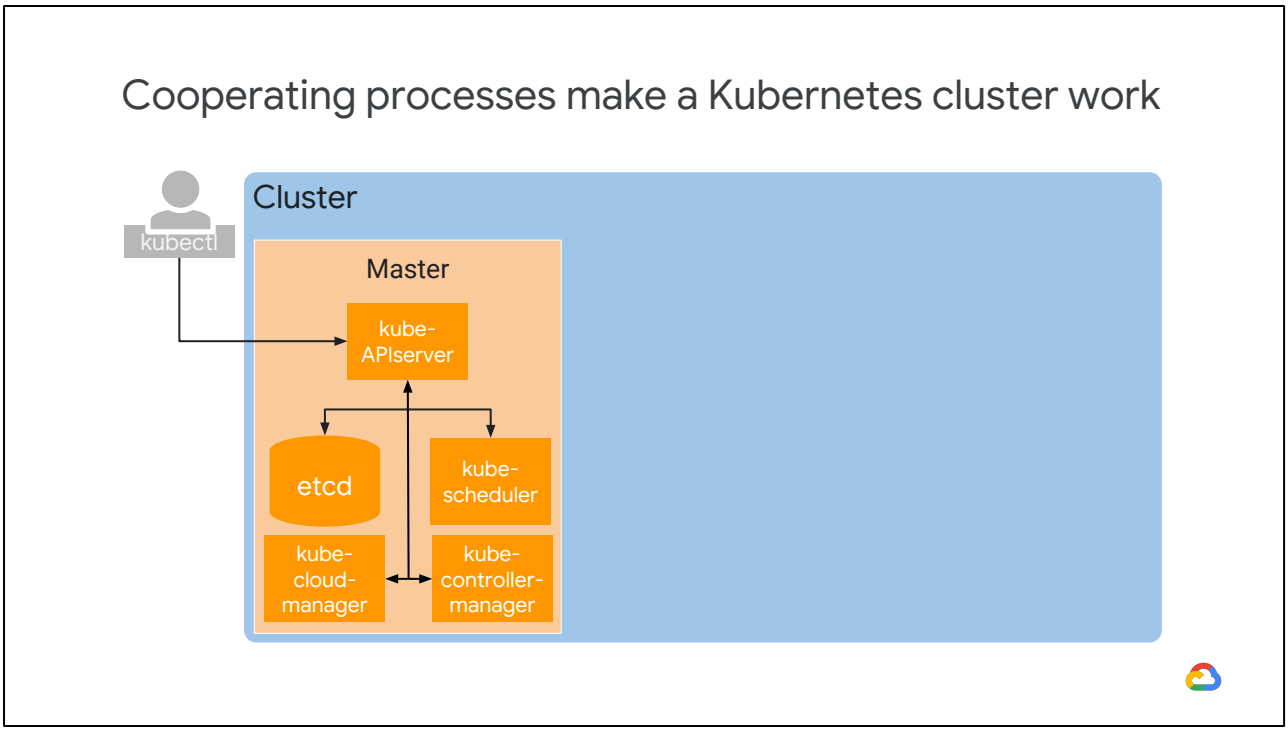
Kubernetes Components

Google Kubernetes Engine Concepts

Object Management



In the previous lesson, I mentioned the Kubernetes control plane, which is the fleet of cooperating processes that make a Kubernetes cluster work. Even though you'll only work directly with a few of these components, it helps to know about them and the role each plays. I'll build up a Kubernetes cluster part by part, explaining each piece as I go. After I'm done, I'll show you how a Kubernetes cluster running in GKE is a lot less work to manage than one you provisioned yourself.



First and foremost, your cluster needs computers. Nowadays the computers that compose your clusters are usually virtual machines. They *always* are in GKE, but they could be physical computers too. One computer is called the “master,” and the others are called simply “nodes.” The job of the nodes is to run Pods. The job of the master is to coordinate the entire cluster. We will meet its control-plane components first.

Several critical Kubernetes components run on the master. The single component that you interact with directly is the kube-apiserver. This component’s job is to accept commands that view or change the state of the cluster, including launching Pods.

In this specialization, you will use the kubectl command frequently; this command’s job is to connect to kube-apiserver and communicate with it using the Kubernetes API. kube-apiserver also

authenticates incoming requests, determines whether they are authorized and valid, and manages admission control. But it's not just kubectl that talks with kube-apiserver. In fact, any query or change to the cluster's state must be addressed to the kube-apiserver.

etcd is the cluster's database. Its job is to reliably store the state of the cluster. This includes all the cluster configuration data; and more dynamic information such as what nodes are part of the cluster, what Pods should be running, and where they should be running. You never interact directly with etcd; instead, kube-apiserver interacts with the database on behalf of the rest of the system.

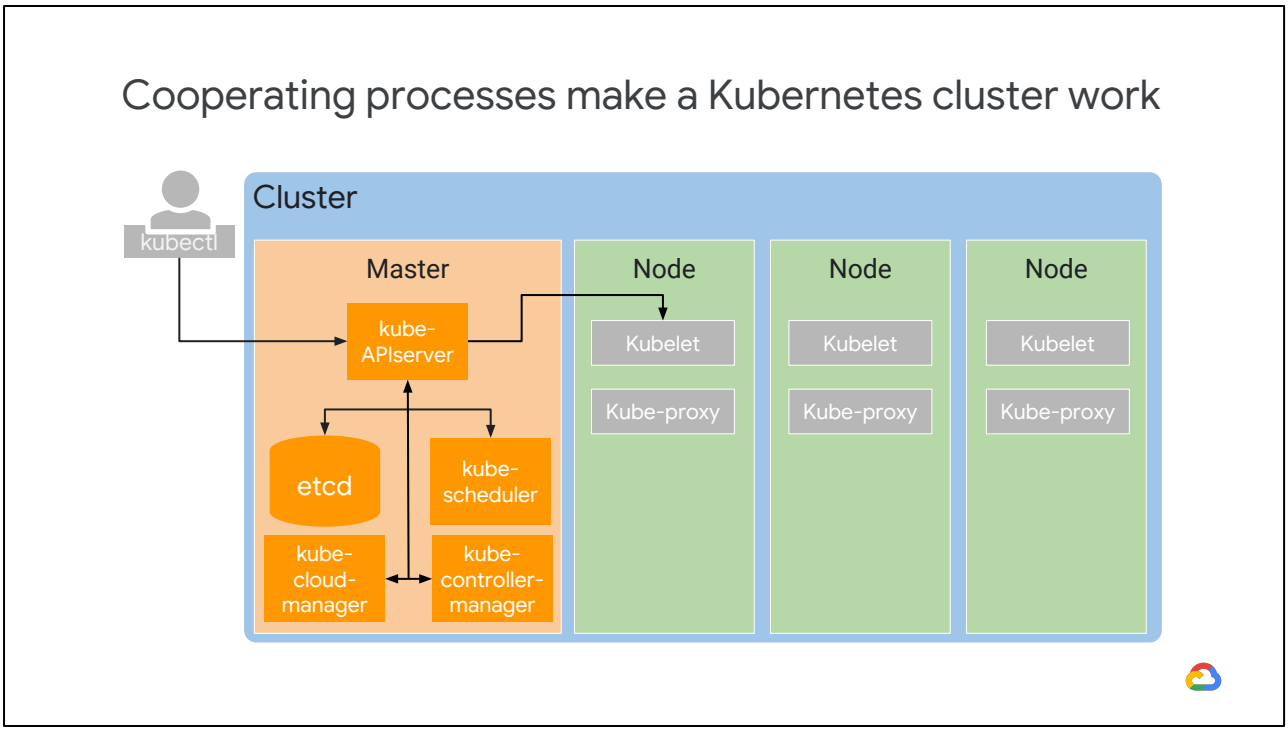
kube-scheduler is responsible for scheduling Pods onto the nodes. To do that, it evaluates the requirements of each individual Pod and selecting which node is most suitable. But it doesn't do the work of actually launching Pods on Nodes. Instead, whenever it discovers a Pod object that doesn't yet have an assignment to a node, it chooses a node and simply writes that name of that node into the Pod object. Another component of the system is responsible for then launching the Pods, and you will see it very soon.

But how does kube-scheduler decide where to run a Pod? It knows the state of all the nodes, and it will also obey constraints that you define on where a Pod may run, based on hardware, software, and policy. For example, you might specify that a certain Pod is only allowed to run on nodes with a certain amount of memory. You can also define affinity specifications, which cause groups of pods to prefer running on the same node; or anti-affinity specifications, which ensure that pods do not run on the same node. You will learn

more about some of these tools in later modules.

kube-controller-manager has a broader job. It continuously monitors the state of a cluster through Kube-APIserver. Whenever the current state of the cluster doesn't match the desired state, kube-controller-manager will attempt to make changes to achieve the desired state. It's called the "controller manager" because many Kubernetes objects are maintained by loops of code called controllers. These loops of code handle the process of remediation. Controllers will be very useful to you. To be specific, you'll use certain kinds of Kubernetes controllers to manage workloads. For example, remember our problem of keeping 3 nginx Pods always running. We can gather them together into a controller object called a Deployment that not only keeps them running but also lets us scale them and bring them together underneath a front end. We'll meet Deployments later in this module.

Other kinds of controllers have system-level responsibilities. For example, Node Controller's job is to monitor and respond when a node is offline.
kube-cloud-manager manages controllers that interact with underlying cloud providers. For example, if you manually launched a Kubernetes cluster on Google Compute Engine, kube-cloud-manager would be responsible for bringing in GCP features like load balancers and storage volumes when you needed them.



Each node runs a small family of control-plane components too.

For example, each node runs a kubelet. You can think of kubelet as Kubernetes's agent on each node. When the kube-apiserver wants to start a Pod on a node, it connects to that node's kubelet. Kubelet uses the container runtime to start the Pod and monitors its lifecycle, including readiness and liveness probes, and reports back to Kube-APIserver. Do you remember our use of the term "container runtime" in the previous module? This is the software that knows how to launch a container from a container image. The world of Kubernetes offers several choices of container runtimes, but the Linux distribution that GKE uses for its nodes launches containers using containerd, the runtime component of Docker.

kube-proxy's job is to maintain network connectivity among the Pods in a cluster. In open-source Kubernetes, it does so using the

firewalling capabilities of iptables, which are built into the Linux kernel. Later in this specialization, we will learn how GKE handles pod networking.

Agenda

Kubernetes Concepts

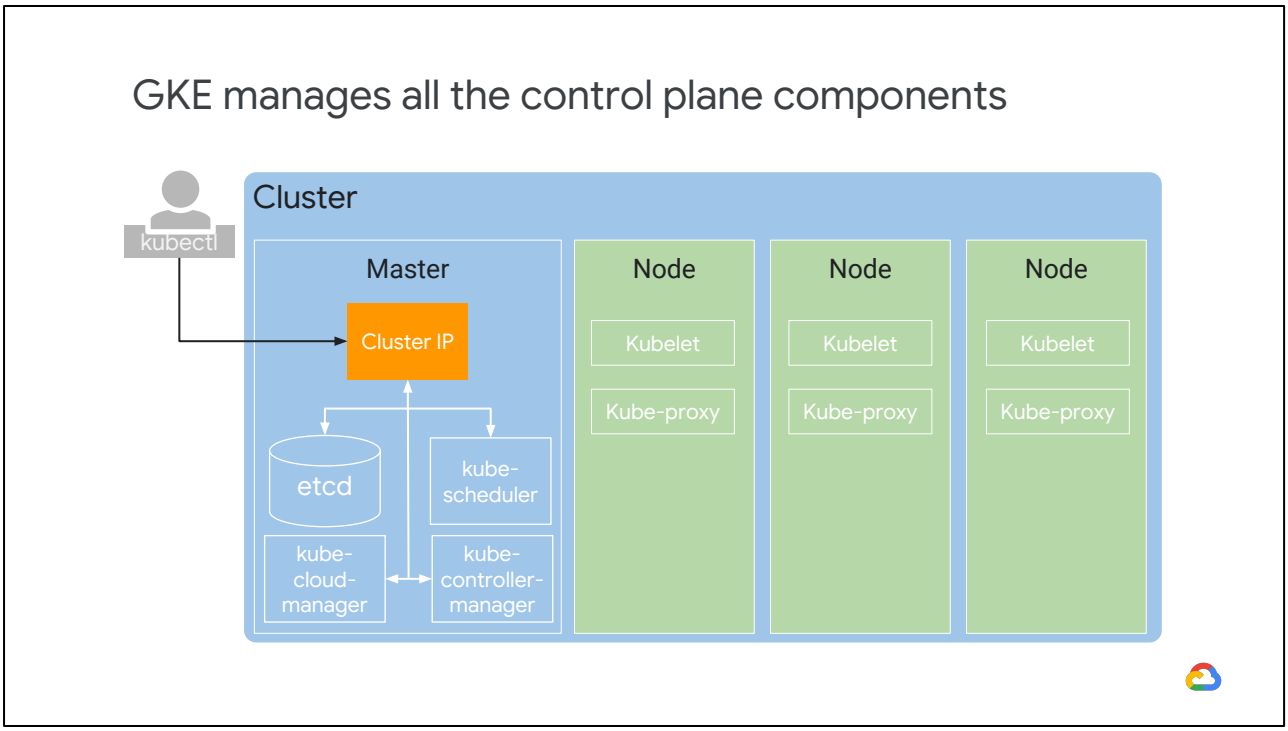
Kubernetes Components

Google Kubernetes Engine Concepts

Object Management

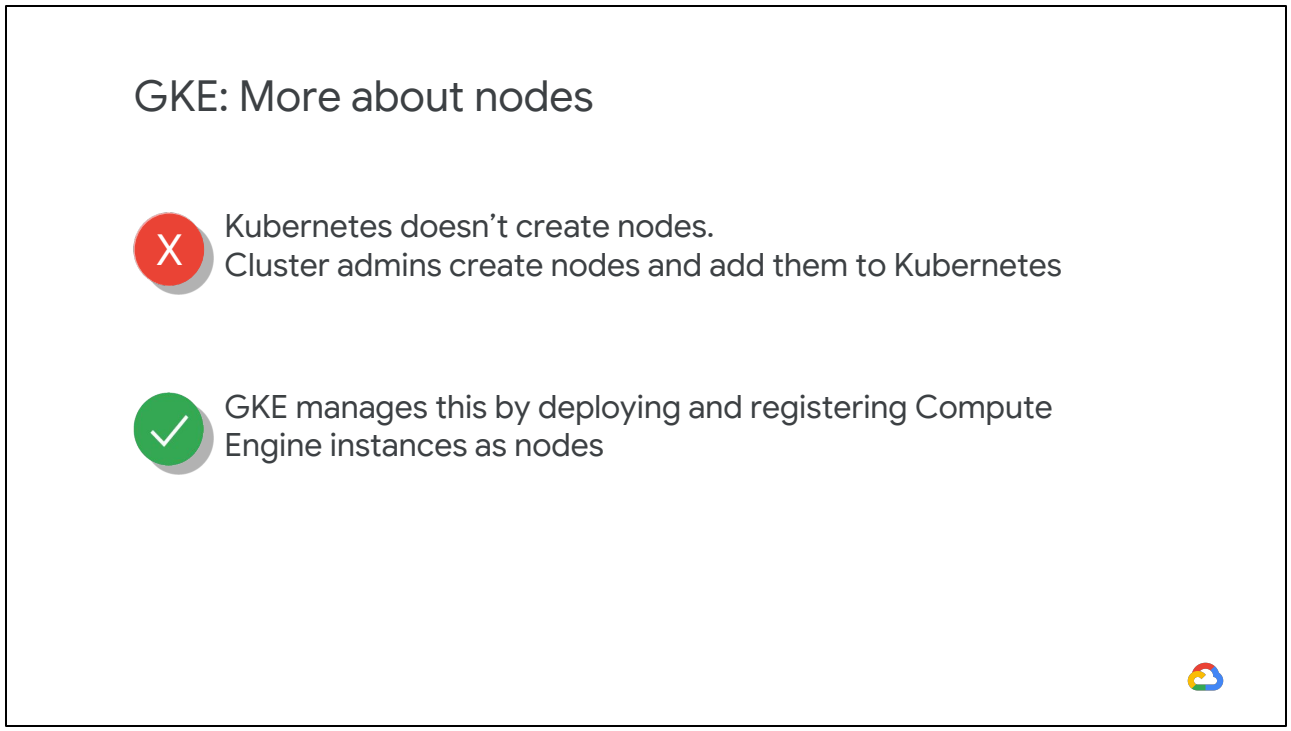


Next, we'll introduce concepts specific to Google Kubernetes Engine. That diagram of the Kubernetes control plane had a lot of components, didn't it? Setting up a Kubernetes cluster by hand is tons of work.



Fortunately, there is an open-source command called kubeadm that can automate much of the initial setup of a cluster. But if a node fails or needs maintenance, a human administrator has to respond manually. I suspect you can see why many people like the idea of a managed service for Kubernetes. You may be wondering how that picture we just saw differs for GKE. Well, here it is:

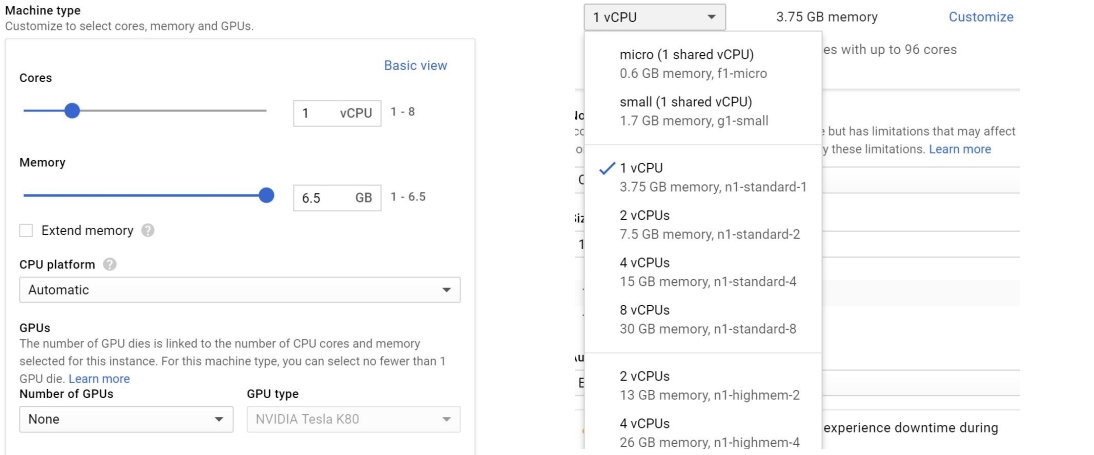
From the user's perspective, it's a lot simpler. GKE manages all the control plane components for us. It still exposes an IP address to which we send all of our Kubernetes API requests, but GKE takes responsibility for provisioning and managing all the master infrastructure behind it. It also abstracts away having a separate master. The responsibilities of the master are absorbed by GCP, and you are not separately billed for your master.



Now let's talk about nodes. In any Kubernetes environment, nodes are created externally by cluster administrators, not by Kubernetes itself.

GKE automates this process for you. It launches Compute Engine virtual machine instances and registers them as nodes. You can manage node settings directly from the GCP Console. You are charged per second of allocated time for your nodes (not counting the master).

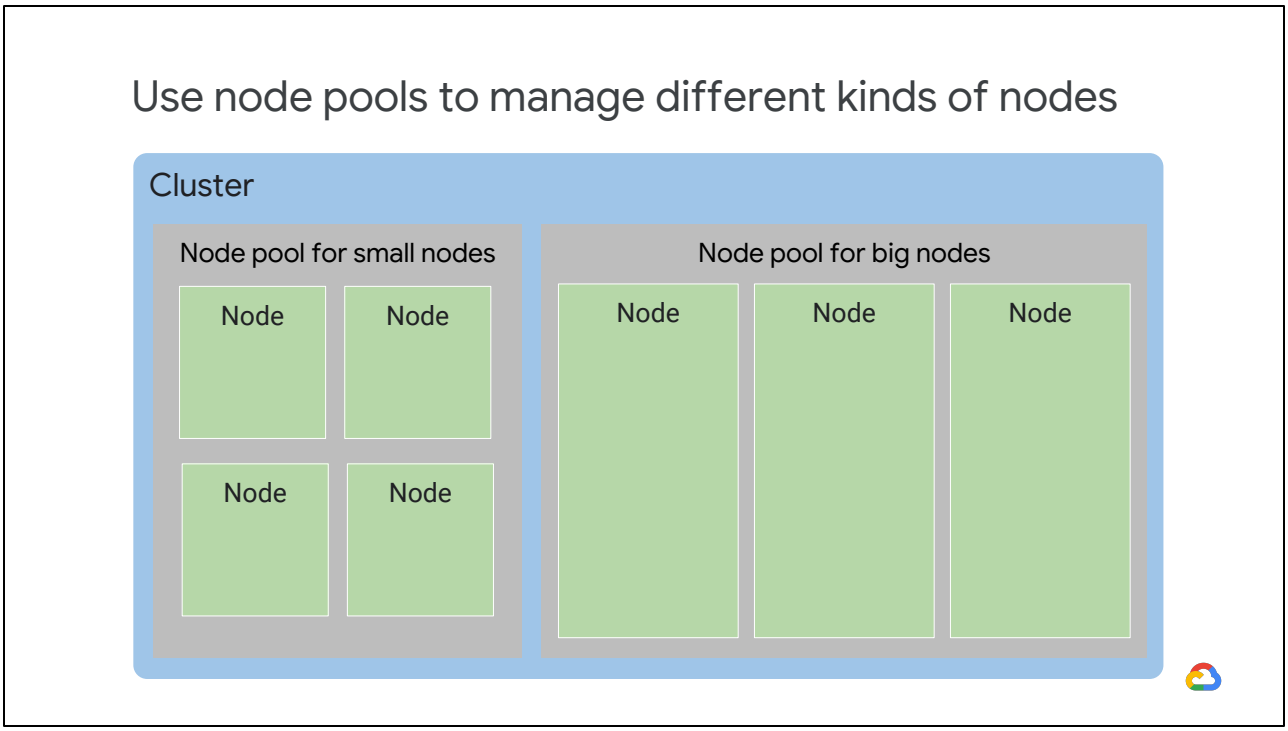
GKE: More about nodes



Because nodes run on Compute Engine, you choose your node machine type when you create your cluster. By default, the node machine type is n1-standard-1, which providing 1 vCPU and 3.75 gigabytes of memory. Google Cloud offers a wide variety of Compute Engine options. At the time this course was developed, the generally available maximum was 96 vCPU cores. That's a moderately big virtual machine.

You can customize your nodes' number of cores and their memory capacity. You can select a CPU platform.

You can choose a baseline minimum CPU platform for the nodes or node pool. This allows you to improve node performance. GKE will never use a platform that is older than the CPU platform you specify, and if it picks a newer platform, the cost will be same as the specified platform.



You can also select multiple node machine types by creating multiple node pools. A node pool is a subset of nodes within a cluster that share a configuration, such as their amount of memory, or their CPU generation. Node pools also provide an easy way to ensure that workloads run on the right hardware within your cluster: you just label them with a desired node pool.

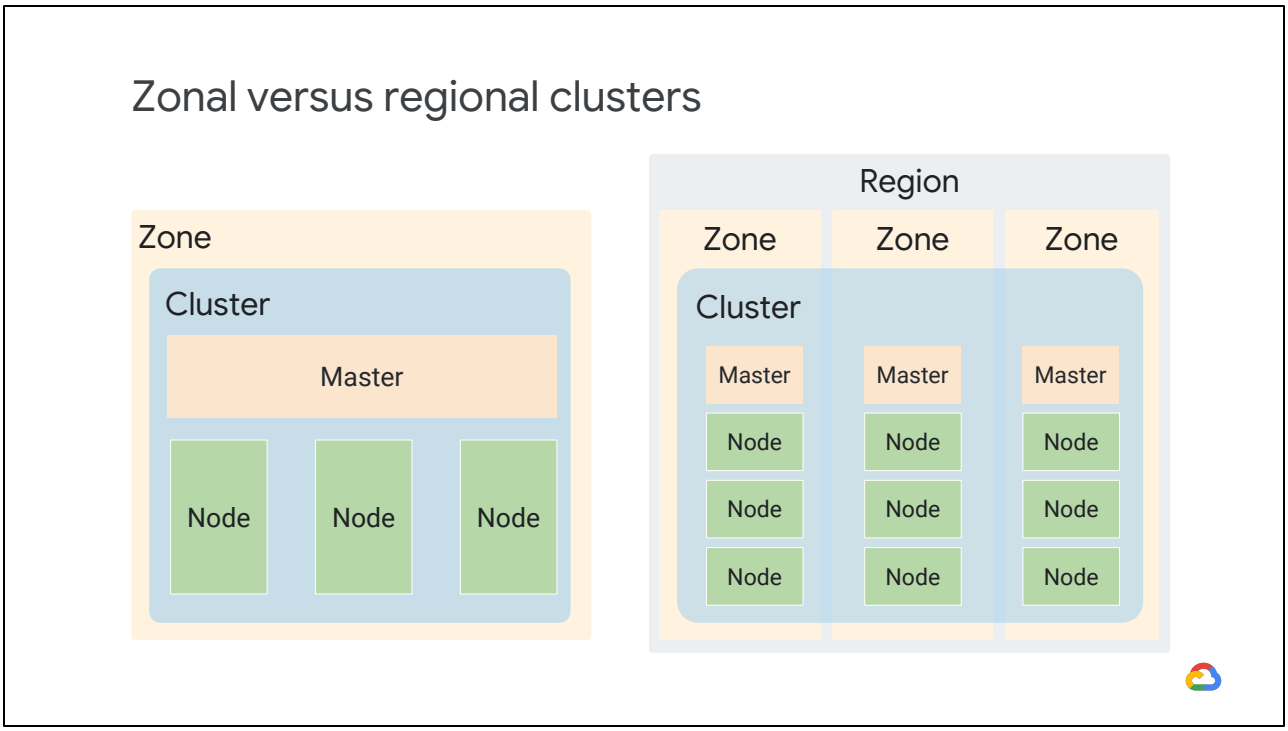
By the way, node pools are a GKE feature rather than a Kubernetes feature. You can build an analogous mechanism within open-source Kubernetes, but you would have to maintain it yourself.

You can enable automatic node upgrades, automatic node repairs, and cluster autoscaling at this node pool level.

Here's a word of caution. Some of each node's CPU and memory are needed to run the GKE and Kubernetes components that let it work as part of your cluster. So, for example, if you allocate nodes with 15 gigabytes of memory, not quite all of that 15 gigabytes will

be available for use by Pods. This module has a documentation link that explains how much CPU and memory are reserved.

https://cloud.google.com/kubernetes-engine/docs/concepts/cluster-architecture

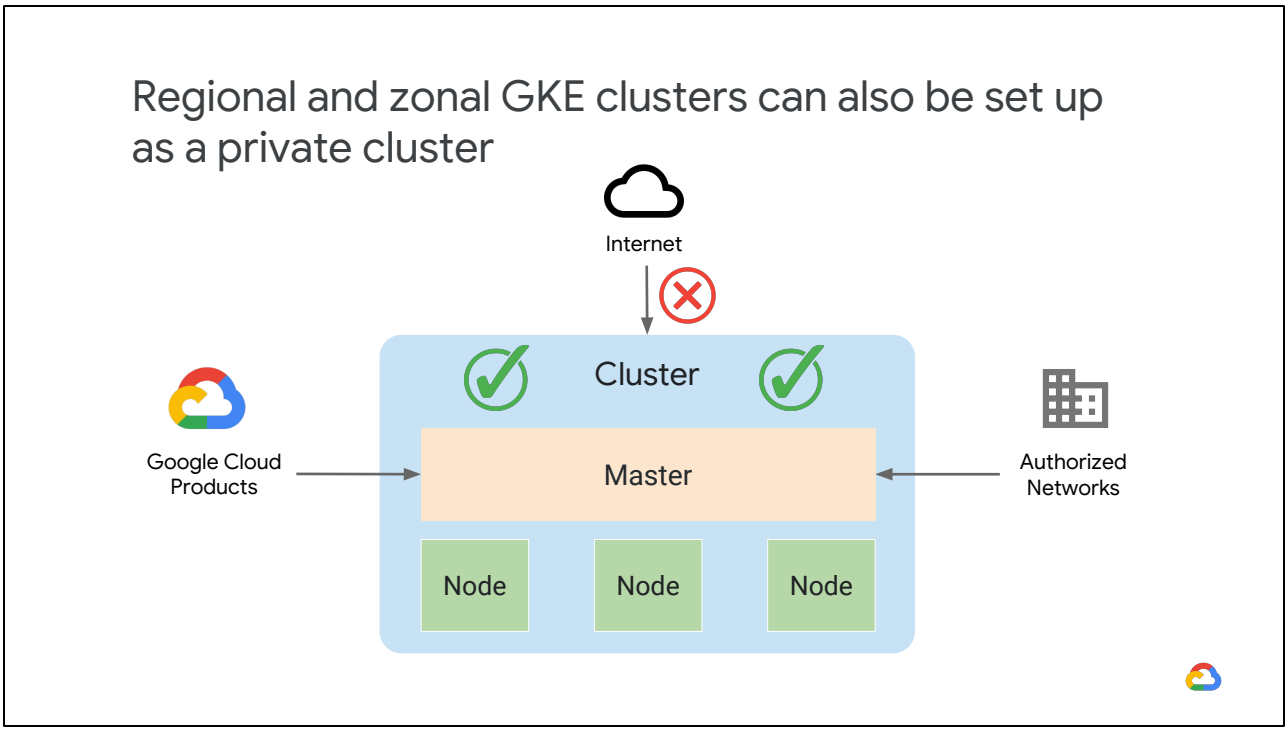


By default, a cluster launches in a single GCP compute zone with three identical nodes, all in one node pool. The number of nodes can be changed during or after the creation of the cluster. Adding more nodes and deploying multiple replicas of an application will improve an application's availability. But only up to a point. What happens if the entire compute zone goes down?

You can address this concern by using a GKE regional cluster. Regional clusters have a single API endpoint for the cluster. However, its masters and nodes are spread across multiple Compute Engine zones within a region.

Regional clusters ensure that the availability of the application is maintained across multiple zones in a single region. In addition, the availability of the master is also maintained so that both the application and management functionality can withstand the loss of one or more, but not all, zones. By default, a regional cluster is

spread across 3 zones, each containing 1 master and 3 nodes. These numbers can be increased or decreased. For example, if you have five nodes in Zone 1, you will have exactly the same number of nodes in each of the other zones, for a total of 15 nodes. Once you build a zonal cluster, you can’t convert it into a regional cluster, or vice versa.



The entire cluster (that is, the master and its nodes) are hidden from the public internet.

Cluster masters can be accessed by Google Cloud products, such as Stackdriver, through an internal IP address.

They can also be accessed by authorized networks through an external IP address. Authorized networks are basically IP address ranges that are trusted to access the master. In addition, nodes can have limited outbound access through Private Google Access, which allows them them to communicate with other GCP services. For example, nodes can pull container images from Google Container Registry without needing external IP addresses. The topic of private clusters is discussed in more detail in another module in this specialization.

Agenda

Kubernetes Concepts

Kubernetes Components

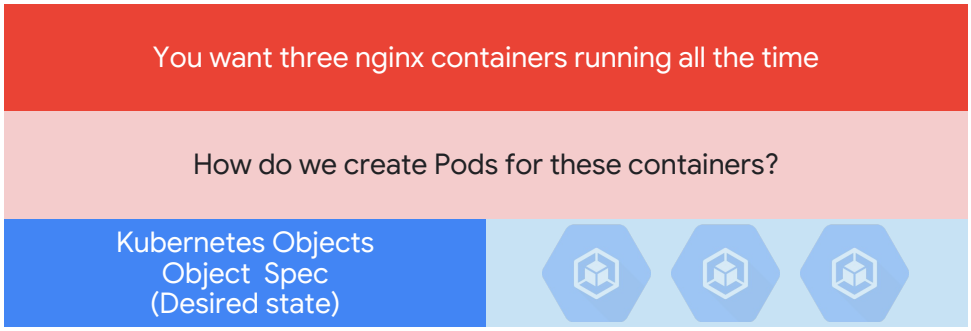
Google Kubernetes Engine Concepts

Object Management



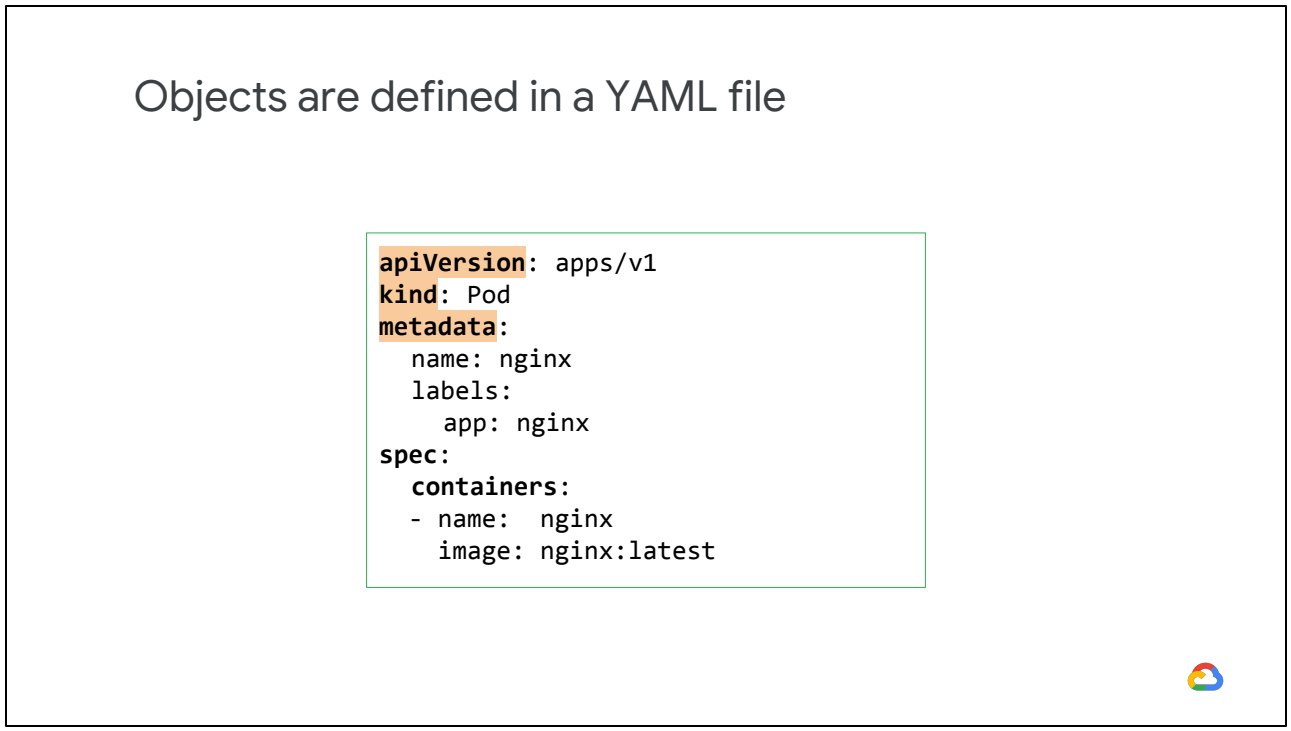
Finally, we'll discuss Kubernetes object management. All Kubernetes objects are identified by a unique name and a unique identifier.

Running three nginx containers



Let's return once again to our example, in which we want three nginx Web servers running all the time.

Well, the simplest way would before us to declare three Pod objects and specify their state: that, for each, a Pod must be created and an nginx container image must be used. Let's see how we declare this.

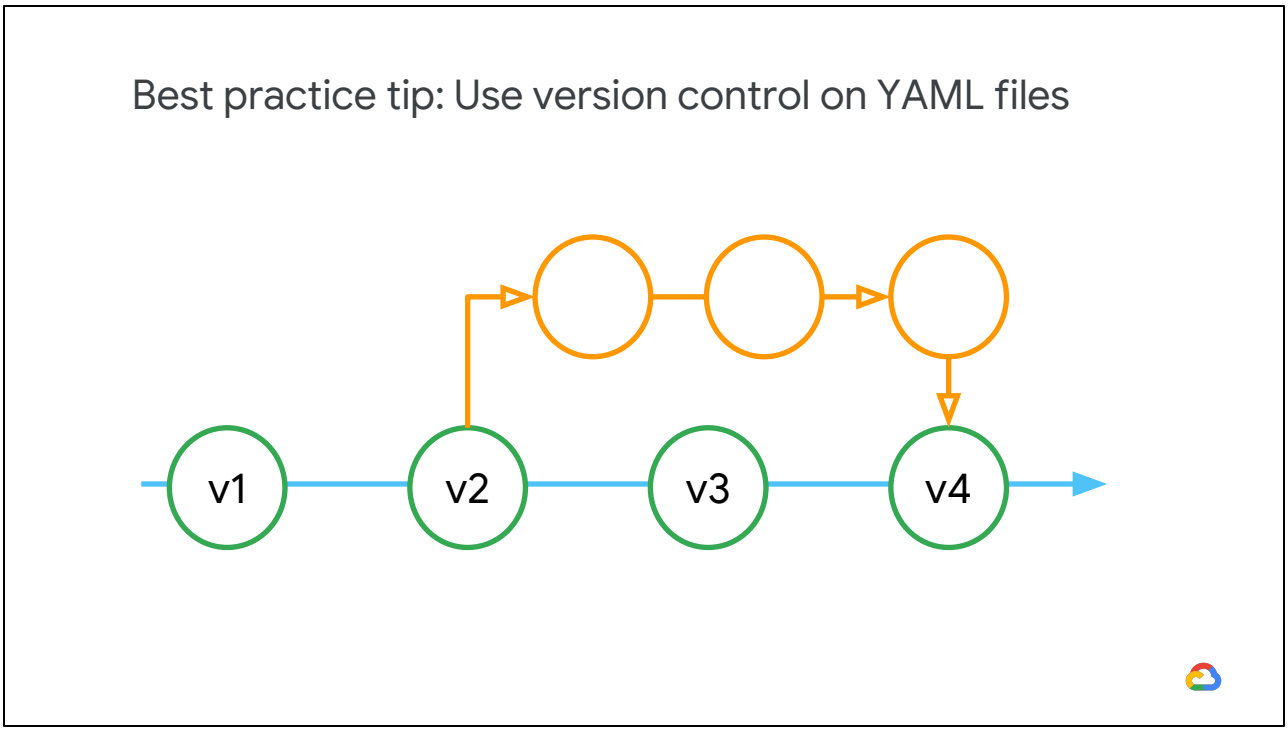


You define the objects you want Kubernetes to create and maintain with manifest files. These are ordinary text files. You may write them in YAML or JSON format. YAML is more human-readable and less tedious to edit, and we will use it throughout this specialization. This YAML file defines a desired state for a pod: its name and a specific container image for it to run.

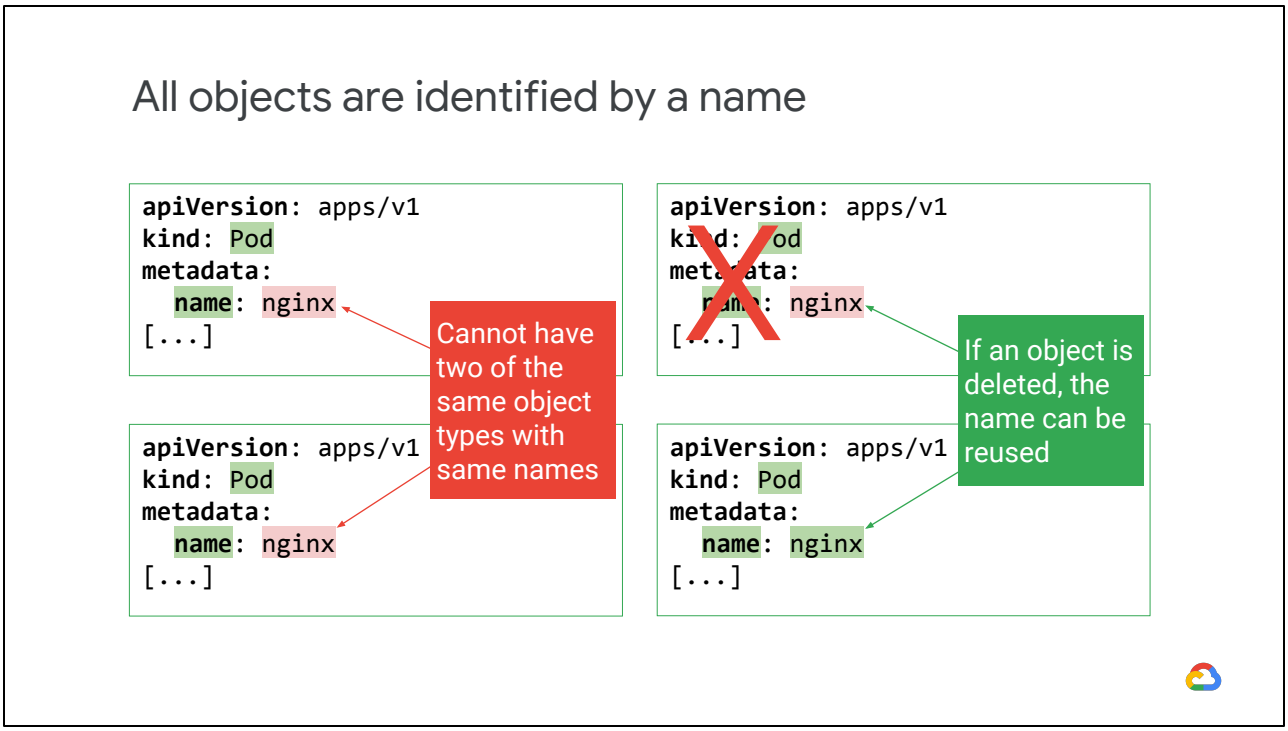
Your manifest files have certain required fields. ApiVersion describes which Kubernetes API version is used to create the object. The Kubernetes protocol is versioned so as to help maintain backwards compatibility.

Kind identifies the object you want (in this case a Pod) and Metadata helps identify the object using Name, Unique ID, and an optional Namespace. You can define several related objects in the same YAML file, and it is a best practice to do so. One file is often

easier to manage than several.



Another, even more important tip: You should save your YAML files in version-controlled repositories. This practice makes it easier to track and manage changes and to back-out those changes when necessary. It's also a big help when you need to recreate or restore a cluster. Many GCP customers use Cloud Source Repositories for this purpose, because that service lets them control the permissions of those files in the same way as their other GCP resources.



When you create a Kubernetes object, you name it with a string. Names must be unique. Only one object of a particular kind can have a particular name at the same time in the same Kubernetes namespace. However, if an object is deleted, its name can be reused. Alphanumeric characters, hyphens, and periods are allowed in the names, with a maximum character length of 253.

All objects are assigned a unique identifier (UID) by Kubernetes

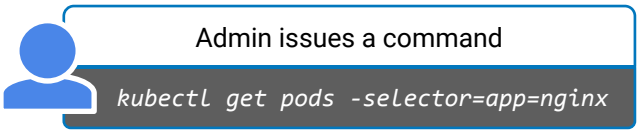
```
apiVersion: apps/v1
kind: Pod
metadata:
  name: nginx
   uid: 4dd474fn-f389-11f8-b38c-42010a8009z7
[...]
```



Every object created throughout the life of a cluster has a unique UID generated by Kubernetes. This means that no two objects will have same UID throughout the life of a cluster.

Labels can be matched by label selectors

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx
  labels:
    app: nginx
    env: dev
    stack: frontend
spec:
  replicas: 3
  selector:
    matchLabels
    app: nginx
```





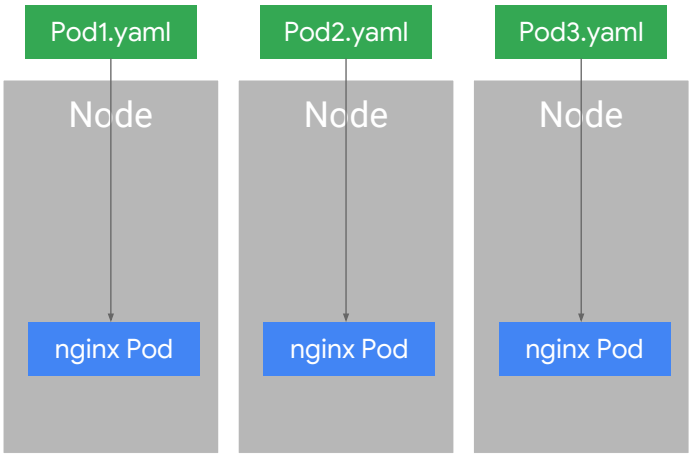
Labels are key-value pairs with which you tag your objects during or after their creation. Labels help you identify and organize objects and subsets of objects. For example, you could create a label called “app” and give as its value the application of which this object is a part.

In this simple example, a Deployment object is labeled with three different key-values: its application, its environment, and which stack it forms a part of.

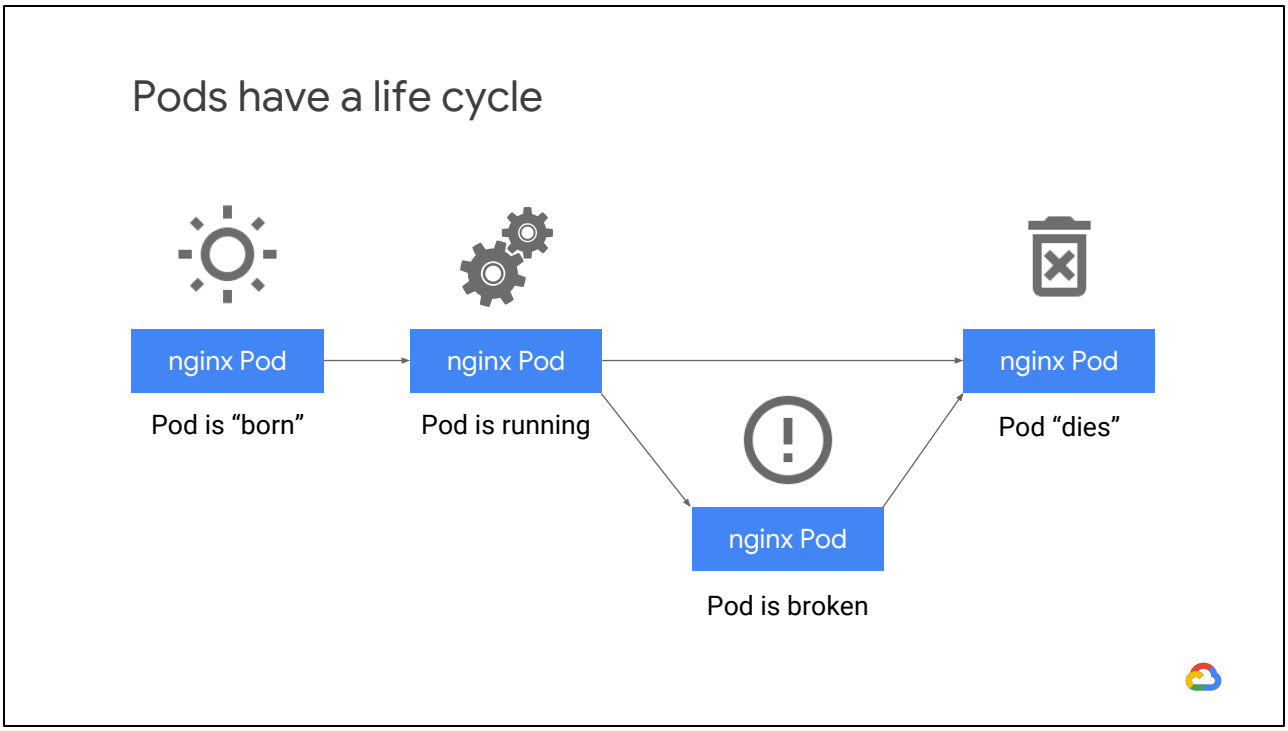
Various contexts offer ways to select Kubernetes resources by their labels. In this specialization, you will spend plenty of time with the kubectl command; here’s an example of using it to show all the pods that contain a label called “app” with a value of “nginx.” Label selectors are very expressive. You can ask for all the resources that have a certain value for a label, all those that don’t have a

certain value, or even all those that have a value in a set you supply.

A workload is spread evenly across available nodes by default



So one way to bring three nginx Web servers into being would be to declare three Pod objects, each with its own section of YAML. Kubernetes's default scheduling algorithm prefers to spread the workload evenly across the nodes available to it, so we'd get a situation like this one. Looks good, doesn't it? Maybe not. Suppose I want 200 more nginx instances. Managing 200 more sections of YAML sounds very inconvenient.

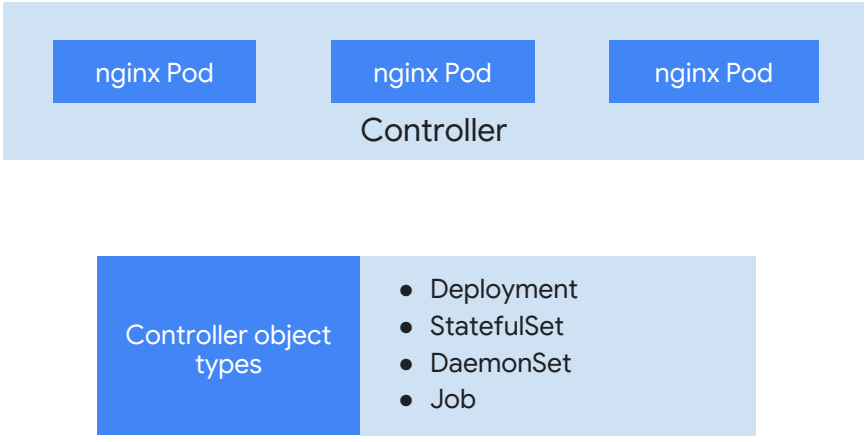


Here’s another problem: Pods don’t heal or repair themselves, and are not meant to run forever. They are designed to be ephemeral and disposable.

For these reasons, there are better ways to manage what you run in Kubernetes than specifying individual Pods. You need a setup like this to maintain an application’s high availability along with horizontal scaling.

So how do you tell Kubernetes to maintain the desired state of three nginx containers?
Image source: Gears
https://svgsilh.com/search/clock-1.html

Pods and Controller Objects



We can instead declare a controller object whose job is to manage the state of the Pods. Some examples of these objects: Deployments, StatefulSets, DaemonSets, and Jobs. We'll meet all of these in our specialization.

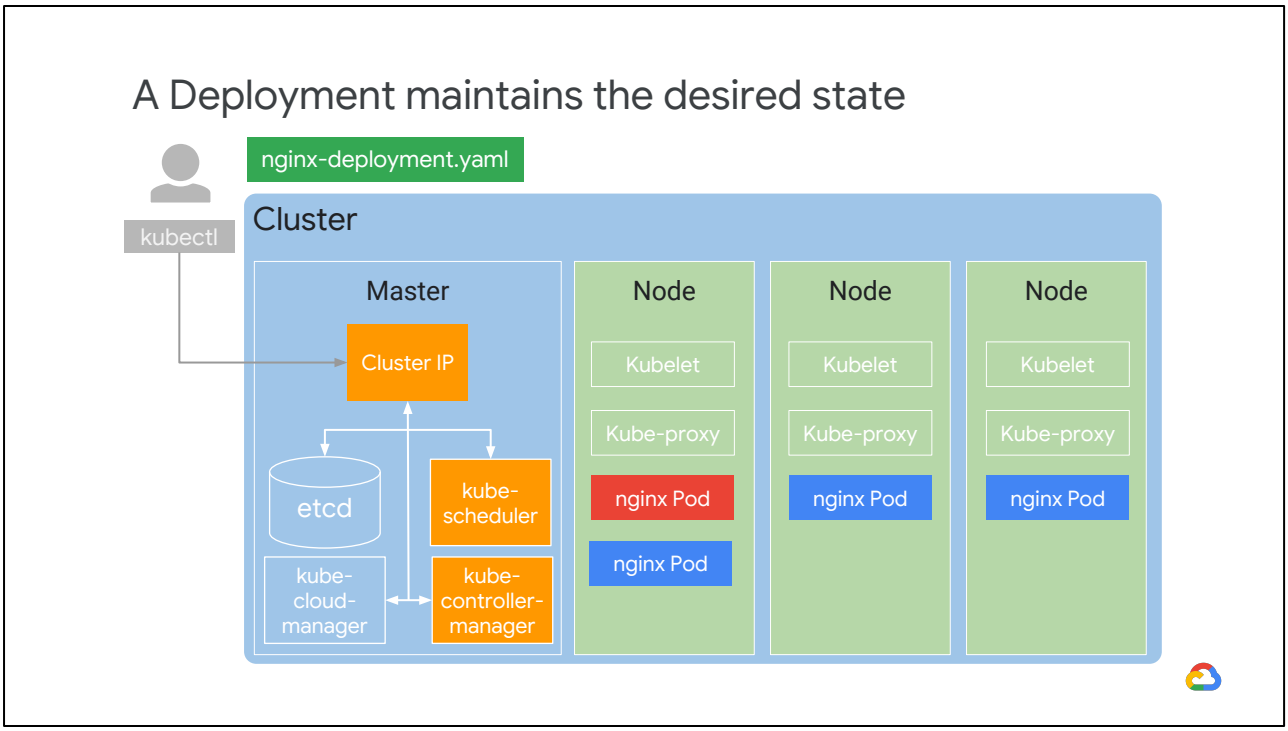
Deployments are a great choice for long-lived software components

You want three nginx containers running all the time

How does Kubernetes maintain 3 nginx containers at any given time?



Deployments are a great choice for long-lived software components like Web servers, especially when we want to manage them as a group.



In our example, when Kube-scheduler schedules Pods for a Deployment, it notifies the Kube-APIserver.
These changes are constantly monitored by controllers—especially by the Deployment controller. The practical effect of the Deployment controller is to monitor and maintain 3 nginx Pods.

The Deployment controller creates a child object, a ReplicaSet, to launch the desired Pods. If one of these Pods fails, the ReplicaSet controller will recognize the difference between the current state and the desired state and will try to fix it by launching a new Pod. Instead of using multiple yaml manifests or files for each Pod, you used a single Deployment yaml to launch 3 replicas of the same container.

Deployments ensure that sets of Pods are running

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: nginx
spec:
  replicas: 3
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:latest
```



A Deployment ensures that a defined set of Pods is running at any given time.

Within its object spec, you specify how many replica Pods you want, how Pods should run, which containers should run within these Pods, and which Volumes should be mounted. Based on these templates, controllers maintain the Pod's desired state within a cluster. Controllers are discussed later in the course.

Deployments can also do a lot more than this, which you will see later in the course.

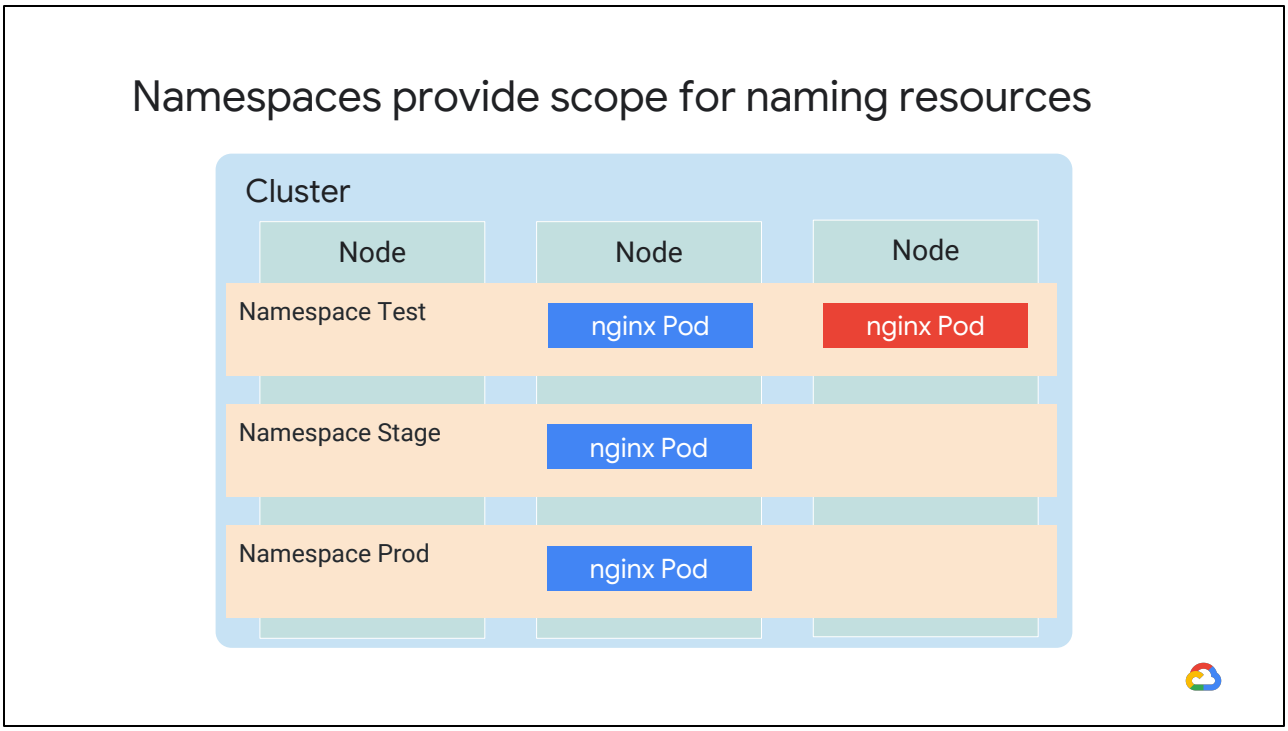
Allocating resource quotas

Multiple projects run on a single cluster

How can I allocate resource quotas?



It's very probable that you'll be using a single cluster for multiple projects. At the same time, it's essential to maintain resource quotas based on projects or teams. By the way: when I say "projects" here, I mean projects in the informal sense of the word: things you and your colleagues are working on. Each Kubernetes cluster is associated with a GCP project, in the formal sense of the word "project", and that's how IAM policies apply to it and how you're billed for it.



So how do you keep everybody's work on your cluster tidy and organized? Kubernetes allows you to abstract a single physical cluster into multiple virtual clusters known as *namespaces*. Namespaces provide scope for naming resources such as Pods, Deployments, and controllers.

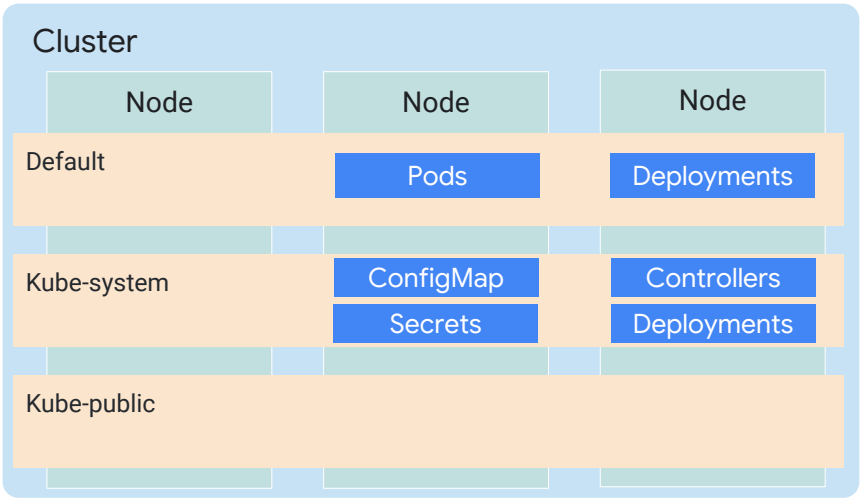
As you can see in this example, there are three namespaces in this cluster: test, stage, and prod.

Remember that you cannot have duplicate object names in the *same* namespace. You can create three Pods with the same name (nginx), but only if they don't share the same namespace. If you attempt to create another Pod with same the name 'nginx Pod' in namespace "test", you won't be allowed. Object names need only be unique within a namespace, not across all namespaces. Namespaces also let you implement resource quotas across the

cluster. These quotas define limits for resource consumption within a namespace. They're not the same as your GCP quotas, which we discussed in an earlier module. These quotas apply specifically to the Kubernetes cluster they're defined on.

You're not require to use namespaces for your day-to-day management; you can also use labels. Still, namespaces are a valuable tool. Suppose you want to spin up a copy of a deployment as a quick test. Doing so in a new namespace makes it easy and free of name collisions.

There are three initial namespaces in a cluster



The first is a default namespace, for objects with no other namespace defined. Your workload resources will use this namespace by default.

Then there is the kube-system namespace for objects created by the Kubernetes system itself. We'll see more of the object kinds in this diagram elsewhere in this specialization. When you use the kubectl command, by default items in the kube-system namespace are excluded, but you can choose to view its contents explicitly.

The third namespace is the kube-public namespace for objects that are publicly readable to all users. kube-public is a tool for disseminating information to everything running in a cluster. You're not required to use it, but it can come in handy, especially when everything running in a cluster is related to the same goal and needs information in common.

Best practice tip: namespace-neutral YAML

Most flexible:

```
kubectl -n demo apply -f mypod.yaml
```

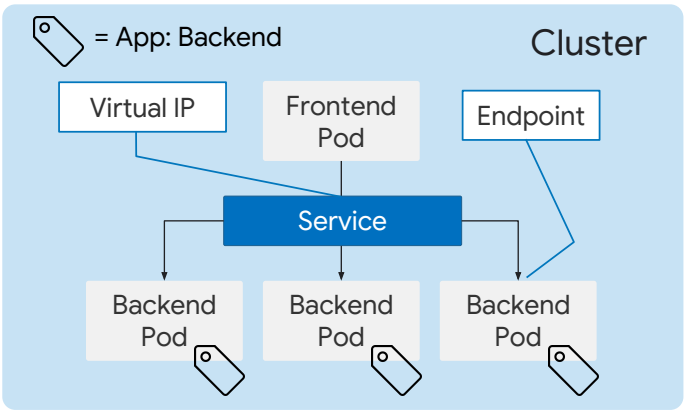
Legal but less flexible:

```
apiVersion: v1
kind: Pod
metadata:
    name: mypod
  namespaces: demo
```



You can apply a resource to a namespace when creating it, using a command-line namespace flag. Or, you can specify a namespace in the YAML file for the resource. Whenever possible, apply namespaces at the command line level. This practice makes your YAML files more flexible. For example, someday you might want to create two identical but completely independent instances of one of your deployments, each in its own namespace. This could be the case if you want to deploy into a separate namespaces for testing before deploying into production. This is difficult if you have chosen to embed namespace names in your YAML files.

Service is an object that directs traffic to pods



Remember that Pods are created and destroyed dynamically. Although Pods can communicate using their assigned Pod IP addresses, these IP addresses are ephemeral; they are not guaranteed to remain constant when Pods are restarted or when scaling changes which nodes are used to run Pods.

Imagine you have two sets of Pods: frontend Pods and backend Pods. How will the frontend Pods discover and keep track of dynamically scaling backend Pods? This is where the concept of Kubernetes Services comes in.

A Kubernetes Service is a static IP address that represents a Service, or a function, in your infrastructure. It's a network abstraction for a set of Pods that deliver that Service, and it hides the ephemeral nature of the IP addresses of the individual Pods. In the example, a set of backend Pods are exposed to the frontend Pod using a Kubernetes Service. Basically, the Service defines a set of Pods and creates a load balancer, of one of a few types, by which those Pods can be accessed.

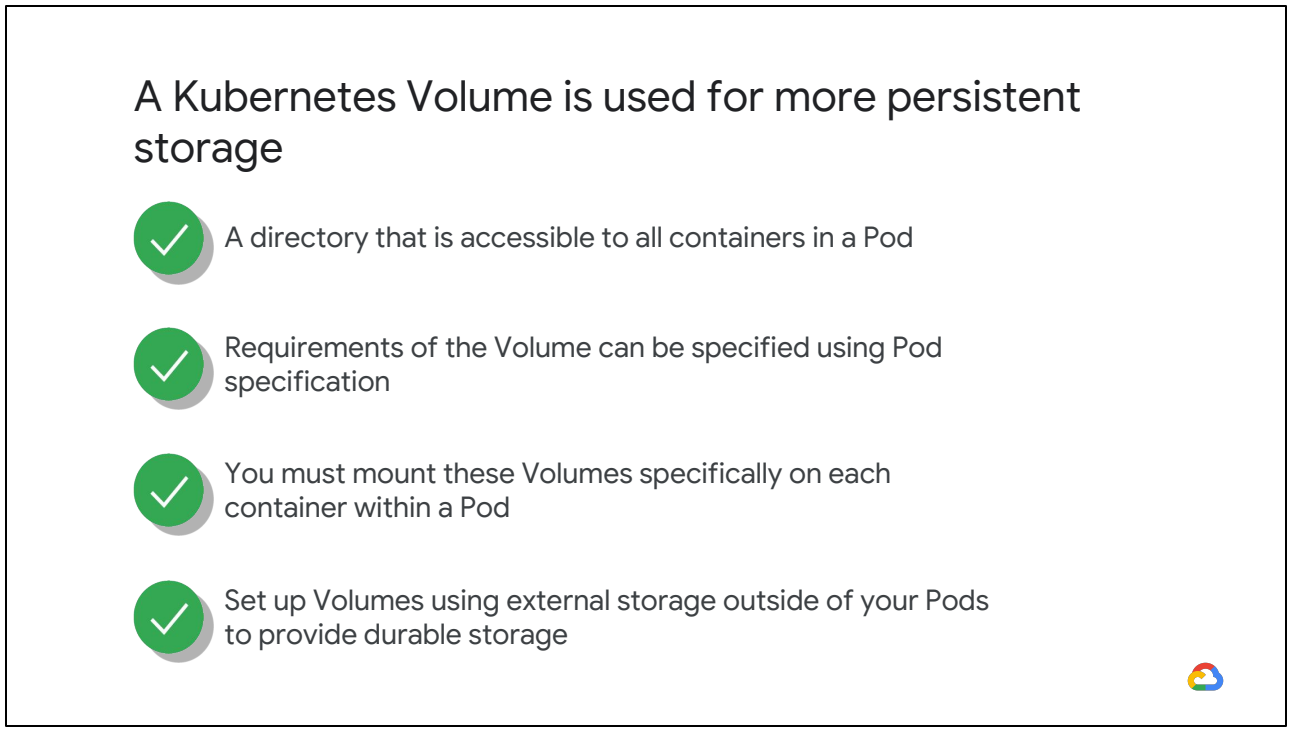
The Pods are selected using a label selector. By the way, you can also get a service quickly by asking Kubernetes to expose a Deployment. When you do that, Kubernetes handles selecting the right pods for you.

Whenever a Service is created, Kubernetes automatically creates endpoints for the selected Pods by creating endpoint resources.

By default, the master assigns a virtual IP address (also known as a ClusterIP) to the Service from internal IP tables. With GKE, this is assigned from the cluster's VPC network.

You will learn more about Services in a later module in this specialization. GKE offers other ways your Service can be exposed, not just through ClusterIPs.

Overall, a Service provides durable endpoints for Pods. These endpoints can be accessed by exposing the Service internally within a cluster, or externally to the outside world. The option to expose a Service internally or externally depends on the Service type itself. The frontend Pod can reliably access the backend Pods internally within the cluster using a Service.



A container application can easily write data to the read/write layer inside the container. But it's ephemeral, so when the container terminates, whatever was written will be lost. What if you want to store data permanently? Or what if you need storage to be shared between tightly coupled containers within a Pod?

That's why a Kubernetes Volume is used for more persistent storage. Kubernetes Volume is another abstraction.

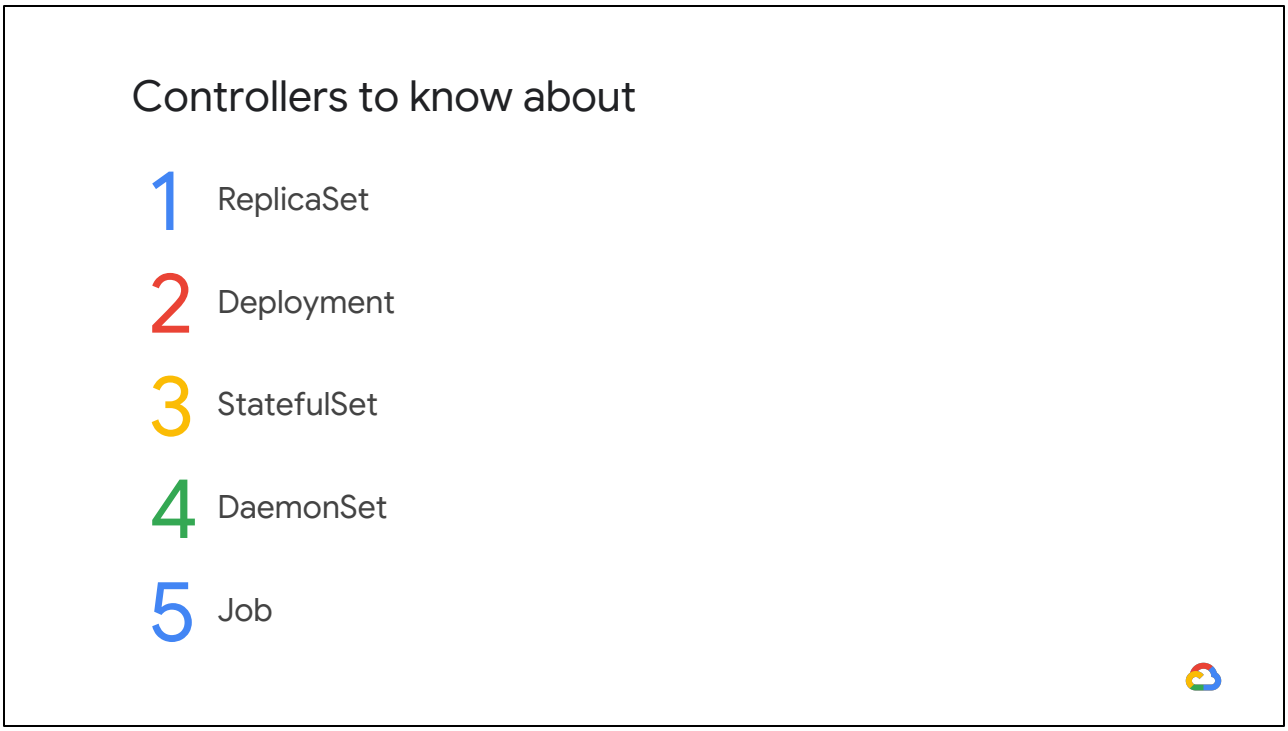
A Volume is simply a directory that is accessible to all the containers in a Pod.

The requirements for a Volume are defined through the Pod specification. This declares how the directory is created, what storage medium should be used, and its initial contents.

You don't want container failures or restarts don't affect the data

within these Volumes. And you want your volume to be shared among multiple containers within a Pod. Docker containers have their own filesystem; therefore, in order to access these Volumes, they must be mounted specifically on each container within a Pod.

However, Pods themselves are also ephemeral. A failing node or deleted Pod could lead to its Volume being deleted too. To avoid this, you can configure Volumes using network based storage from outside of your Pods to provide durable storage that is not lost when a Pod or node fails. You'll learn about Persistent Volumes later in this specialization.



Other advanced Kubernetes objects are discussed in more depth later in the course. Let's look at the Controllers you should be aware of.

A ReplicaSet controller ensures that a population of Pods, all identical to one another, are running at the same time. Deployments let you do declarative updates to ReplicaSets and Pods. In fact, Deployments manage their own ReplicaSets to achieve the declarative goals you prescribe, so you will most commonly work with Deployment objects.

Deployments let you create, update, roll back, and scale Pods, using ReplicaSets as needed to do so. For example, when you perform a rolling upgrade of a Deployment, the Deployment object creates a second ReplicaSet, and then increases the number of Pods in the new ReplicaSet as it decreases the number of Pods in

its original ReplicaSet.

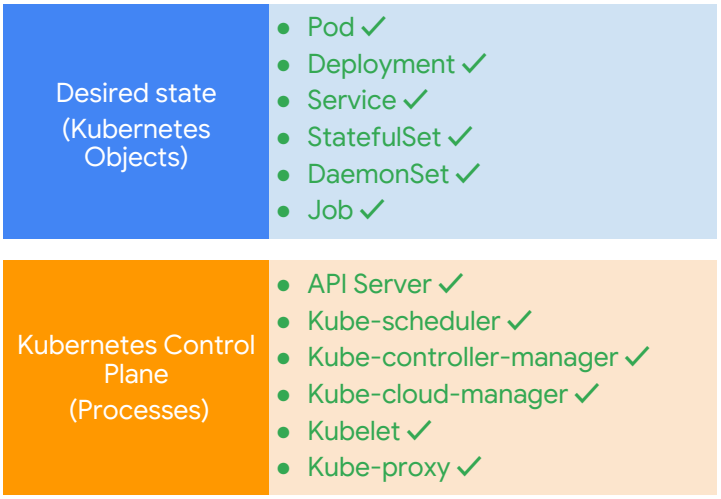
Replication Controllers perform a similar role to the combination of ReplicaSets and Deployments, but their use is no longer recommended. Because Deployments provide a helpful "front end" to ReplicaSets, this training course chiefly focuses on Deployments.

If you need to deploy applications that maintain local state, StatefulSet is a better option. A StatefulSet is similar to a Deployment in that the Pods use the same container spec. The Pods created through Deployment are not given persistent identities, however; by contrast, Pods created using StatefulSet have unique persistent identities with stable network identity and persistent disk storage.

If you need to run certain Pods on all the nodes within the cluster or on a selection of nodes, use DaemonSet. DaemonSet ensures that a specific Pod is always running on all or some subset of the nodes. If new nodes are added, DaemonSet will automatically set up Pods in those nodes with the required specification. The word "daemon" is a computer science term meaning a non-interactive process that provides useful services to other processes. A Kubernetes cluster might use a DaemonSet to ensure that a logging agent like fluentd is running on all nodes in the cluster.

The Job controller creates one or more Pods required to run a task. When the task is completed, Job will then terminate all those Pods. A related controller is CronJob, which runs Pods on a time-based schedule.

Kubernetes architecture recap



That wraps up this lesson. You've learned how the desired state is declared through Kubernetes objects, such as Pods and Deployments. You also saw how different components of the Control Plane work in coordination to achieve the desired state of the cluster.

Lab

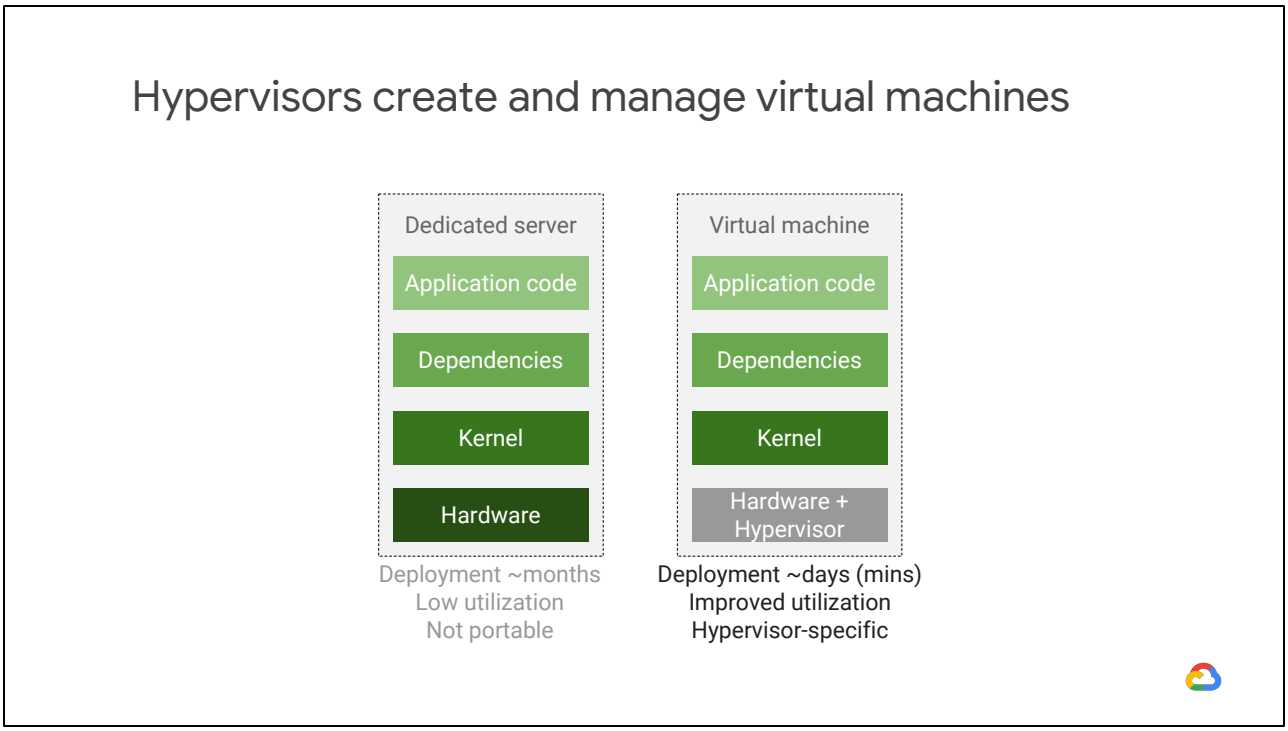
Deploying Google Kubernetes Engine



In this lab, you'll build and use GKE clusters and deploy a sample Pod. The tasks that you'll learn to perform include using the GCP Console to build and manipulate GKE clusters, deploy a Pod, and examine the cluster and Pods.



Welcome to the ‘Introduction to Containers and Kubernetes’ module. In this module you will learn what Containers are, what their benefits are for application deployment, how containers are configured and built, what functions container management solutions like Kubernetes provide, and what the advantages of Google Kubernetes Engine are compared to building your own Container Management infrastructure.



Not very long ago, the default way to deploy an application was on its own physical computer. To set one up, you'd find physical space, power, cooling, network connectivity for it, and then install an operating system, any software dependencies, and finally the application. If you need more processing power, redundancy, security, or scalability, add more computers. It was very common for each computer to have a single purpose: for example database, web server, or content delivery.

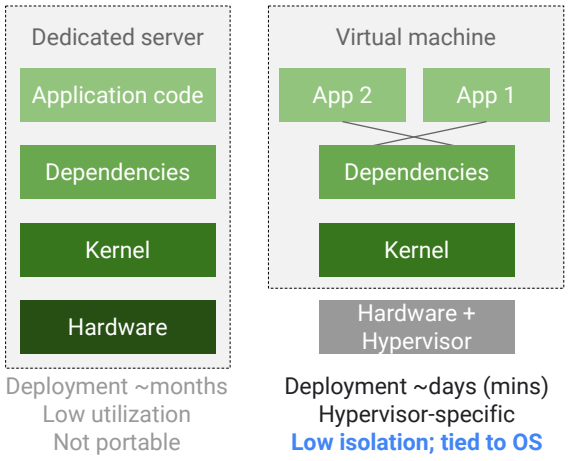
This practice wasted resources and took a lot of time to deploy, maintain, and scale. It also wasn't very portable: applications were built for a specific operating system and sometimes for specific hardware.

Virtualization helped by making it possible to run multiple virtual servers and operating systems on the same physical computer. A hypervisor is the software layer that breaks the dependencies of an operating system on the underlying hardware and allows several virtual machines to share that hardware. KVM is one well-known

hypervisor. Today, you can use virtualization to deploy new servers fairly quickly.

Adopting virtualization means that it takes us less time to deploy new solutions. We waste less of the resources of the physical computers we use. And we get some improved portability, because virtual machines can be imaged and moved around. However, the application, all its dependencies, and operating system are still bundled together. It’s not easy to move a VM from one hypervisor product to another, and every time you start a VM, its operating system takes time to boot up.

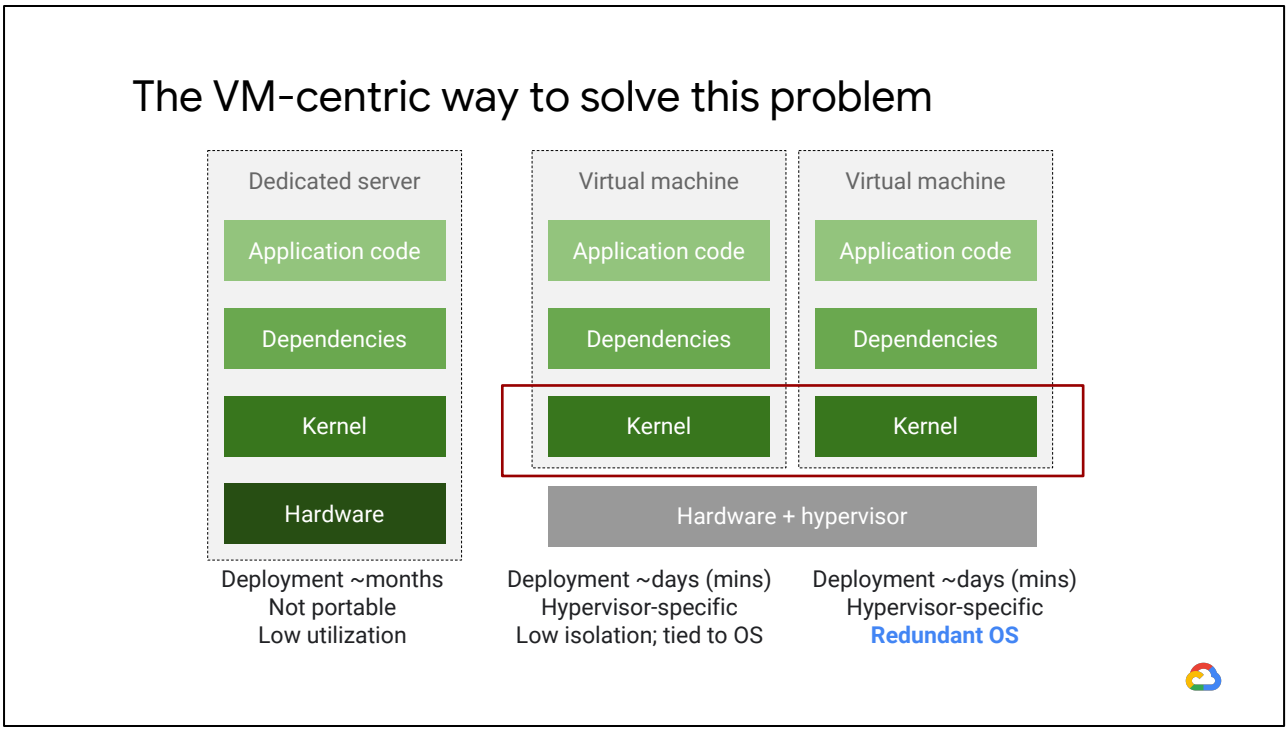
Running multiple apps on a single VM



Running multiple applications within a single VM creates another problem: applications that share dependencies are not isolated from each other. The resource requirements of one application can starve other applications of the resources they need. Also, a dependency upgrade for one application might cause another to stop working.

You can try to solve this problem with rigorous software engineering policies. For example, you can lock down the dependencies so that no application is allowed to make changes; but this leads to new problems because dependencies need to be upgraded occasionally.

You can add integration tests to ensure that applications work. Integration tests are great. But dependency problems can cause novel failure modes that are hard to troubleshoot. And it really slows down development if you have to rely on integration tests to confirm the basic integrity of your application environment.

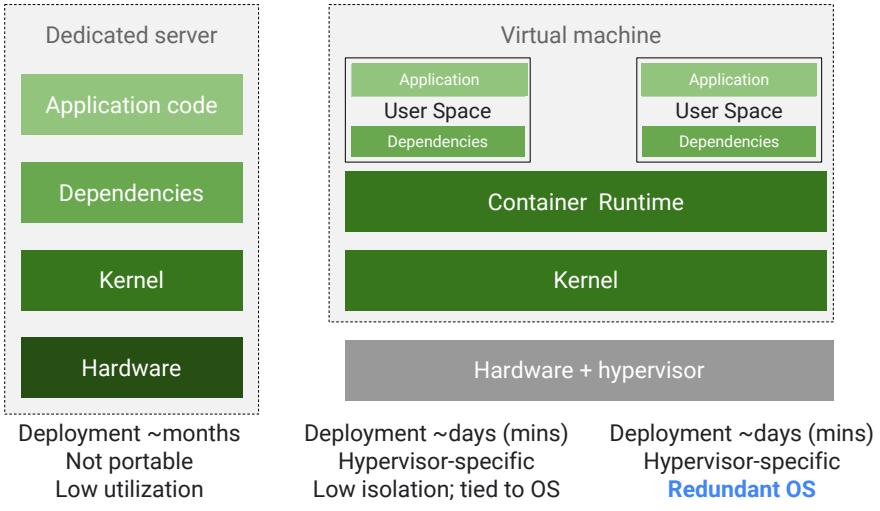


The VM-centric way to solve this problem is to run a dedicated virtual machine for each application.

Each application maintains its own dependencies, and the kernel is isolated so one application won't affect the performance of another. The result is that two complete copies of the kernel are running.

Scale this to hundreds or thousands of applications and you see its limitations; imagine trying to do a kernel update. So, for large systems, dedicated VMs are redundant and wasteful. VMs are also relatively slow to start up, because an entire operating system has to boot.

User space abstraction and containers



A more efficient way to resolve the dependency problem is to implement abstraction at the level of the application and its dependencies. You don't have to virtualize the entire machine, or even the entire operating system, but just the user space. The user space is all of the code that resides above the kernel, and it includes applications and their dependencies.

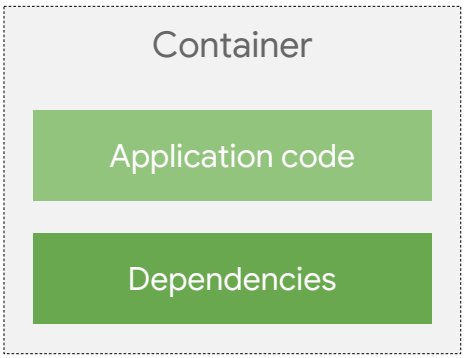
This is what it means to create containers. Containers are isolated user spaces for running application code.

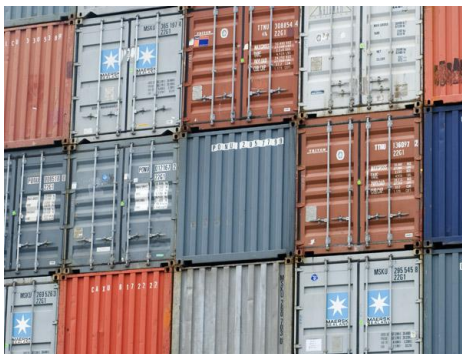
Containers are lightweight because they don't carry a full operating system. They can be scheduled or packed tightly onto the underlying system, which is very efficient. And they can be created and shut down very quickly, because you're just starting and stopping operating system processes, and not booting an entire VM and initializing an operating system for each application.

Developers appreciate this level of abstraction because they don't want to worry about the rest of the system. Containerization is the

next step in the evolution of managing code.

Containers are lightweight, standalone, resource-efficient, portable, executable packages



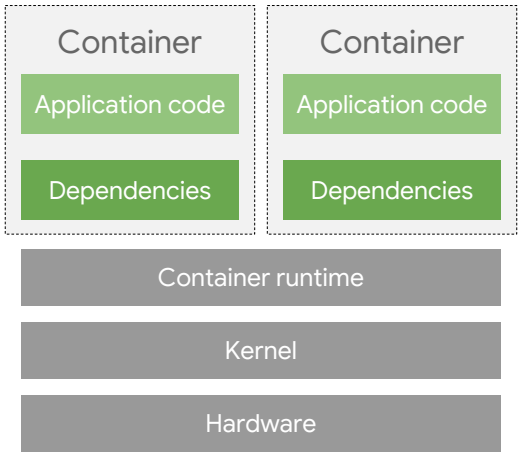


You now understand containers as delivery vehicles for application code. They're lightweight, standalone, resource-efficient, portable, executable packages.

You develop application code in the usual way: on desktops, laptops, and servers. The container allows you to execute your final code on VMs without worrying about software dependencies like application runtimes, system tools, system libraries, and settings. You packaged your code with all the dependencies it needs, and the engine that executes your container is responsible for making them available at runtime.

Image source:
By Steve Gibson hosted at https://commons.wikimedia.org/wiki/File:Shipping_containers_at_Clyde.jpg
Covered by Wikimedia Creative Commons, Attribution Generic 2.0 license https://creativecommons.org/licenses/by/2.0/deed.en

Why developers like containers



Containers appeal to developers because they're an application-centric way to deliver high-performing, scalable applications.

Containers also allow developers to safely make assumptions about the underlying hardware and software.

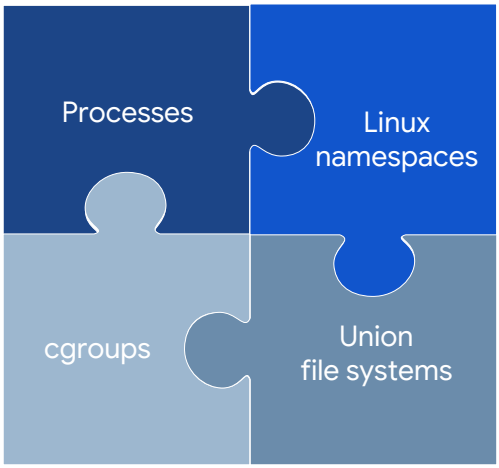
With a Linux kernel underneath, you no longer have code that works on your laptop but doesn't work in production; the container is the same and runs anywhere. If you make incremental changes to a container based on a production image, you can deploy it very quickly with a single file copy. This speeds development.

Finally, containers make it easier to build applications that use the microservices design pattern: that is, loosely coupled, fine-grained components. This modular design pattern allows the operating system to scale and upgrade components of an application without affecting the application as a whole.

An application and its dependencies are called an *image*. A container is simply a running instance of an image. By building software into container images, developers can easily package and ship an application without worrying about the system it will be running on.

You need software to build container images and to run them. Docker is one tool that does both. Docker is an open-source technology that allows you to create and run applications in containers, but it doesn't offer a way to orchestrate those applications at scale as Kubernetes does. In this course, we will use Google's Cloud Build to create Docker-formatted container images.

Containers use a varied set of Linux technologies



Containers are not an intrinsic, primitive feature of Linux. Instead, their power to isolate workloads is derived from the composition of several technologies.

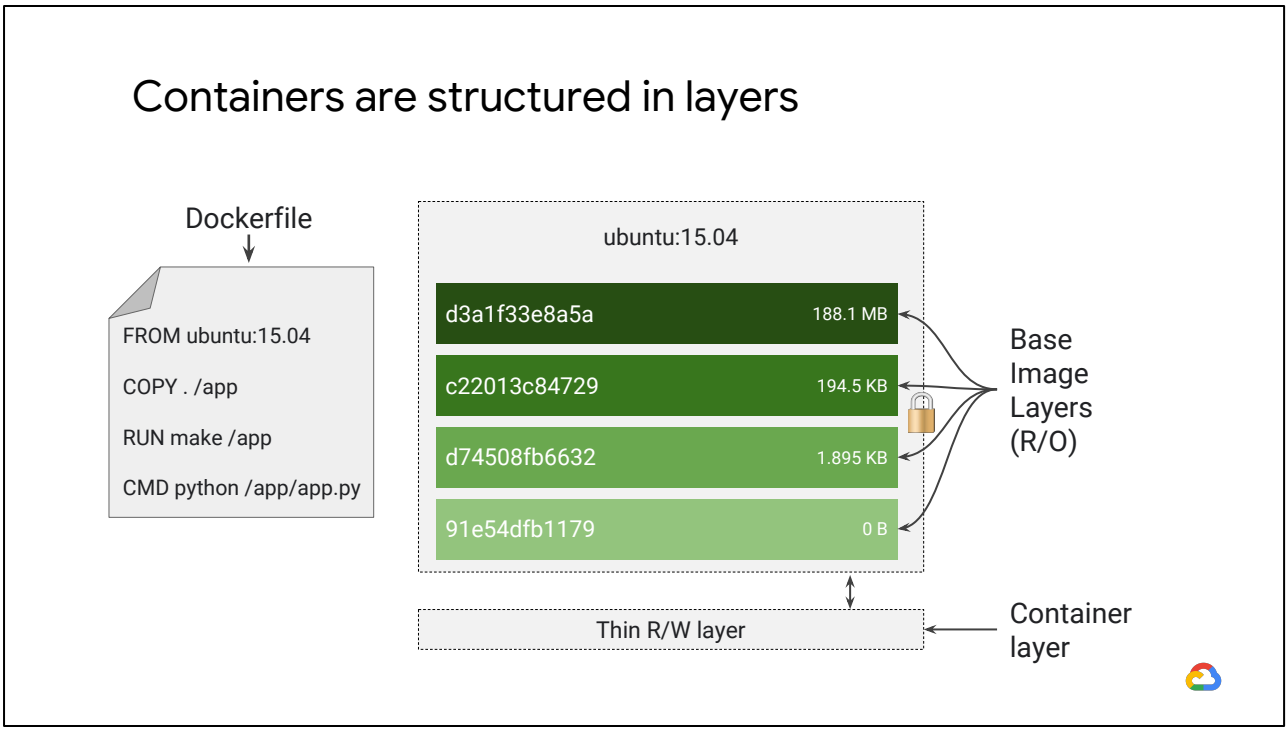
One foundation is the Linux process. Each Linux process has its own virtual memory address space, separate from all others, and Linux processes are rapidly created and destroyed.

Containers use Linux namespaces to control what an application can see: process ID numbers, directory trees, IP addresses, and more. By the way, Linux namespaces are not the same thing as Kubernetes namespaces, which you will learn about later in this course.

Containers use Linux cgroups to control what an application can use: its maximum consumption of CPU time, memory, I/O bandwidth, and other resources.

Finally, containers use union file systems to efficiently encapsulate

applications and their dependencies into a set of clean, minimal layers. Now let’s see how this works.



A container image is structured in layers. The tool you use to build the image reads instructions from a file called the “container manifest.” In the case of Docker-formatted container images, that’s called a Dockerfile. Each instruction in the Dockerfile specifies a layer inside the container image. Each layer is read-only. (When a container runs from this image, it will also have a writable, ephemeral topmost layer.)

Let’s look at a simple Dockerfile. This Dockerfile will contain four commands, each of which creates a layer. (At the end of this discussion, I’ll explain why this Dockerfile is a little oversimplified for modern use.)

The FROM statement starts out by creating a base layer, pulled from a public repository. This one happens to be the Ubuntu Linux runtime environment of a specific version.
The COPY command adds a new layer, containing some files copied in from your build tool’s current directory.
The RUN command builds your application using the “make”

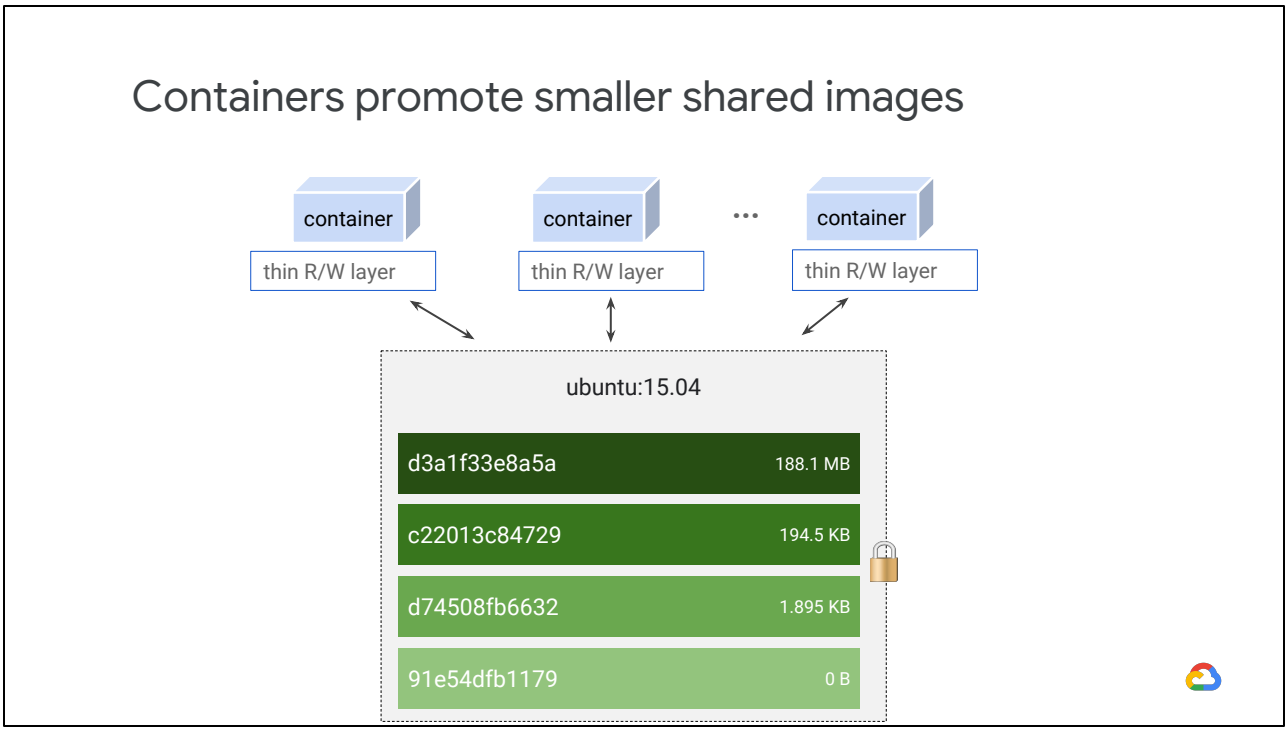
command and puts the results of the build into a third layer.

And finally, the last layer specifies what command to run within the container when it is launched. Each layer is only a set of differences from the layer before it. When you write a Dockerfile, you should organize from the layers likely to change, through to the layers most likely to change.

By the way, I promised that I would explain how this Dockerfile example is oversimplified. These days, the best practice is *not* to build your application in the very same container that you ship and run. After all, your build tools are at best just clutter in a deployed container, and at worst they are an additional attack surface. Today, application packaging relies on a multi-stage build process, in which one container *builds* the final executable image, and a separate container receives only what is needed to *run* the application. Fortunately for us, the tools we use support this practice.

When you launch a new container from an image, the container runtime adds a new writable layer on top of the underlying layers. This layer is often called the *container* layer.

All changes made to the running container, such as writing new files, modifying existing files, and deleting files, are written to this thin writable container layer. And they're ephemeral: When the container is deleted, the contents of this writable layer are lost forever. The underlying container image remains unchanged. This fact about containers has an implication for your application design: whenever you want to store data permanently, you must do so somewhere *other than* a running container image. You will learn about several choices in this specialization.



Because each container has its own writable container layer, and all changes are stored in this layer, multiple containers can share access to the same underlying image and yet have their own data state. The diagram shows multiple containers sharing the same Ubuntu 15.04 image.

Because each layer is only a set of differences from the layer before it, you get smaller images.

For example, your base application image may be 200 MB, but the difference to the next point release might only be 200 KB. When you build a container, instead of copying the whole image, it creates a layer with just the difference. When you run a container, the container runtime pulls down the layers it needs. When you update, you only need to copy the difference. This is much faster than running a new virtual machine.

How can you get or create containers?



Download containerized software from a container registry such as gcr.io.

`docker`

Build your own container using the open-source docker command.



Build your own container using Cloud Build.



It's very common to use publicly available open-source container images as the base for your own images, or for unmodified use. For example, you've already seen the "ubuntu" container image, which provides a Ubuntu Linux environment inside a container. "Alpine" is a popular Linux environment in a container, noted for being very small. The nginx web server is frequently used in its container packaging.

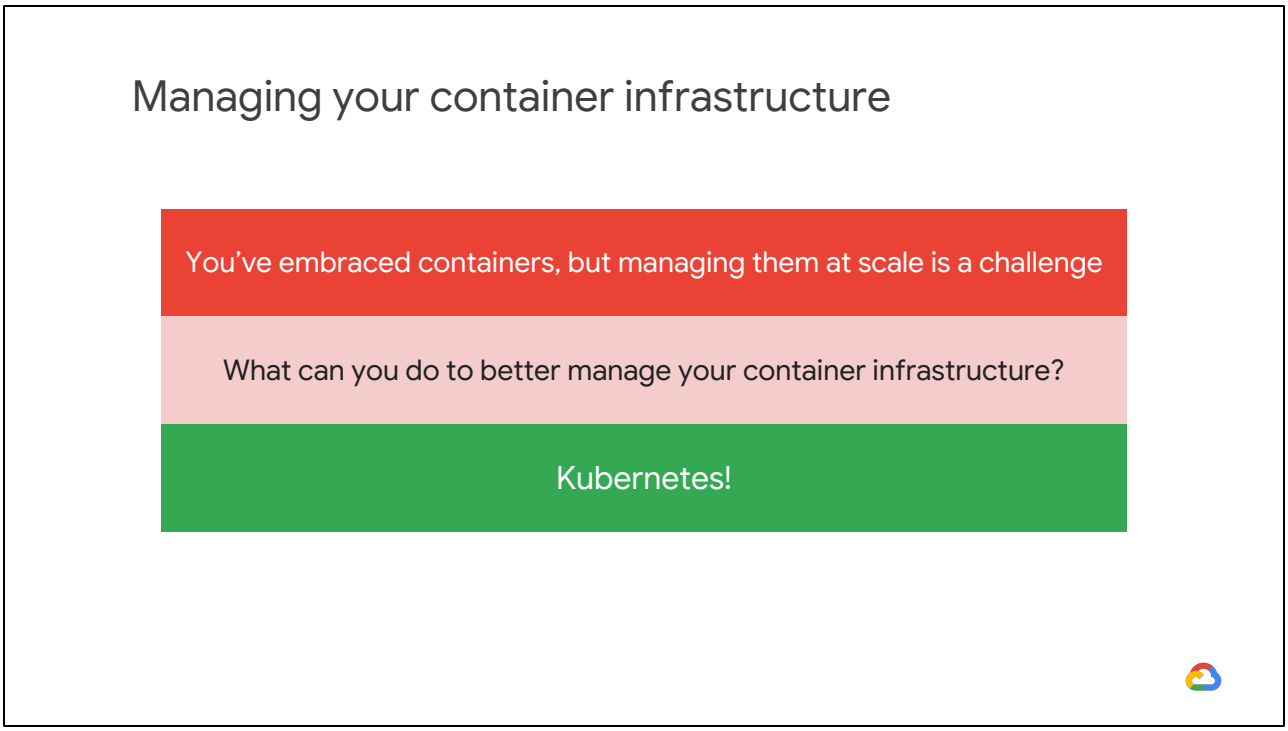
Google maintains a container registry, gcr.io. This registry contains many public, open-source images, and Google Cloud customers also use it to store their private images in a way that integrates with Cloud IAM. Google Container Registry is integrated with Cloud IAM, so, for example, you can use it to store images that aren't public -- instead, they are private to your project.

You can also find container images in other public repositories: Docker Hub Registry, GitLab, and others.

The open-source docker command is a popular way to build your

own container images. It's widely known and widely available.

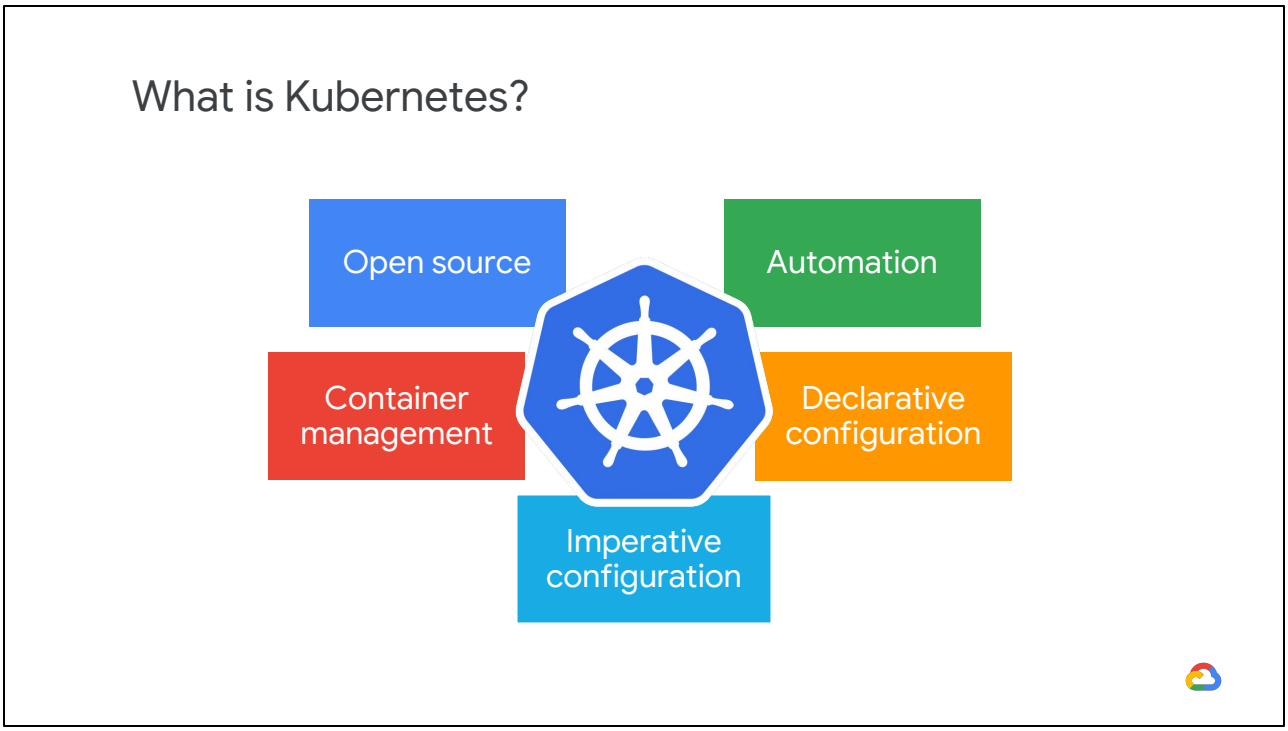
One downside of building containers with the docker command is that you must trust the computer that you do your builds on. Google provides a managed service for building containers that's integrated with Cloud IAM. This service is called Cloud Build, and we'll use it in this course.



Your organization has really embraced the idea of containers. Because containers are so lean, your coworkers are creating them in numbers far exceeding the counts of virtual machines you used to have. And the applications running in them need to communicate over the network, but you don't have a network fabric that lets containers find each other. You need help.

How can you manage your container infrastructure better?

Kubernetes is an open-source platform that helps you orchestrate and manage your container infrastructure on-premises or in the cloud.



So what is Kubernetes? It's a container-centric management environment. Google originated it and donated it to the open-source community. Now it's a project of the vendor-neutral Cloud Native Computing Foundation.

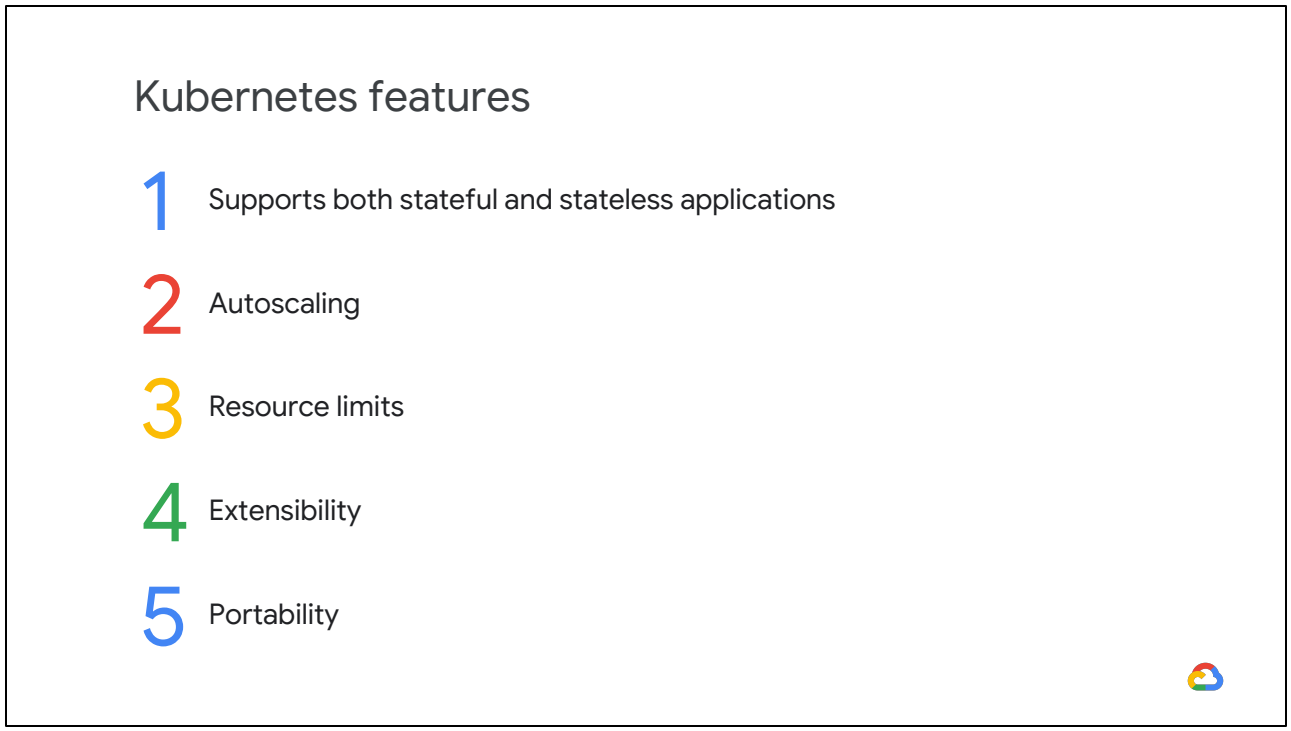
It automates the deployment, scaling, load balancing, logging, monitoring, and other management features of containerized applications. These are the features that are characteristic of typical platform-as-a-service solutions.

Kubernetes also facilitates the features of infrastructure-as-a-service, such as allowing a wide range of user preferences and configuration flexibility.

Kubernetes supports declarative configurations. When you administer your infrastructure declaratively, you describe the

desired state you want to achieve, instead of issuing a series of commands to achieve that desired state. Kubernetes's job is to make the deployed system conform to your desired state and to keep it there in spite of failures. Declarative configuration saves you work. Because the system's desired state is always documented, it also reduces the risk of error.

Kubernetes also allows imperative configuration, in which you issue commands to change the system's state. But administering Kubernetes at scale imperatively would be a big missed opportunity. One of the primary strengths of Kubernetes is its ability to automatically keep a system in a state you declare. Experienced Kubernetes administrators use imperative configuration only for quick temporary fixes and as a tool in building a declarative configuration.



Now that you know what Kubernetes is, let's talk about some of its features.

Kubernetes supports different workload types. It supports stateless applications, such as Nginx or Apache web servers, and stateful applications where user and session data can be stored persistently. It also supports batch jobs and daemon tasks.

Kubernetes can automatically scale in and out containerized applications based on resource utilization.

You can specify resource request levels and resource limits for your workloads, and Kubernetes will obey them. These resource controls let Kubernetes improve overall workload performance within a cluster.

Developers extend Kubernetes through a rich ecosystem of plugins and addons. For example, there is a lot of creativity going on currently with Kubernetes Custom Resource Definitions, which bring the Kubernetes declarative management model to an amazing variety of other things that need to be managed. The primary focus of this specialization, though, is architecting with Kubernetes, because it's provided as a service by Google Cloud, so extending Kubernetes is not in our scope.

Because it's open-source, Kubernetes also supports workload portability across on-premises or multiple cloud service providers such as GCP and others. This allows Kubernetes to be deployed anywhere. You can move Kubernetes workloads freely without vendor lock-in.

Managing Kubernetes within GCP

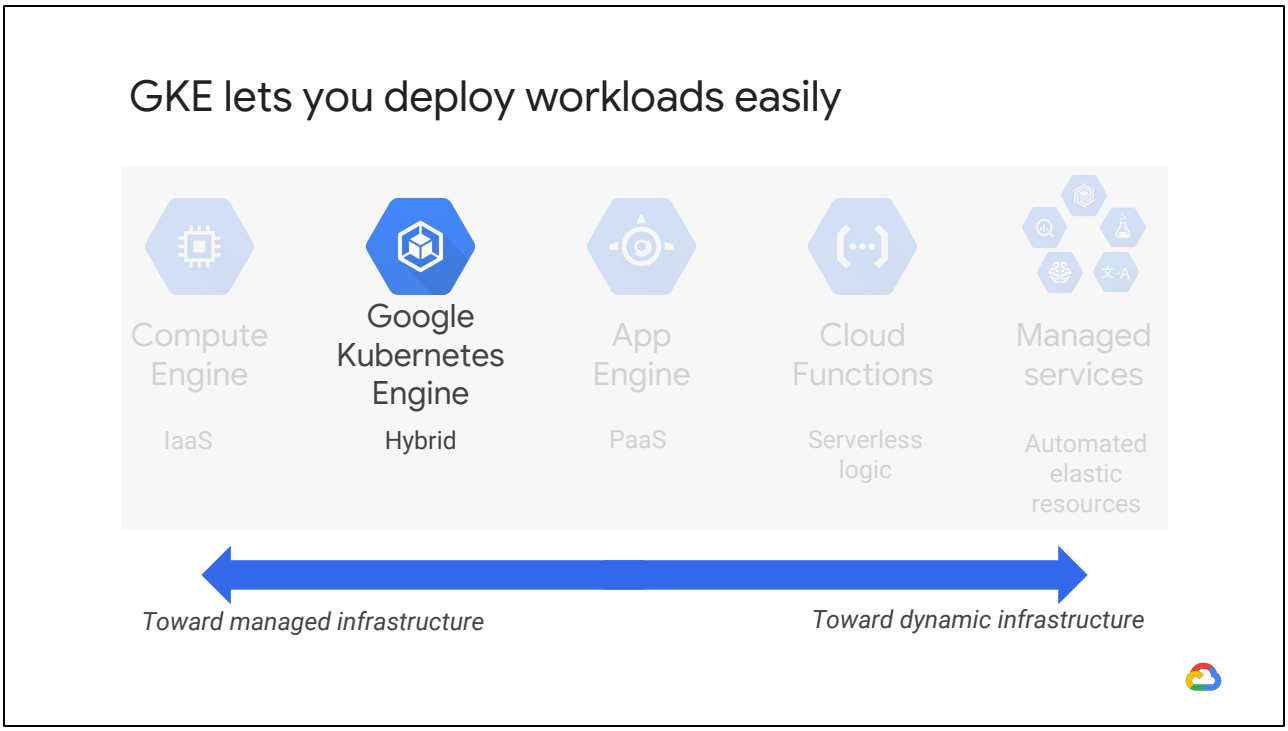
Kubernetes is powerful, but it's a full-time job managing the infrastructure

Is there a managed service for Kubernetes within GCP?

Yes! Google Kubernetes Engine

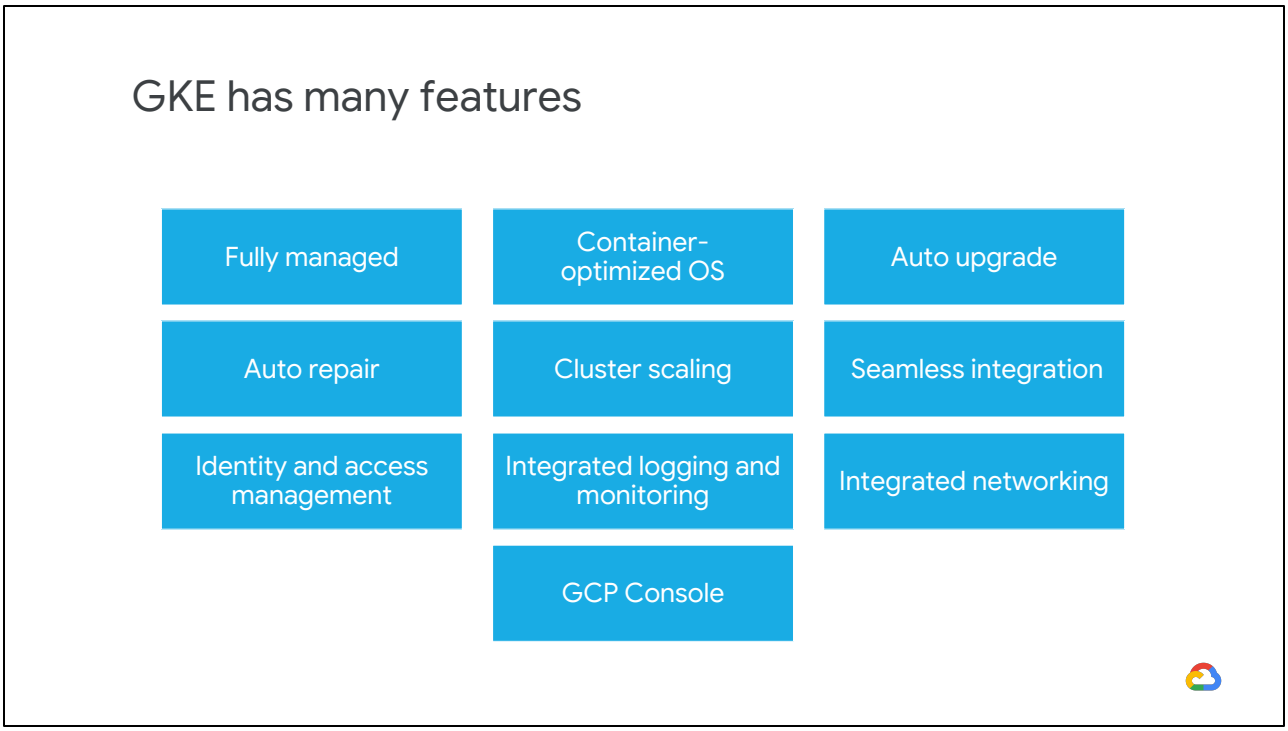


What if you have begun using Kubernetes in your environment, but the infrastructure has become a burden to maintain?
Is there anything within Google Cloud Platform that can help you?
Absolutely yes. GCP offers a managed Kubernetes solution called Google Kubernetes Engine.



Google Kubernetes Engine is a managed Kubernetes service on Google infrastructure. GKE helps you to deploy, manage, and scale Kubernetes environments for your containerized applications on GCP.

More specifically, GKE is a component of the GCP compute offerings. It makes it easy to bring your Kubernetes workloads into the cloud.



GKE is fully managed, which means you don't have to provision the underlying resources.

GKE uses a container-optimized operating system to run your workloads. These operating systems are maintained by Google and are optimized to scale quickly with a minimal resource footprint. The container-optimized OS is discussed later in this course.

When you use GKE, you start by directing the service to instantiate a Kubernetes system for you. This system is called a cluster. GKE's auto-upgrade feature can be enabled to ensure that your clusters are always automatically upgraded with the latest stable version of Kubernetes.

The virtual machines that host your containers in a GKE cluster are called nodes. If you enable GKE's auto-repair feature, the service

will repair unhealthy nodes for you. It'll make periodic health checks on each node of the cluster. If a node is determined to be unhealthy and require repair, GKE will drain the node (in other words, cause its workloads to gracefully exit) and recreate the node.

Just as Kubernetes supports scaling workloads, GKE supports scaling the cluster itself.

GKE seamlessly integrates with Google's Cloud Build and Container Registry. **This allows you to automate deployment using private container images that you have securely stored in Container Registry.**

GKE also integrates with Google's Identity and Access Management, which allows you to control access through the use of accounts and role permissions.

Stackdriver is Google Cloud's system for monitoring and management for services, containers, applications, and infrastructure. GKE integrates with Stackdriver Monitoring to help you understand your applications' performance.

GKE is integrated with Google Virtual Private Clouds and makes use of GCP's networking features.

And finally the GCP Console provides insights into GKE clusters and their resources and allows you to view, inspect and delete resources in those clusters. You might be aware that open-source Kubernetes contains a dashboard, but it takes a lot of work to set it up securely. But the GCP Console is a dashboard for your GKE clusters and workloads that you don't have to manage, and it's